

ROS

Robot Operating System

- Part 1 -

Cheolmin Jeong

강사 소개



■ 정철민 (Cheolmin Jeong)

■ 現 한양대학교 전기공학과 박사과정

■ 現 Unicore Robotics CTO

■ 한양대학교 지능형시스템제어 연구실 소속

(지도교수: 강창묵 교수, <https://uniconlab.wixsite.com/main>)

■ 관심분야: 최적제어, 선형제어, 예측제어, 고장진단, 자율주행, SLAM

■ 적용플랫폼: 자율주행차량, 모바일로봇, 드론, 4족보행로봇 등

■ Projects:

- 자율주행 전동휠체어 핵심 기술 개발
- 순찰로봇의 도심지 적용을 위한 실외 자율주행 기술 개발
- 인천공항 폐기물 이송 로봇 개발
- UAV 기체 상태 이상 진단 검증 및 연계 모듈 개발
- 다목적 개인이동형 서비스 로봇 플랫폼 기술개발 및 실증
- 자율주행 전동휠체어 의료기기인증 및 표준안을 위한 기능 구현
- 온디바이스 AI 컴퓨팅 기반 AI 융합형 모빌리티 디바이스 기술 개발 및 실증
- ...



Contents

1. **지능형 자율주행**
 2. **로봇 자율주행 기술**
 3. **Linux**
 4. **ROS**
 5. **ROS Basic**
 6. **ROS Programming**
- Appendix**

지능형 자율주행

1. 지능형 자율주행

1) 자율주행 자동차(Autonomous Vehicle)란?

- 자율주행 자동차는 운전자 없이도 스스로 상황을 판단하고 주행하는 자동차를 의미
- 자율주행 자동차는 출발지에서 목적지까지 이동하는 경로를 스스로 결정하여 주행함
- 경로상에는 건물, 가로수, 표지판과 같은 정적인 객체들과 주행중인 차량, 보행자 등의 동적인 객체가 존재
- 주변 객체들을 정확하게 인지하여 안전한 주행을 위한 적절한 대응과 최적의 이동 경로를 탐색하기 위한 판단 과정을 수행함
- 주행 과정에서 신호등, 교통 표지판과 같은 교통 법규와 규칙들을 반드시 준수해야 함

1. 지능형 자율주행

자율주행 기술의 단계



현재 기술 수준



1. 지능형 자율주행

SAE 자율주행 단계(Levels of Driving Automation)

- 국제자동차기술자협회(SAE International)에서는 운전 자동화에 대한 자율주행 단계를 총 6단계로 구분
- 자율주행 기능이 없는 비자동화 단계부터 완전 자율주행이 가능한 수준까지의 운전 자동화를 포함
- 가장 낮은 단계인 **레벨 0(No Automation)**은 운전과 관련한 모든 조작과 모니터링 활동을 시스템의 개입 없이 운전자가 직접 통제하는 수준임
- **레벨 1(Driver Assistance)**은 차선 위치를 유지하기 위한 조향 제어, 속도 조절을 위한 가·감속 제어와 같은 운전 보조 기능이 추가된 수준을 의미함
- **레벨 2(Partial Automation)**는 자동차가 스스로 주행하는 첫 단계로, 고속도로와 같은 제한적인 상황에서 자율주행 시스템이 차량의 속도와 차선 위치 유지를 위한 제어를 담당함

1. 지능형 자율주행

SAE 자율주행 단계(Levels of Driving Automation)

- **레벨 3(Conditional Automation)** 단계부터는 운전의 주체가 운전자에서 자율주행 시스템으로 변경
- **레벨 3(Conditional Automation)**는 특정 조건에서는 자율주행 시스템이 운전을 완전하게 통제하며, 운전자는 지속적으로 모니터링할 필요가 없지만 시스템의 요청이나 경고에 맞춰 대응할 수 있어야 함
- **레벨 4(High Automation)**는 일정하게 정의된 범위와 환경에서 모든 동작을 자율주행 시스템이 담당하여, 운전자는 모니터링하거나 대기할 필요가 없는 수준
- **레벨 5(Full Automation)**는 운전에 관련한 모든 범위의 작업을 시스템이 완전히 통제하고 책임지는 완전 자율주행의 단계를 의미

1. 지능형 자율주행

대표적인 자율주행 기업



자료 출처. <https://www.autoherald.co.kr/news/articleView.html?idxno=33375>

정밀지도와 LiDAR 센서를 기반으로 제한된
영역에서의 완전 자율주행(Level 4)을 지향



자료 출처. <https://topictree.co.kr/newcar/tesla-new-roadster/>

기존에 판매한 수 십만대의 차량에서 업로드한 영상
정보를 바탕으로 운전자 보조 기능(Level 2) 고도화 지향

	Google Waymo	TESLA
핵심 기술	LiDAR + HD Map	Camera(Vision) + Deep Learning
기능	Replace driver	Support driver

1. 지능형 자율주행

자율주행 참고 영상 - Google Waymo

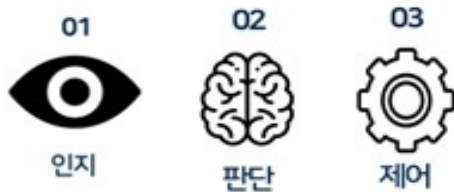


한번 직접 불러 봤습니다

자료 출처. <https://www.youtube.com/watch?v=U3wA3QM1pxc>

1. 지능형 자율주행

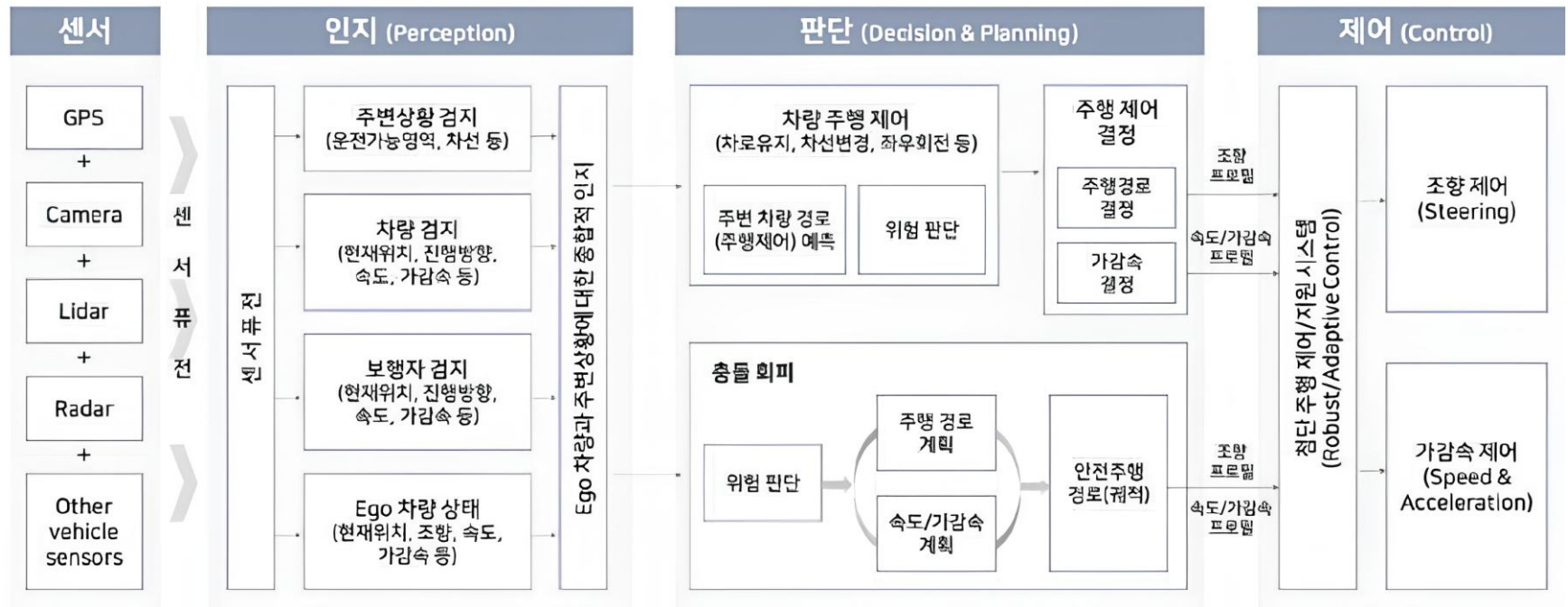
2) 자율주행의 3 요소



- 인지(Perception): “**인지**”는 자율주행 시스템에서 차량 주변 환경을 감지하고 이해하는 역할.
센서(Camera, Radar, LiDAR)와 알고리즘을 통해 도로의 상태, 교통 표지판, 다른 차량, 보행자 등을 식별하고 그 위치와 속도를 파악함.
- 판단(Decision): “**판단**”은 인지 과정에서 수집된 정보를 바탕으로 차량의 행동을 결정함.
이는 경로 계획, 장애물 회피 전략 수립, 안전하고 법규를 준수하는 운전 결정을 포함.
해당 단계에서는 다양한 시나리오와 예상되는 결과를 평가하여 최적의 운전 계획 결정.
- 제어(Control): “**제어**”는 결정된 운전 계획에 따라 차량의 스티어링, 가속, 제동 등을 조절.
해당 과정을 매우 정밀하게 이루어져야 하며, 실시간으로 차량의 동적 상태를 조정하여 안정적이고 효율적으로 차량이 운행되도록 함.

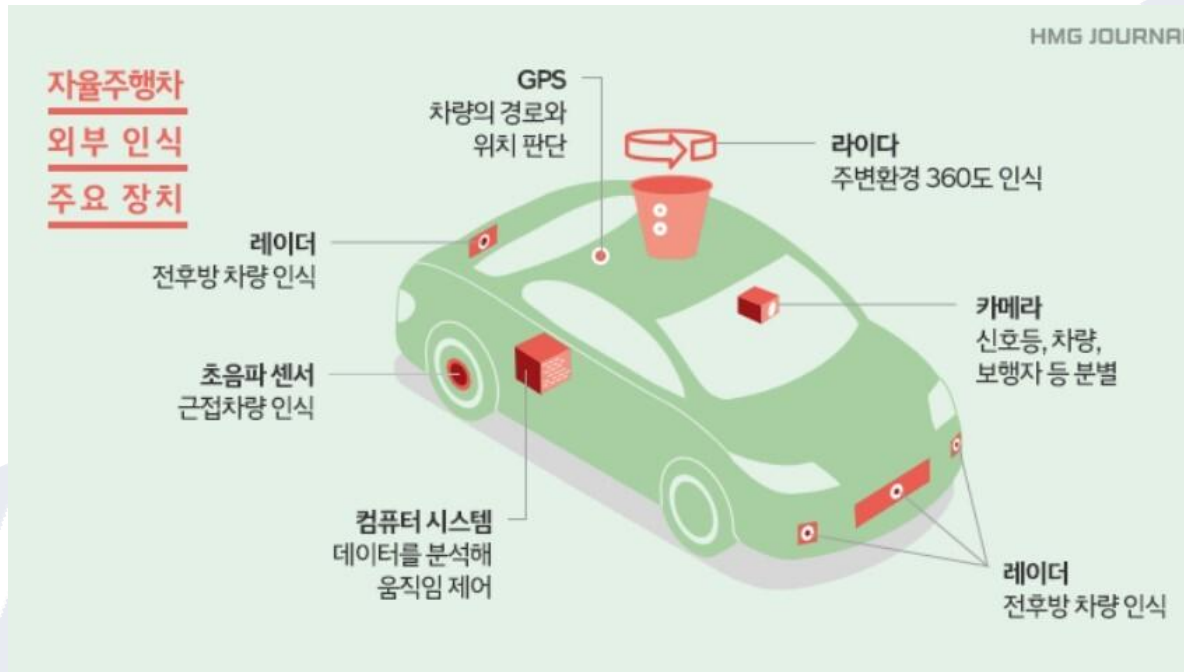
1. 지능형 자율주행

2) 자율주행의 3 요소



1. 지능형 자율주행

3) 자율주행을 위한 센서



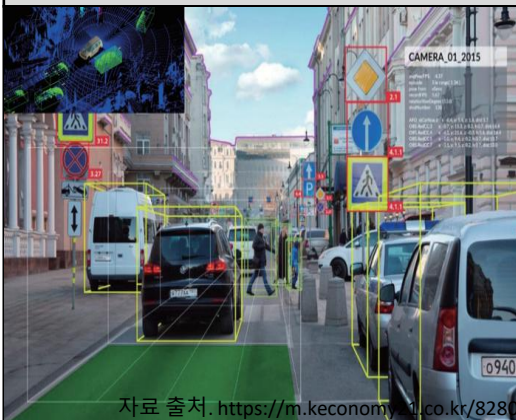
- 자율주행 자동차의 안전한 주행을 위해서는 주변 환경을 인지해 주행에 필요한 정보를 제공하는 센서 시스템이 필수적
- 자율주행 자동차의 센서 시스템은 개별 센서의 기능, 비용, 디자인을 비롯한 여러 요인을 고려해 최적으로 설계
- 각 센서의 장점과 단점을 상호 보완할 수 있도록 여러 센서 데이터를 조합하는 센서 퓨전(Sensor fusion) 기술은 자율주행 시스템의 신뢰성과 안정성을 높이는데 매우 중요

1. 지능형 자율주행

3) 자율주행을 위한 센서

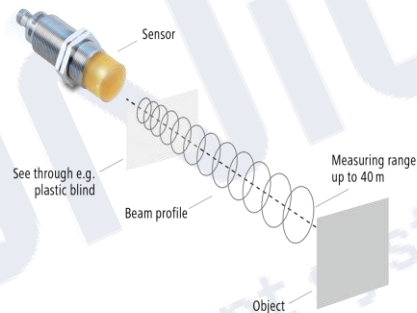
● 인지(Perception)

Camera



- 객체 인지 및 위치 추정에 활용
- 텍스처 정보를 포함한 영상 정보 제공
- 30~60 Hz의 빠른 측정 주기
- HD급 카메라를 다수 장착해 FOV 확보
- 고해상도 데이터, 저렴한 비용
- 조명 환경 변화에 취약, 정확한 깊이 추정 불가

Radar



자료 출처: <https://www.baumer.com/>

- 장애물 회피 및 충돌 감지에 활용
- 전자기파를 이용해 측정된 거리, 속도 정보 제공
- 근거리와 원거리에 대해 모두 측정 가능
- 눈, 비, 조명 등 외부 환경에 강건
- 작은 물체 측정에 취약, 낮은 해상도

LiDAR



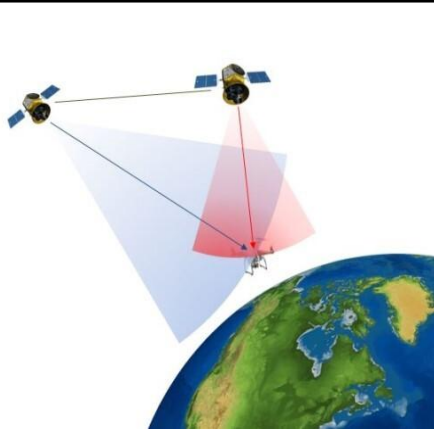
- 객체 인지 및 위치 추정에 활용
- 레이저를 이용해 고해상도의 정확한 3차원 거리 정보 제공
- 10~20 Hz의 주기
- 대상체에 대한 반사도 정보 제공
- 조명 환경 변화에 강건
- 눈, 비, 안개 등 악천후에 민감
- 높은 비용, 제품 양산 및 상용화

1. 지능형 자율주행

3) 자율주행을 위한 센서

● 위치 추정(Localization)

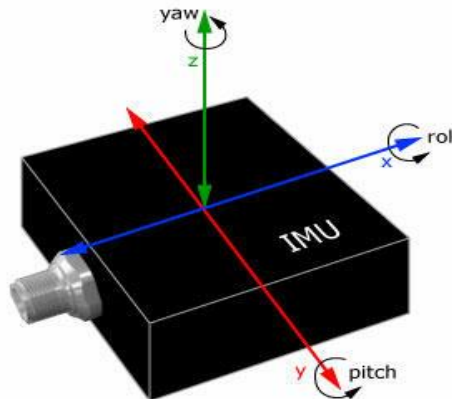
GPS



자료 출처. <https://brunch.co.kr/@matthewmin/122>

- 차량의 위치 추정에 필요
- 위성 신호를 이용해 지구좌표계에 따른 절대 위치 정보 제공 (위도, 경도, 고도)
- 1~10Hz 수준의 느린 측정 주기
- 지하/터널/도심 등에서 음영 지역 발생
- 일반적인 GPS는 수 m 오차로 자율주행 용으로는 부정확함
- DGPS, GPS-RTK는 수십 cm 내 정확도

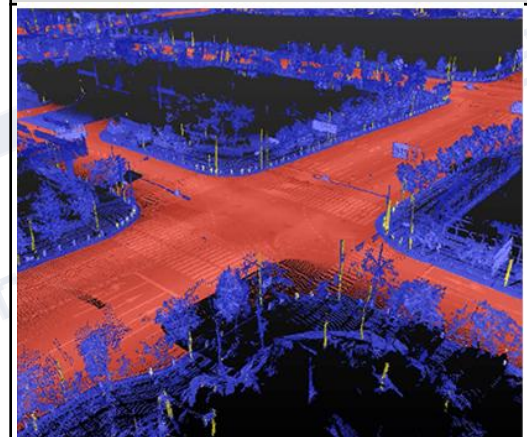
IMU



자료 출처. <https://www.usgs.gov/centers/pcmsc/science/inertial-measurement-unit-imu>

- 차량 움직임, 위치 추정에 필요
- 6축 또는 9축 관성(가속도, 각속도, 지자기) 정보를 제공
- 200 Hz 이상의 빠른 측정 주기
- 추측 방법을 통해 차량의 이동 거리 및 방향 추정 가능
- 외부 환경에 강건하나, 시간에 따른 누적 오차 발생

HD Map

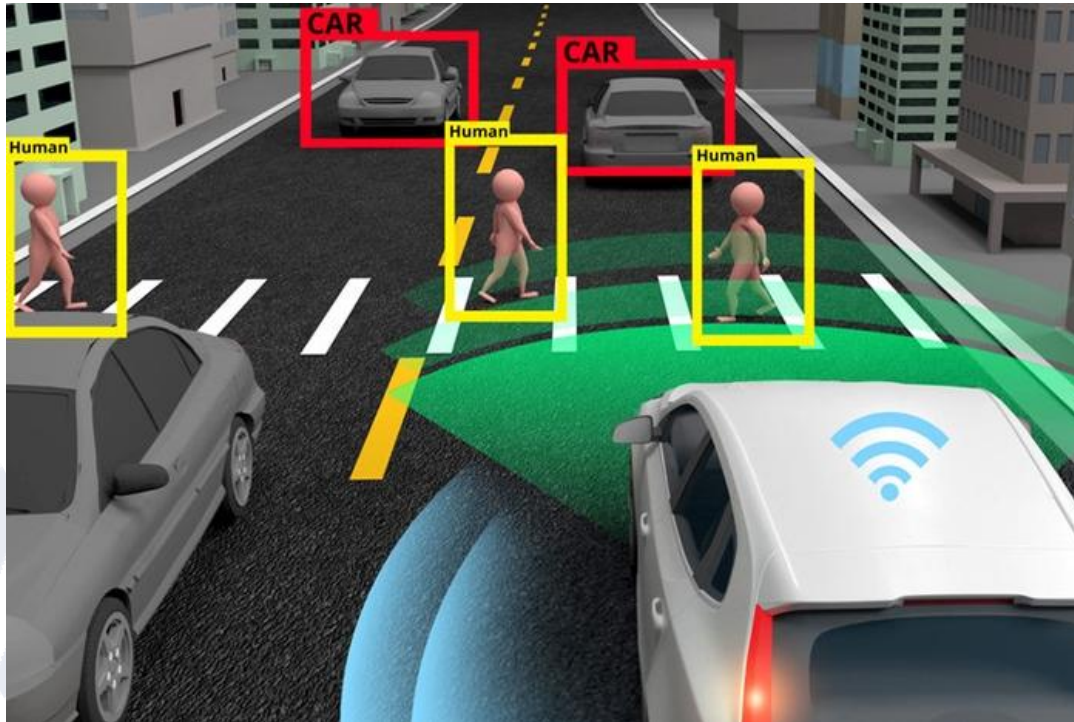


자료 출처. <https://simkkong.tistory.com/14>

- 자율주행 차량을 위한 다양한 도로 정보가 정밀하게 기록되어 있는 지도
- 위치 추정, 경로 계획, 객체 인지에 모두 사용
- 도로 규칙 정보, 구조물 형상 정보, 실시간 교통 정보, 센서 데이터 등
- 구축에 많은 비용이 들고, 지속적인 업데이트가 필요

1. 지능형 자율주행

- 인지 (Perception)

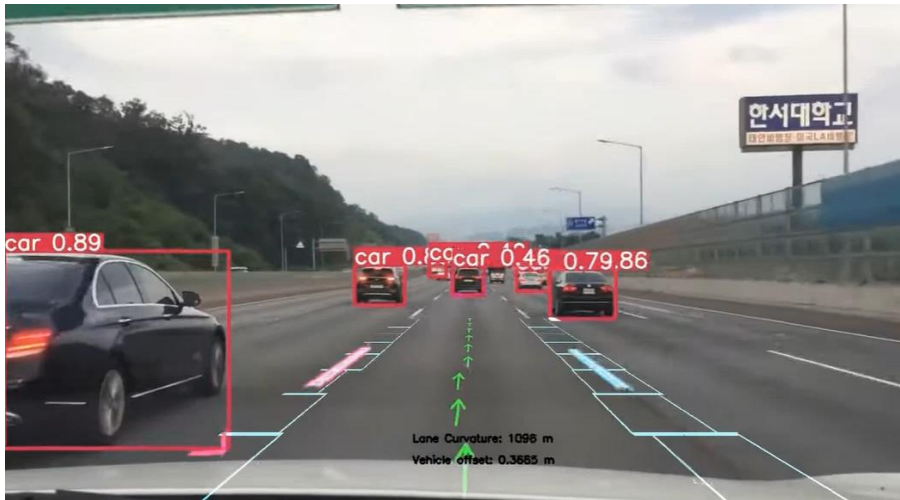


자료 출처. <https://kidd.co.kr/news/210655>

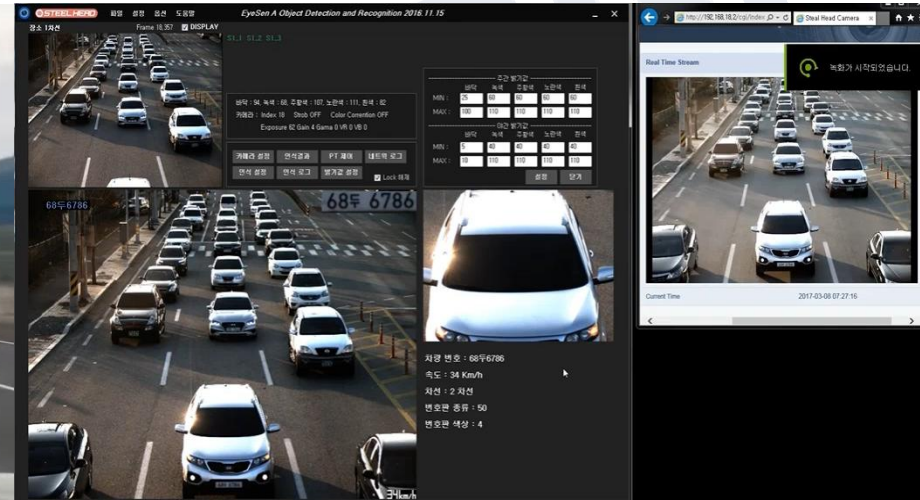
- 차량 주변의 정적 및 동적인 환경을 측정하고 주변 환경을 묘사
- 보행자, 차량, 자전거, 도로 표면, 차선 경계, 교통 신호 등
- 인지 정보를 바탕으로 주행에 대한 제어 방식을 계획하고 결정
- 다양한 센서(Camera, Radar, LiDAR)와 딥러닝 기술을 활용

1. 지능형 자율주행

● 인지 (Perception) - Camera 센서 기반 객체 인식



자료 출처: <https://www.youtube.com/watch?v=chzq2E75M84>



자료 출처: <https://www.youtube.com/watch?v=5zOFzh36CS0>

- 딥러닝 기반의 방법들로 우수한 성능을 보장할 뿐만 아니라, 높은 수준의 의미론적 정보 또한 추론 가능
- 계산 성능(Computing power)의 발달 및 모델 경량화로 On-device AI 가능
- Camera 센서의 내재적 한계점을 내포 : 조도 변화에 민감하며, 객체와의 정확한 거리 측정 불가
- LiDAR, Radar, HD Map 등과의 센서 퓨전을 이용한 상호 보완이 필수적

1. 지능형 자율주행

● 인지 (Perception) - LiDAR 센서 기반 객체 인식



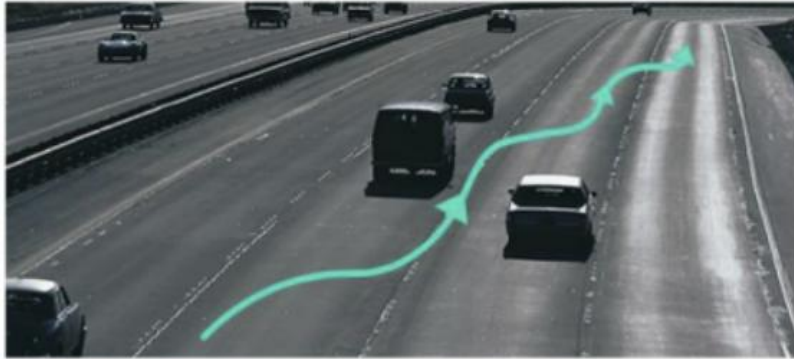
자료 출처. <https://www.youtube.com/watch?v=si9gamz07LA>

자료 출처. <https://www.youtube.com/watch?v=llDq9W1oSgw>

- LiDAR 기반 객체 인식 기술을 이용하여 대상 객체에 대한 3차원 공간 상의 정확한 위치, 거리, 이동 방향 등을 추정 가능
- 딥러닝 기반의 방법들로 성능 개선이 이루어지고 있으며, Camera 센서의 한계점을 상호 보완하는 형태로 사용됨
- 보다 많은 LiDAR 데이터셋의 확보와 딥러닝 모델의 성능 향상을 위한 연구 개발이 진행 중

1. 지능형 자율주행

● 판단 (Decision)



전역 및 지역 경로 생성
(Path Generation)



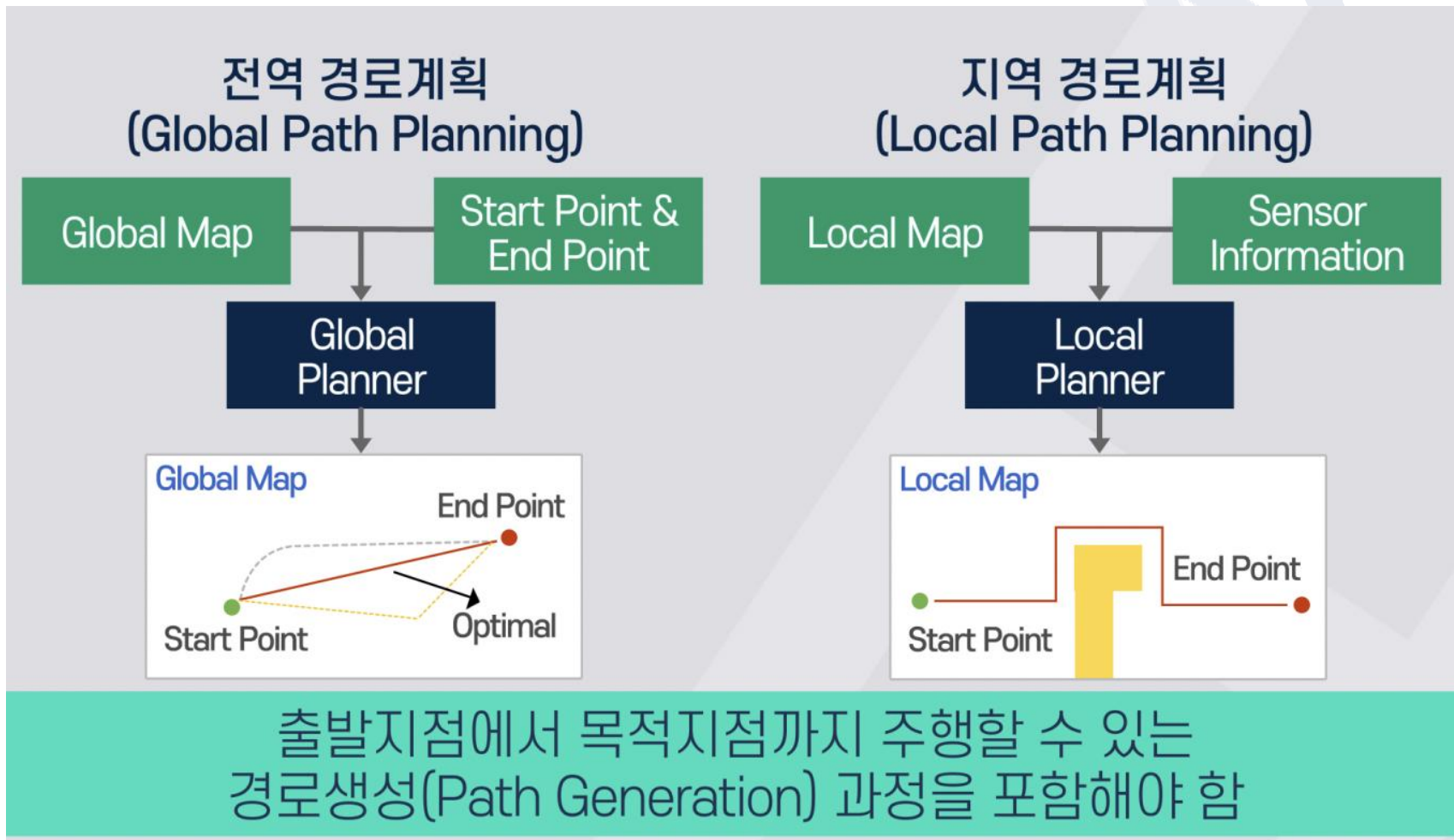
주행상황 판단
(Decision-Making)

자료 출처: <https://velog.io/@jaehawilly/posts>

- 지도, 차량의 위치, 도착 위치, 주변 환경 정보를 기반으로 가능한 경로를 예측/생성하는 기술
- 지도를 그래프 구조로 변환하여 최적의 경로를 탐색
- 최근에는 강화학습 기반의 경로 계획 기법들에 대한 연구가 활발히 이루어지고 있음
- 실제 환경에서의 실험이 어렵기 때문에 주로 가상 환경에서 시뮬레이션이 수행됨

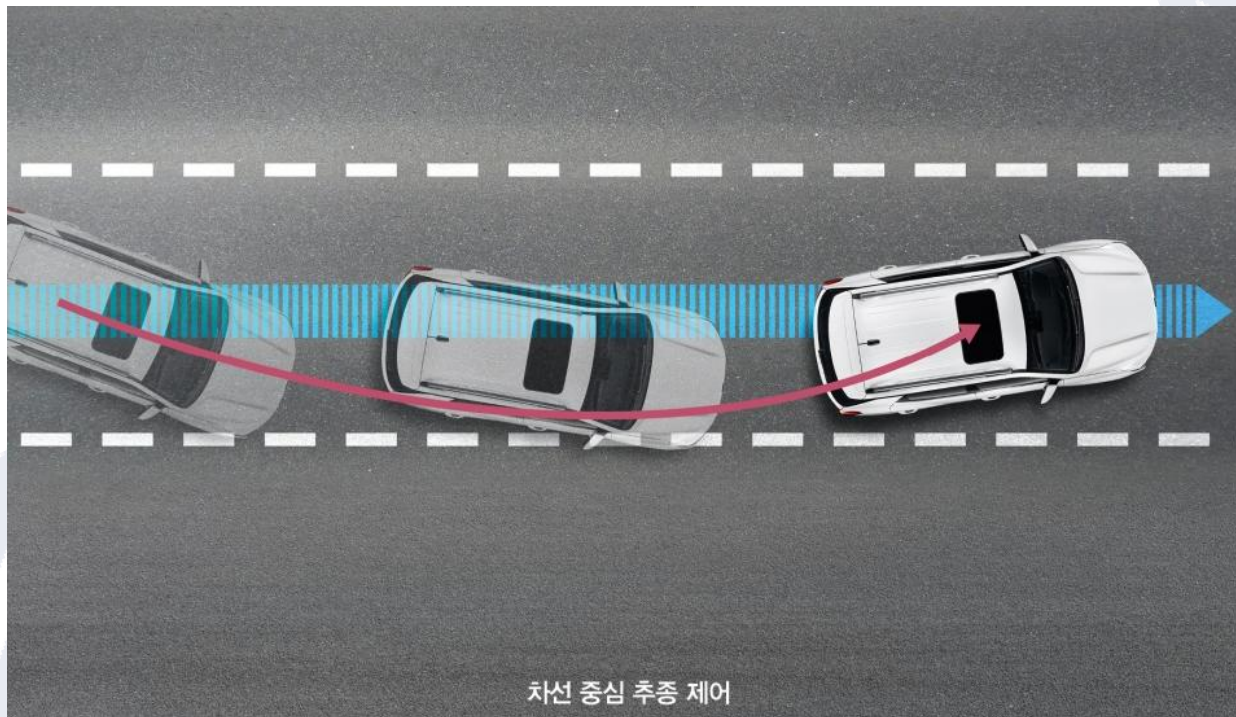
1. 지능형 자율주행

- 판단 (Decision)



1. 지능형 자율주행

- 제어 (Control)

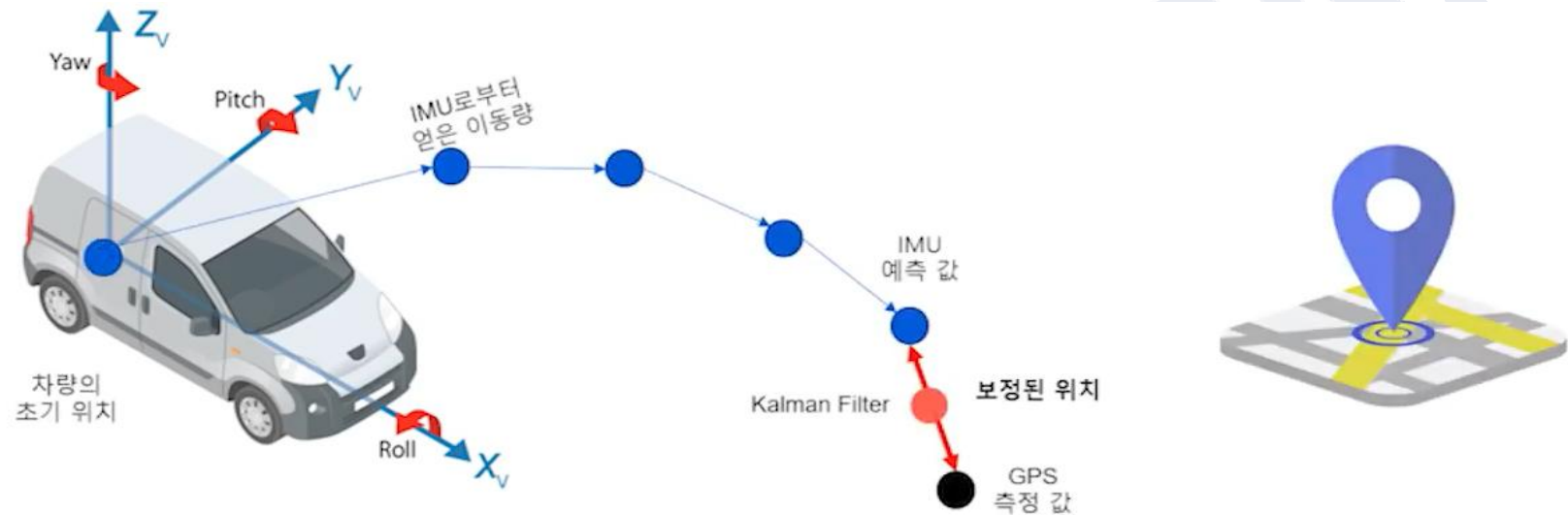


자료 출처. https://www.motorgraph.com/news/articleView.html?idxno=21621#google_vignette

- ‘판단’ 단계에서 결정된 전역 경로를 올바르게 추종하기 위해 구체적인 속도와 가속도, 조향각 등의 제어를 수행
- 제어 시스템은 차량의 실시간 위치와 계획된 경로 사이의 차이를 지속적으로 분석하여, 필요한 조정을 즉각적으로 수행

1. 지능형 자율주행

● 위치 추정 (Localization)



자료 출처: <https://www.youtube.com/watch?v=KQeKVLfgkX0&t=1134s>

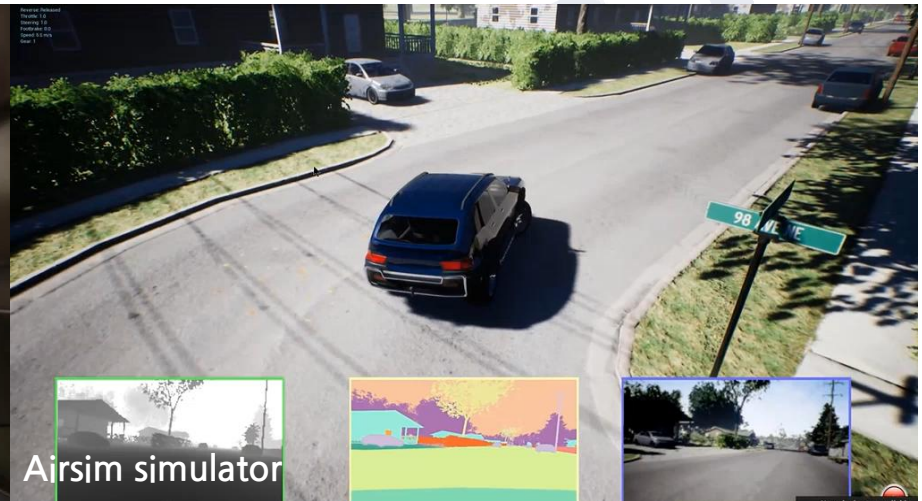
- 차량의 위치를 실시간으로 정확하게 추정(예측)하는 기술
- 정확도 향상을 위해 다양한 센서(GPS, IMU 등)를 조합하여 사용
- 일반적으로 GPS의 절대 위치와 IMU 추측 방법(dead reckoning)의 상대 위치를 결합하여 위치를 추정
- GPS는 주기가 느리고 음영 지역이 많아 도심지에서는 신뢰도가 떨어짐

1. 지능형 자율주행

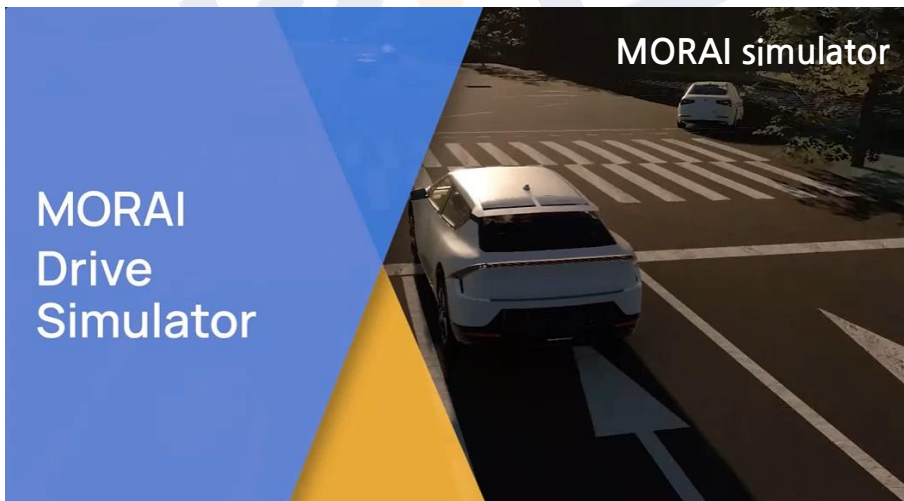
● 자율주행 시뮬레이션



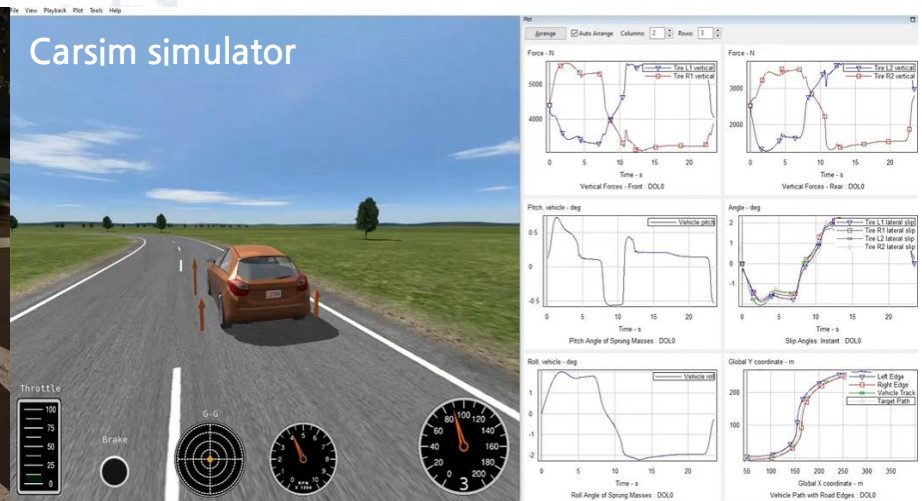
자료 출처: <https://www.youtube.com/watch?v=jNYstyOKxIA>



자료 출처: <https://www.youtube.com/watch?v=CauKo089zm0>



자료 출처: <https://www.youtube.com/watch?v=jNYstyOKxIA>



자료 출처: <https://www.youtube.com/watch?v=CauKo089zm0>

1. 지능형 자율주행

4) 대표적인 자율주행 기술

- ADAS (Advanced Driver Assistance System)

- ADAS(Advanced Driver Assistance Systems)는 고급 운전자 보조 시스템으로, 차량의 안전과 운전 편의를 증진하기 위해 설계된 다양한 기술과 기능의 총칭. 이 시스템들은 운전자의 입력을 보완하거나 일부 운전 작업을 자동화하여 사고를 예방하고, 운전 부담을 줄이는 데 중점을 둠.

ADAS는 각종 센서와 카메라, 레이더, 라이다 같은 기술을 활용하여 차량 주변의 환경 데이터를 수집 및 분석.

- **ADAS**에는 전방 충돌 경고(FCW), 자동 긴급 제동(AEB), 차선 이탈 경고(LDW), 차선 유지 보조(LKA), 적응형 크루즈 컨트롤(ACC) 등의 기술들이 포함.

1. 지능형 자율주행

4) 대표적인 자율주행 기술 – 횡방향 제어(Lateral control)

- 차선 이탈 방지 보조 (LKA, Lane Keeping Assistance)
 - 차량이 차선을 이탈하지 않고, 차선 중앙을 유지할 수 있도록 스티어링을 자동 조정.



1. 지능형 자율주행

4) 대표적인 자율주행 기술 – 종방향 제어(Longitudinal control)

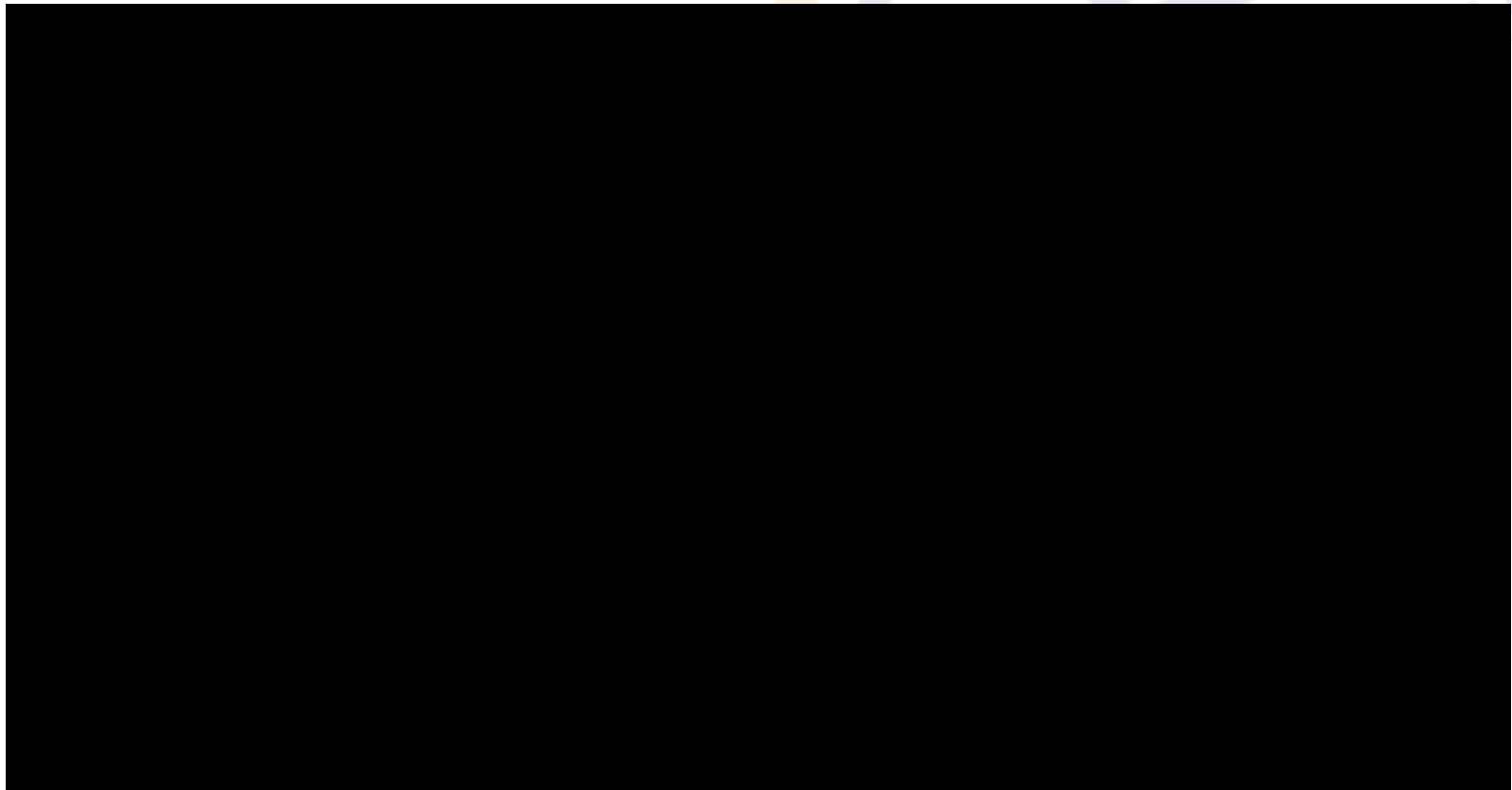
- 적응형 크루즈 컨트롤 (ACC, Adaptive Cruise Control)
또는 스마트 크루즈 컨트롤 (SCC, Smart Cruise Control)
→ 앞 차와의 거리를 단계별로 자동 조절하면서, 설정된 속도를 유지하도록 자동 제어.



1. 지능형 자율주행

4) 대표적인 자율주행 기술 – 충돌 제어(Collision control)

- 자동 긴급 제동 (AEB, Autonomous Emergency Braking)
 - 충돌 위험이 감지되면 차량이 자동으로 제동을 걸어 충돌을 피하거나 충돌의 심각성을 줄임.



1. 지능형 자율주행

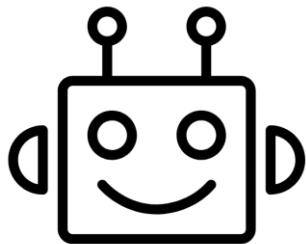
5) 자율주행 기술의 기대 효과

- **레벨 5**의 완전 자율주행 기술이 실현되면 개인 생활과 사회 전반에 큰 변화를 가져올 것으로 기대됨
- 완전 자율주행 자동차는 피로를 느끼지도 않고, 교통 규칙을 위반하는 일도 없으며, 사람 운전자보다 더 빠른 속도로 의사 결정을 할 수 있게 되어 운전자의 실수로 인한 **교통 사고와 피해를 크게 줄이고 안전도를 더욱 높일 수 있음**
- 운전자는 목적지까지 이동하는 동안 운전을 위해 집중할 필요 없이 **개인 용무를 보거나, 휴식을 취하거나, 오락적인 활동을 수행할 수 있게 되어** 자동차가 단순한 이동 수단을 넘어 **생활 공간으로 변하게 될 것으로 예상**
- 도로 위에 모든 자동차에서 자율 주행이 가능해지면 교통 흐름은 지금보다 더 효율적이고 최적화되어 **운송 수단에 의한 에너지 소비와 대기 오염을 줄임으로써 환경을 보존하고 깨끗한 미래를 실현하는데 기여할 수 있음**

로봇 자율주행 기술

2. 로봇 자율주행 기술

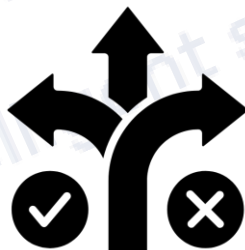
1) 로봇의 정의



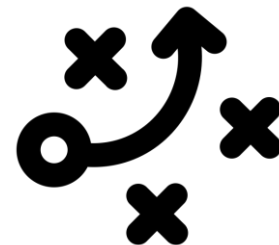
로봇 = 인간의 작업을 보조하는 지능형 기계



감지 (sensing)



판단 (Processing)



행동 (Acting)

2. 로봇 자율주행 기술

1) 로봇의 정의

- 로봇은 일반적으로 인간의 작업을 대체하거나 보조하기 위해 설계된 기계 장치를 의미
- 단순한 기계와 달리, 로봇은 환경을 인식하고, 정보를 처리하며, 특정한 작업을 자율적 또는 반자율적으로 수행할 수 있는 능력을 갖추고 있음

1) 감지 (Sensing)

로봇은 카메라, 라이다, 마이크 등 다양한 장치를 통해 주변 환경과 자신의 상태를 인식함.

예를 들어, 산업용 로봇은 부품의 위치를 감지하고, 자율주행 로봇은 장애물과 경로를 탐지.

2) 판단 (Thinking/Processing)

로봇은 수집한 정보를 바탕으로 적절한 행동을 결정. 이 과정에는 간단한 제어 알고리즘에서부터 인공지능 기반의 복잡한 의사결정까지 포함.

3) 행동 (Acting)

판단한 내용을 실제로 수행하는 단계로, 로봇의 팔, 바퀴, 다리 등 구동 장치를 이용해 작업을 실행함. 산업용 로봇의 용접, 의료용 로봇의 수술 보조, 서비스 로봇의 배달 등이 여기에 해당.

2. 로봇 자율주행 기술

1) 로봇의 정의

https://en.wikipedia.org/wiki/History_of_robots

시대	주요 사건 및 발명	대표 사례/특징
고대	인간형 자동 장치 등장	- 그리스: 헤파이스토스가 만든 금속 자동인형, 기계 장치로 움직이며 단순 작업 수행 - 중국: 동양의 초기 자동 장치, 불경을 읊는 금속 인형과 물 시계 자동화
르네상스	기계적 자동화 발전	- 레오나르도 다 빈치 (15세기 말): 인간의 움직임을 모방한 '기계 기사' 설계, 팔·손·머리·무릎까지 관절 구현 - 유럽: 자동 악기, 시계 장치 등 정밀 기계 발달
산업혁명	생산 자동화	- 직물 산업: 자동 방적기·직조기 개발, 반복 작업 자동화 - 증기 동력 기계: 공장 내 생산 효율 극대화
1920년대	'로봇' 용어 최초 등장	- 체코 카렐 차펙의 희곡 'R.U.R.' (1921): '로봇'이라는 용어 최초 사용, 인간형 노동자 기계 등장 - 일본: 초기 동양 로봇 개발, 기계 장치와 인형 결합
1940~50년대	전자식 자율 로봇 등장	- 윌리엄 그레이 월터 (1949): 빛에 반응하는 전자 로봇 '엘머'·'엘시', 자율적 이동 및 환경 반응 - 아이작 아시모프 (1942): 로봇 3원칙 제시, 로봇 윤리 논의 시작
1960~70년대	산업용 로봇 상용화	- 조지 드볼·조셉 앵거 (1956~60년대): 최초 산업용 로봇 '유니메이션' 개발, 용접·조립 작업에 활용 - 자동차 공장: 생산 라인 로봇 도입으로 반복 작업 자동화
1980~90년대	AI와 로봇 융합	- 일본 ASIMO 개발: 인간과 유사한 보행, 계단 오르기 등 동작 구현 - 미국 로보틱스 연구소: 자율 주행, 센서 기반 로봇 연구, 인공지능 적용
2000년대 이후	휴머노이드·협동 로봇 발전	- 의료 로봇: 수술 보조 및 정밀 조작 가능 - 협동 로봇(Cobots): 인간과 함께 작업 가능, 소규모 제조·서비스 산업 적용 - 휴머노이드 로봇: 인간과 유사한 동작, 사회적 상호작용 가능

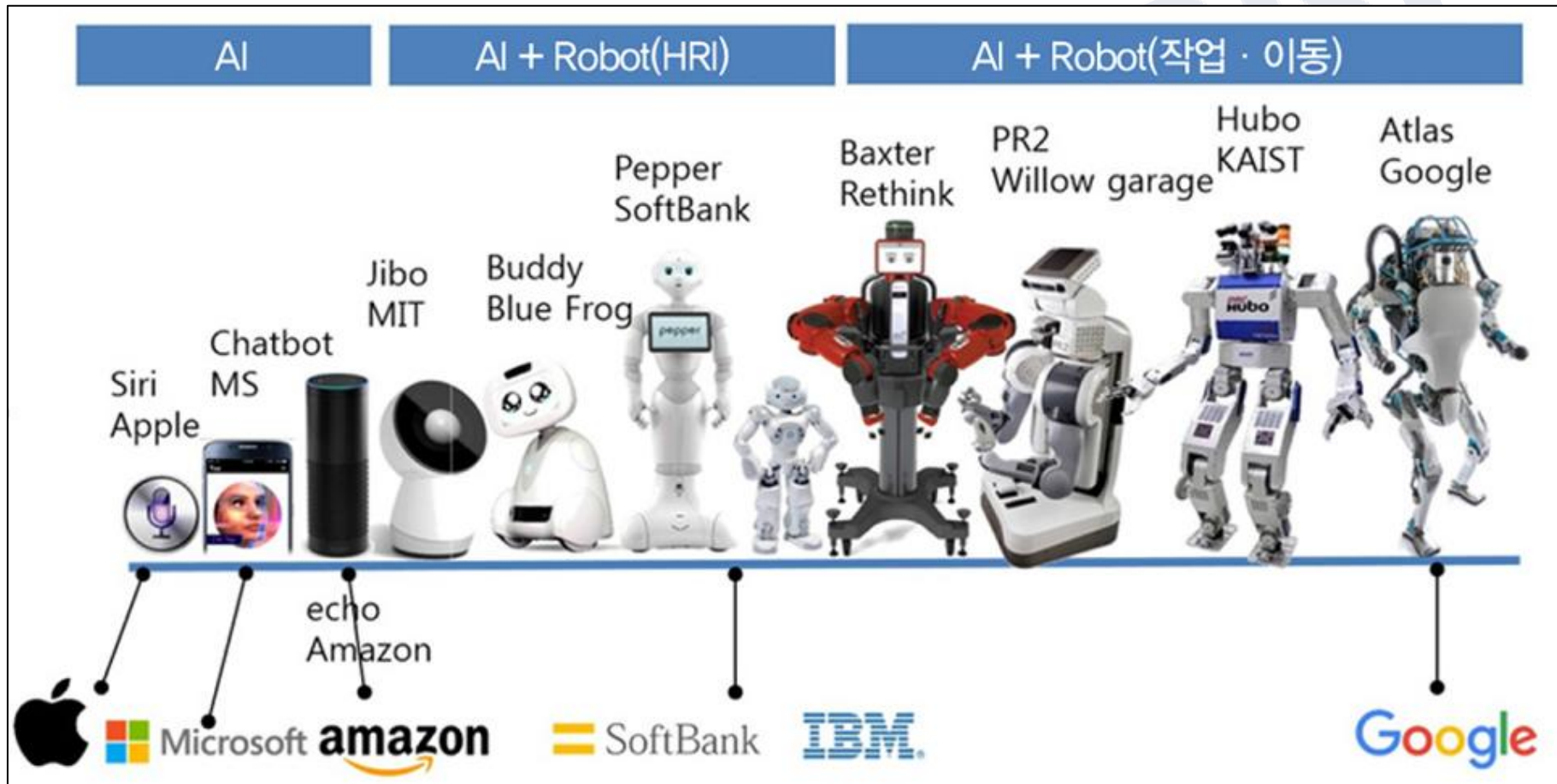
2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

서비스용로봇		제조용 로봇
개인서비스용	전문서비스용	
인간의 생활범주에서 제반서비스를 제공하는 로봇 예) 청소, 오락, 교육로봇	불특정 다수를 위한 서비스 제공 및 전문화된 작업을 수행하는 로봇 예) 안내, 소방, 의료로봇	각 산업의 제조현장에서 작업을 수행하기 위한 로봇 예) 용접, 도장로봇 등
   	   	  

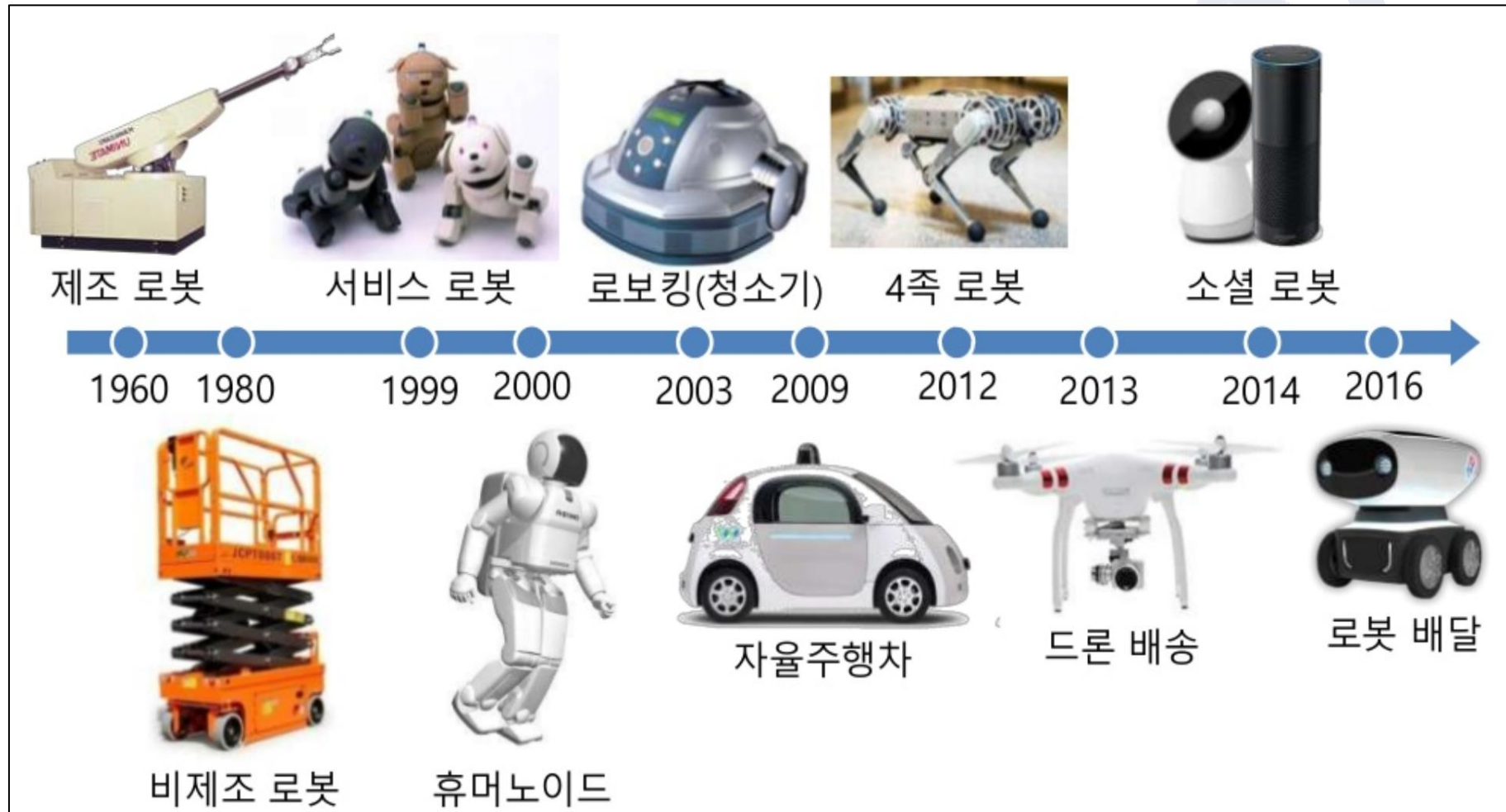
2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화



2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화



2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

● 서비스 로봇

- 전문 서비스 로봇은 의료로봇, 국방로봇과 같이 전문가를 보조하며, 서비스를 제공하는 서비스 로봇임. 로봇기술의 발달로 건설로봇, 의사로봇, 교사로봇, 경찰로봇 배달 등 다양한 서비스가 가능한 로봇으로 진화될 것
- 개인서비스 로봇은 청소로봇, 교육용 로봇과 같이 일반인을 대상으로 서비스를 제공하는 서비스 로봇임. 로봇기술의 발달로 오락로봇, 개인비서로봇, 간병로봇, 친구로봇, 웨어러블 로봇 등 다양한 서비스가 가능한 로봇으로 진화될 것



커피와 도넛을 싣고 출발한 배달 로봇

2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

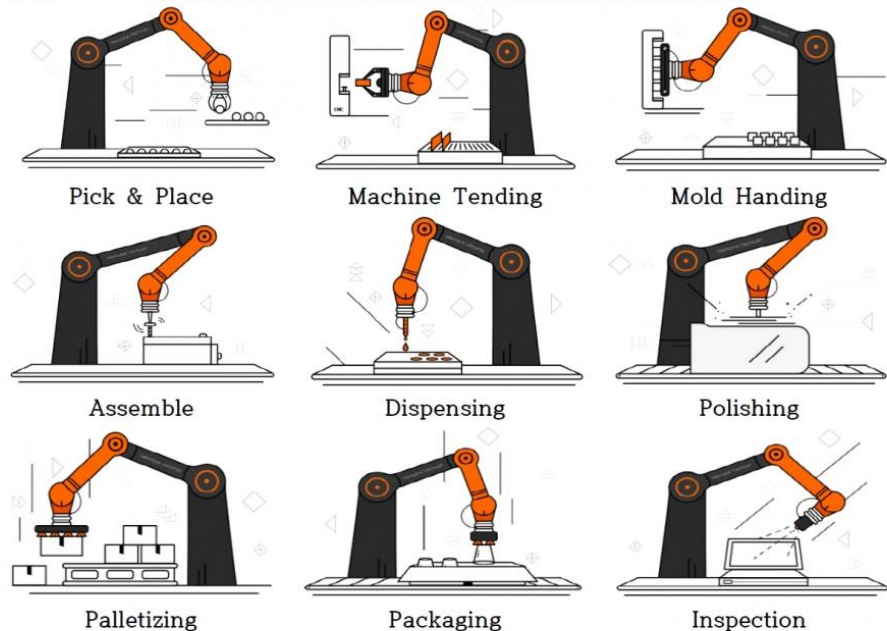
● 제조용 로봇

- 산업용 로봇은 제조용 로봇과 협동로봇으로 분류
- 협동로봇은 기존의 제조용 로봇과 달리 인간과 로봇이 같은 작업 공간 공유, 기존 생산라인에 추가로봇 설치 및 운영이 편리



제조용 로봇 분류				
구분	수직다관절로봇	수평다관절로봇	직교로봇	병렬로봇
사진				
용도	대부분의 제조공정	이적재, 조립, 실링	이적재, 나사체결, 검사장비	포장, 조립, 이송, 검사

협동로봇 적용 응용분야



2. 로봇 자율주행 기술

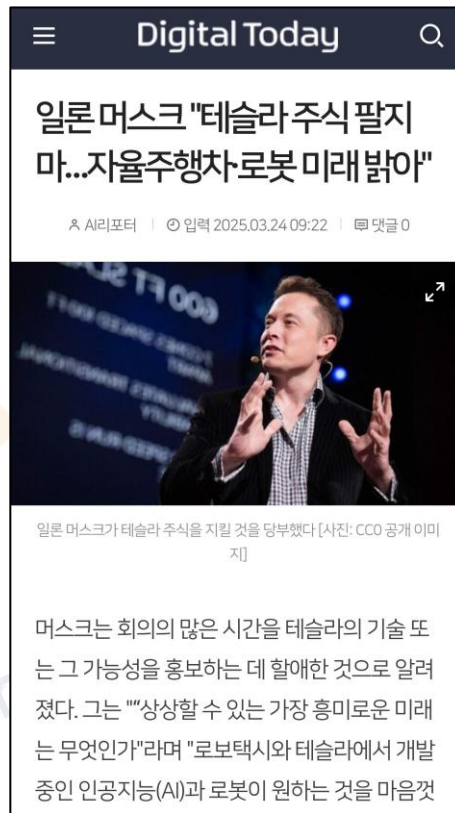
2) 자율주행 로봇의 다양화

- 국내외 관심 증대, 시장 확대

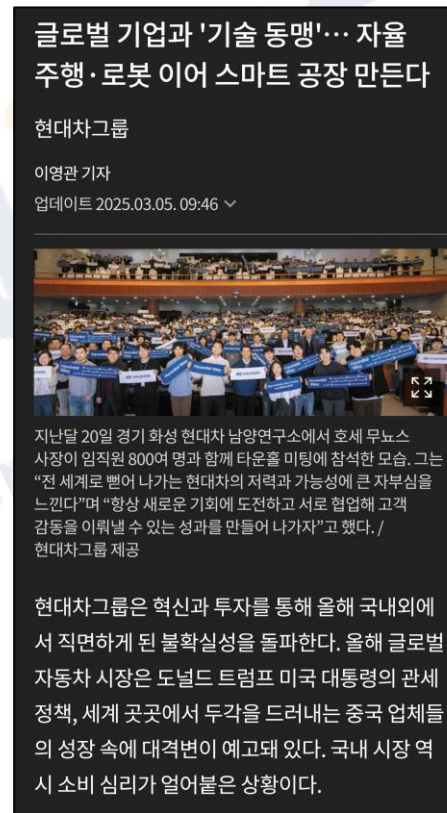
<https://zdnet.co.kr/view/?no=20250319001453>
<https://www.digitaltoday.co.kr/news/articleView.html?idxno=558339>
https://www.chosun.com/special/special_section/2025/03/05/X4E2HKJN4RCRRIODRA4XYJOMWE/
<https://www.irobotnews.com/news/articleView.html?idxno=38012>



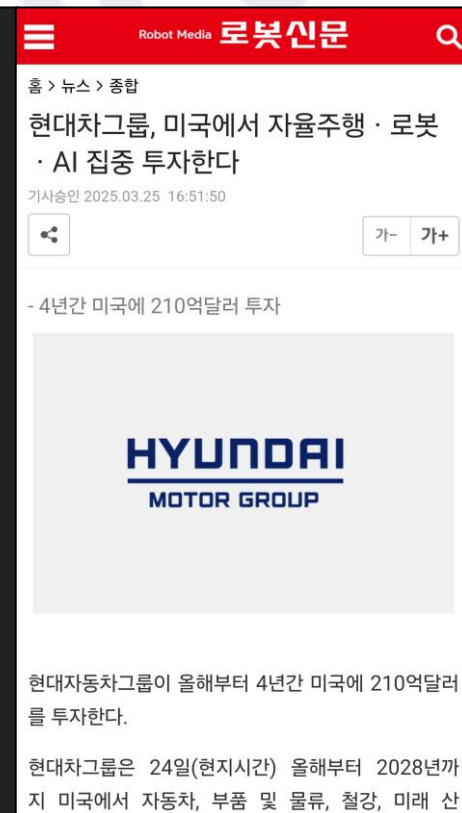
NVIDIA



Tesla



현대자동차



2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

- 국내외 관심 증대, 시장 확대

<https://www.youtube.com/shorts/zmEpDXfiP6w>
<https://www.youtube.com/watch?v=ZibMCHKh3AQ4>

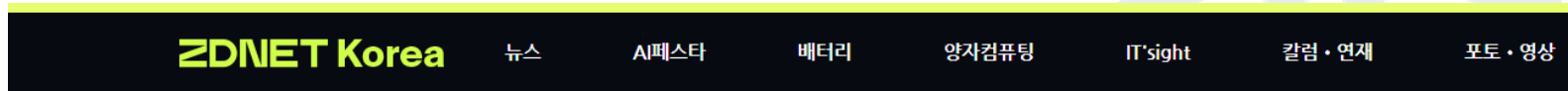


2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

- 해결하지 못한 문제

<https://zdnet.co.kr/view/?no=20250923155836>



HP 디자인젯 최초 잉크 탱크 프린터
K-Print 2025에서 그랜드 런칭
전시, 프린터 문의는 080 880 7979



Adobe Creative Cloud

Add an app and see what new can do.

Add now



테슬라 공장서 로봇 공격으로 직원 중상...대규모 소송

프리몬트 공장서 일하던 직원, 테슬라에 약 710억 원 규모 소송 제기

컴퓨팅 | 입력 : 2025/09/23 17:11 수정 : 2025/09/23 22:02



이정현 미디어연구소 | ✉ 기자 페이지 구독 | 📁 기자의 다른기사 보기



[이벤트] 워크로드 최적화를 위한 선택, HPE 설문조사 참여 시 ALIO 노트북 거치대 증정!

미국 캘리포니아주 프리몬트 공장에서 일하던 테슬라 로봇 기술자가 테슬라를 상대로 5천100만 달러(약 710억 원) 규모 소송을 제기했다고 영국 인디펜던트가 22일(이하 현지시간) 보도했다.

보도에 따르면, 테슬라 직원 피터 힌터도블러(Peter Hinterdobler)는 통제불능 상태에 빠진 로봇이 아무 경고도 없이 자신을 치어 중상을 입었다고 주장했다.



Bluetooth

모두에게 접근성이 높은 세상 만들기

더 알아보기



지금 뜨는 기사

- ▲ "도심에서 40분"...김동선의 하이엔드 리조트 '안토' 가보니
- ▼ 카톡, SNS처럼 바뀐다..."내가 원하는 만큼만 공개"
- ▲ 중고나라, 앱에서만 상품 등록 허용...안전 거래 강화 속 사용자 불만도 ↑
- ▼ 테슬라 공장서 로봇 공격으로 직원 중상...
- ▼ [Q&A] 2차 민생지원금 신청 방법과 지...

2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

<https://www.youtube.com/watch?v=DnISdsQ7rql>

- 미래는?



2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

<https://zdnet.co.kr/view/?no=20250304001043>
<https://mediahub.seoul.go.kr/archives/2013800>
<https://www.yna.co.kr/view/AKR20221205079300004>

하이코어, MWC25서 AI 자율주행 퍼스널 모빌리티 선봬

오는 9월 인천공항서 시범 서비스 개시

디지털경제 | 입력 2025/03/04 08:25 수정 2025/03/05 11:17

신영빈 기자 | ✉         

[백서다운] 로움 기술 자료 : IGBT 입문자를 위한 핸드북 'IGBT의 기초' 지금 다운받으세요!

하이코어는 3일부터 6일까지(현지시간) 스페인 바르셀로나에서 열리는 세계 최대 정보통신 박람회 MWC25에서 AI 자율주행 퍼스널 모빌리티를 최초로 선보인다고 4일 밝혔다.

하이코어는 모델솔루션과 공동 개발한 AI 자율주행 퍼스널 모빌리티를 최초로 공개한다. 부스 내에서 방문객들이 제품을 체험할 수 있는 공간을 마련했다.



MWC 2025에 공개된 하이코어 AI 자율주행 퍼스널 모빌리티 (사진=하이코어)

제품은 오는 9월부터 인천국제공항에서 제2터미널 확장구간에서 시범 운영에 투입된다. 연세구간에서 최종 목적지까지 캐리어를 적재한 채 자율주행으로 이동할 수 있게 된다.

이동 약자 및 영유아 동반 여행객들도 누구나 편리하게 사용할 수 있어 공항 내 이동을 지원한다. 최종 목적지에 도착한 후에는 자동으로 회귀하는 기능도 탑재했다.

하이코어는 스마트시티 환경에서 실제 운영을 위한 첨단 AI 자율주행 기술, 관제 시스템, 통신 시스템을 선보인다.

'자율주행 휠체어'가 금강해! 'AI 페스타'서 미래도시 서울을 맛보다

시민기자 배정연 | 발행일 2025.03.21. 13:00 | 수정일 2025.03.21. 16:04 | 조회 465

초고령화 사회, 교통약자를 위한 1인 모빌리티와 AI의 만남

세계 10대 도시 중 하나인 서울시는 인공지능(AI)을 활용하여 어떤 미래 도시를 만들어갈까? 지난 3월 8-9일 진행된 '서울AI페스타 2025'에서 선보인 다양한 AI 기술을 통해 시민이 사람을 위한, 사람의 한계를 초월하는 혁신의 경험을 제공하고 있음을 실감할 수 있었다.

'서울AI페스타 2025'에서 여러 AI 기술들을 만나 볼 수 있었는데, 행사 참여 기업 중 하나인 '유니코어 로보틱스(UNICORE ROBOTICS)'는 1인 퍼스널 모빌리티 기반 자율주행 기술을 선보이며 다가올 미래 도시에서 AI 기술이 어떻게 활용될 수 있는지 보여주었다. 이들이 왜 1인용 퍼스널 모빌리티에 집중하고 있는지, 그리고 AI 기술을 통해 어떤 비전을 그리고 있는지 궁금해졌다.



유니코어 로보틱스 ©배정연

유니코어 로보틱스는 서울시 한양대학교 연구팀에서 시작하여 AI 기반 로봇 및 자율주행 모빌리티 솔루션을 개발하는 소프트웨어 기업이다. 이들은 초고령화 사회에 접어들 현대 사회에서 교통약자의 실생활을 개선하여 새로운 이동 경험을 제공하는 AI 기술을 연구하고 있다. 이에 유니코어 로보틱스의 '강창목' 대표(이하 CEO)를 만나 이야기를 나눠보았다.

Q. 유니코어 로보틱스는 어떻게 시작하게 되었나요?

A. 유니코어 로보틱스는 원래 자율주행 소프트웨어를 연구하는 연구실에서 출발했습니다. 당시 협력 기업 중 하나가 모바일 플랫폼 기반의 자율주행 휠체어를 개발하고 있었는데, 기존 휠체어에는 긴급제동장치나 장애물 회피 기능 같은 필수적인 안전장치가 부족하다는 점을 알게 되었습니다. "이런 기술을 접목하면 더 안전한 모빌리티가 되지 않을까?"라는 생각에서 연구를 시작하게 되었습니다. 저희는 단순한 이동 수단이 아니라 교통약자를 위한 맞춤형 모빌리티를 필요한 분들에게 제공하기 위한 AI 기반 솔루션을 제공하고 있습니다.

Q. 수많은 자율주행 모빌리티 중에서 '휠체어'를 선택한 이유가 있나요?

A. 휠체어는 자율주행 자동차와 로봇의 중간 형태라고 생각합니다. 자율주행 자동차는 다수를 위한 기술로 발전하고 있고, 로봇은 주로 무인 배송이나 순찰과 같은 용도로 사용됩니다. 반면 휠체어는 한 사람을 위한 맞춤형 이동 수단이기 때문에 탑승자의 승차감과 안전을 세밀하게 고려해야 합니다.

관악구 자율주행 순찰로봇, '디지털 지역혁신' 국무총리 표창

송고 2022-12-05 13:52

윤보람 기자
+ 구독



서울 관악구 '자율주행 순찰로봇' 시범 운영 모습

[관악구 제공, 재판매 및 DB 금지]

(서울=연합뉴스) 윤보람 기자 = 서울 관악구(구청장 박준희)는 행정안전부가 주관한 '2022 탄소중립 지역발전 및 지역혁신 유공' 디지털 지역혁신 분야에서 국무총리 표창을 받았다고 5일 밝혔다.

관악구는 전국 최초로 도심지에서 운영한 '자율주행기반 안심순찰 서비스'로 해당 분야에서 자치단체 중 유일하게 표창을 받았다.

이 서비스는 자율주행 순찰로봇을 활용, 주택가와 공원 등을 실시간 모니터링해 사고 발생을 방지하는 방식이다.

순찰로봇은 심야에 서행동 주택가와 해태어린이공원 인근을, 주간에는 신림동 별빛내린천변을 돌며 촬영한 영상을 관악구 스마트 통합관제센터로 전송한다.

전·후방, 측면, 열화상 등 카메라 총 5대를 갖추고 있어 다각도 촬영이 가능하다. 경찰차와 유사한 경광등을 사용해 방범 효과도 극대화했다.

관악구는 자율주행 등 첨단기술을 활용해 전국으로 확산 가능한 모델을 개발한 점과 주민·과학 기술 전문가·화교가 함께 머리를 맞대 지역 문제를 해결한 점에서 호평을 받았다고 전했다.

2. 로봇 자율주행 기술

2) 자율주행 로봇의 다양화

<https://www.aitimes.kr/news/articleView.html?idxno=22630>
<https://www.woowahan.com/report/detail/590>
<https://www.aitimes.kr/news/articleView.html?idxno=29181>
<https://mediahub.seoul.go.kr/archives/2013837>
https://www.airport.kr/co_ko/index.do

● 배달 로봇



● 순찰 로봇



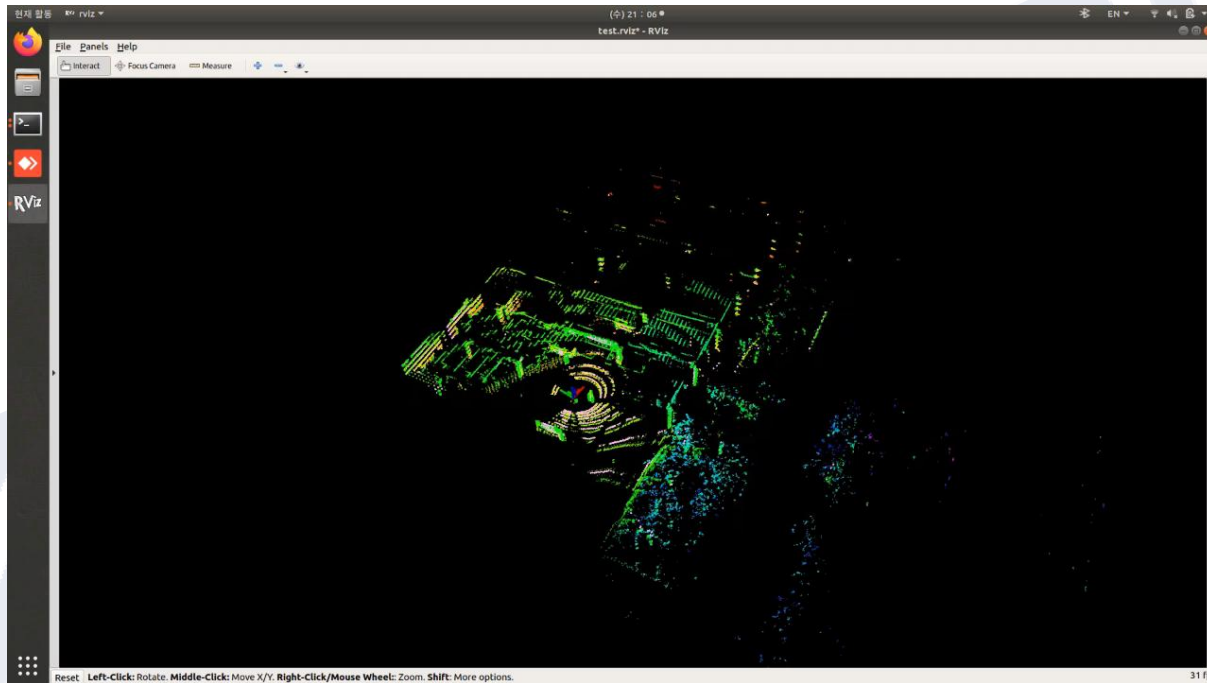
● 인천국제공항



2. 로봇 자율주행 기술

3) 자율주행 기술

- SLAM (Simultaneous Localization and Mapping)

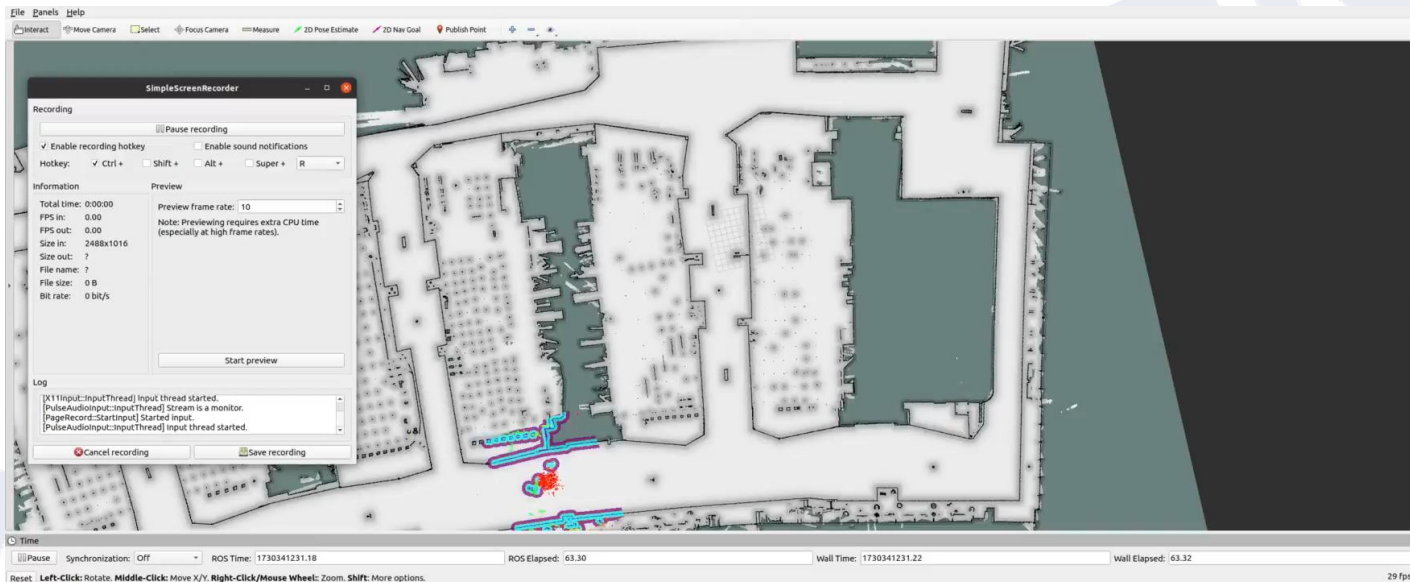


- 연속적인 Camera 센서(Mono, Stereo) 또는 LiDAR 센서로부터 상대적 위치를 추정하는 기술
- 랜드마크에 대한 특징을 추출하고 대응 관계를 수립하여 프레임 간 차량의 움직임(위치)를 추정
- 위치 추정을 통해 획득한 정보를 기반으로, 측정 데이터들을 누적하여 지도 생성을 동시에 수행
- 누적 오차를 방지하기 위해 정밀하게 제작된 HD Map(High-Definition Map)이 필수적

2. 로봇 자율주행 기술

3) 자율주행 기술

- Navigation

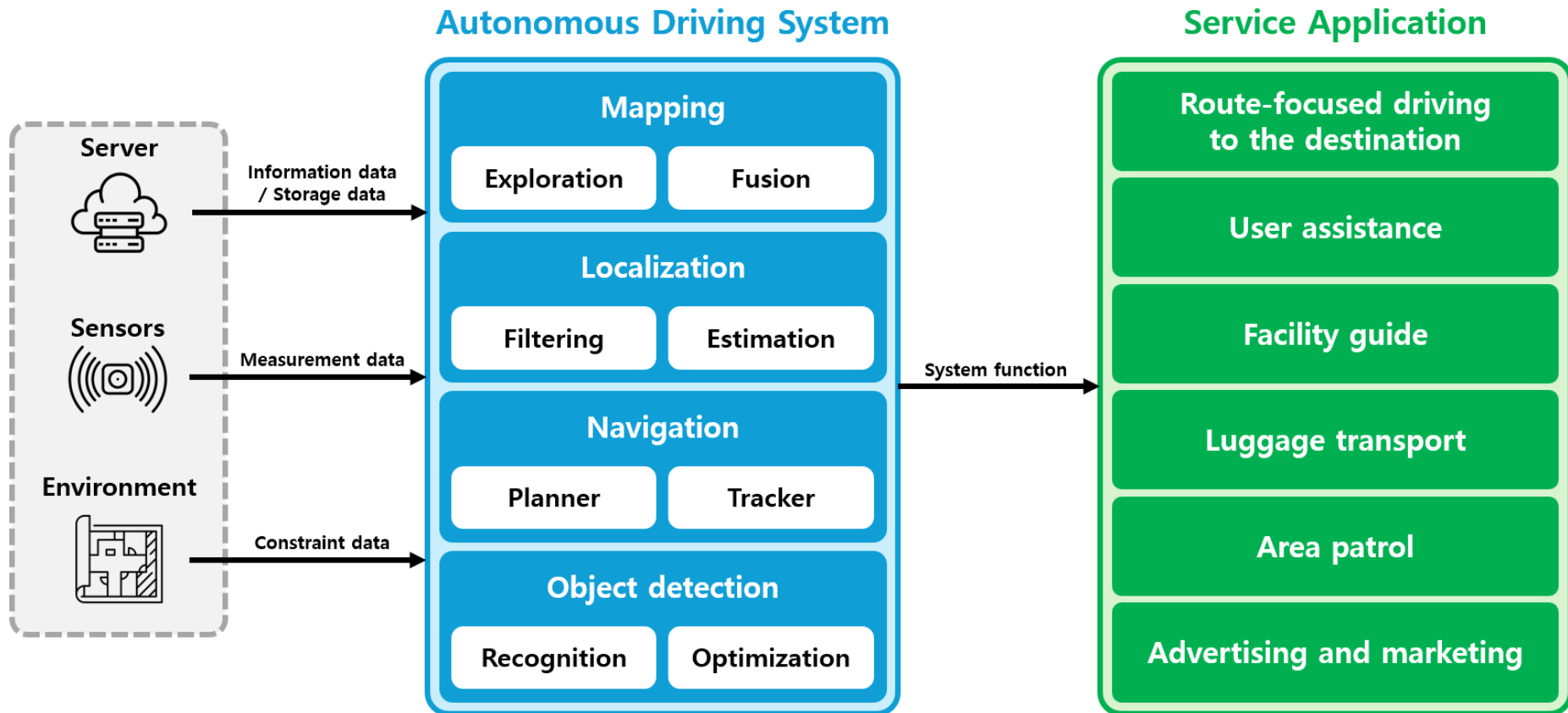


- 로봇이 주변환경을 인식하고 계획된 경로를 따라 자율적으로 이동
- HD Map(High-Definition Map) 기반으로 퓨전된 센서 데이터에 의한 현재 위치 추정
- 실시간 경로 계획 및 최적 제어, 장애물 회피까지 포함
- 주행 제어는 다양한 기법이 연구되었으며, 연산량이 적고 강인한 제어 기법을 추천

2. 로봇 자율주행 기술

3) 자율주행 기술

- Multi Purpose Robot



2. 로봇 자율주행 기술

3) 자율주행 기술

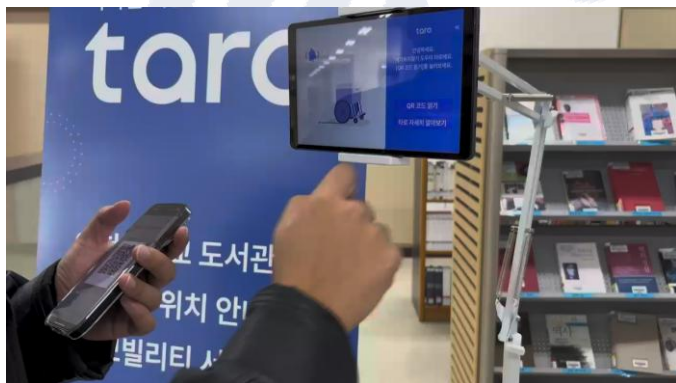
- 연구실 프로젝트



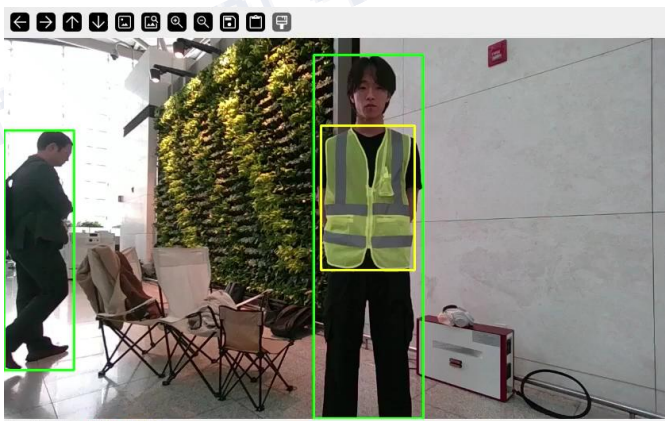
MWC22
바르셀로나, 스페인



도심형 순찰로봇
서울시 관악구



다목적 서비스 로봇
인천대학교 도서관



객체 인식 기술
인천국제공항



폐기물 이송 로봇
인천국제공항

2. 로봇 자율주행 기술

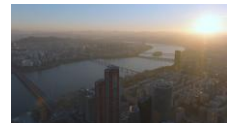
3) 자율주행 기술

- 연구실 프로젝트



HYUNDAI
MOTOR GROUP

Pleios 25



Linux



3. Linux

1) Linux 개요

- 정의

- 1991년 당시 핀란드 헬싱키 공대생이었던 리누스 토발즈(Linus Torvalds)에 의해 오픈소스로 개발된 개인용 컴퓨터, 서버, 메인 프레임, 모바일 기기, 임베디드 기기를 위한 Unix 호환 운영체제
- 리눅스가 리누스 토발즈에 의해 처음 개발된 당시에는 지금과 같은 파일 관리, 네트워크 관리, X Window, 개발자 도구 등의 다양한 패키지를 포함하는 운영체제의 형태는 아니었고, 단지 리눅스 커널만을 의미했음
- 현재는 FSF(Free Software Foundation)가 GNU 프로젝트를 통해 리눅스 커널 기반으로 다양한 라이브러리와 애플리케이션이 포함된 리눅스 배포판을 제공하고 있으며, 이를 강조하고자 특별히 GNU/Linux라고도 부름



3. Linux

1) Linux 개요

● 장점

- 오픈소스이기 때문에 상용 운영체제 대비 라이선스 비용이 들지 않아 경제적
- 오픈소스 정신을 지지하는 전세계의 수많은 개발자들이 공개된 리눅스의 커널 소스를 수정하고 개량함으로써 운영체제를 더욱 안정적으로 개선하고 보안성을 강화하고 있음
- 리눅스는 이더넷(Ethernet), SLIP, PPP 등의 다양한 네트워크 환경을 지원하며 TCP/IP, IPX 등의 네트워크 프로토콜도 지원
- 리눅스는 서버, 개발용, PC용 등 다양한 목적으로 사용할 수 있고, 이에 따른 다양한 배포판이 존재
- 임베디드 리눅스와 같이 특정 하드웨어에 맞게 커널의 기능을 더하거나 덜어내어 제품에 맞게 커스터마이징도 가능

● 단점

- 전세계의 개발자들이 리눅스를 발전시키고 있지만 리눅스를 사용함에 있어 발생하는 여러 사용자들의 이슈를 해결해 줄 수 있는 체계적인 기술지원 네트워크가 없음
- 다양한 오픈소스 소프트웨어가 존재하지만 마이크로소프트 오피스와 같은 쓸만한 리눅스용 상용 소프트웨어가 부족함
- 최신 하드웨어 기기에 대한 디바이스 드라이버 지원이 느림. 하드웨어 제조사에서 리눅스용 드라이버를 제공하는 경우에는 문제가 없지만 그렇지 않은 경우에는 오픈소스 개발자들이 하드웨어 기기를 역분석(reverse engineering)하여 드라이버를 만들어내는 데까지 오랜 시간이 걸림

3. Linux

1) Linux 개요

- 활용 분야

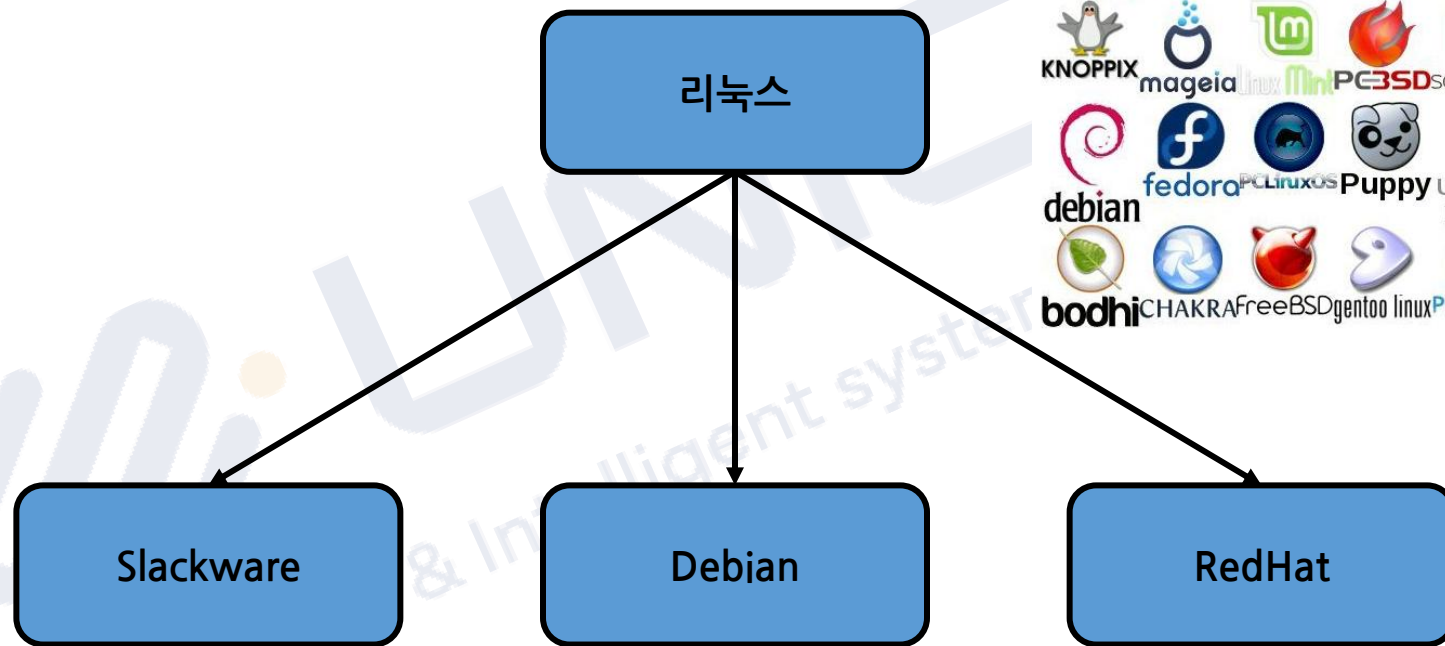
- 서버
- 메인 프레임
- 스마트 디바이스 (스마트폰, 태블릿, 스마트 TV, IVI(In-Vehicle Infotainment) 시스템 등)
- 임베디드 디바이스 (임베디드 보드, 각종 센서류, IoT 등)
- 게이밍 디바이스
- ...

퍼블릭 클라우드 컴퓨팅 워크로드의 90%, 스마트폰의 82%, 임베디드 기기의 62%, 슈퍼 컴퓨터 시장의 99%가 Linux로 작동

3. Linux

1) Linux 개요

● 리눅스 배포판



- 가장 오래된 배포판
- 가장 유닉스다운 리눅스 배포판

- 기업, 재단보다는 자발적 커뮤니티에 의한 배포판
- APT 패키지 관리자를 통한 패키지 설치 및 업그레이드의 편리함

- 고객 유료 서비스를 통한 수익 창출을 추구
- 일반 사용자를 위한 무료 배포판 제공



3. Linux

1) Linux 개요

● 리눅스 배포판

분류	종류	설명
RedHat 계열	RHEL (RedHat Enterprise Linux)	페도라를 기반으로 레드햇에서 개발한 리눅스 배포판이며 안정성을 목적으로 제작되었고 유료 라이선스로 제공되며 레드햇이 기술지원을 제공
	Fedora	- 레드햇의 후원과 페도라 커뮤니티의 지원 아래 범용 목적으로 개발된 것으로, RPM 기반의 소프트웨어가 결합된 리눅스 배포판 - MeeGo, SailFish OS, Tizen
	CentOS (Community ENTERprise Operating System)	페도라가 최신기술을 담아내는데 집중한 나머지 RHEL을 제대로 반영하지 못하자 RHEL을 완벽하게 호환하여 무료 기업용 컴퓨팅 플랫폼 제공 목적으로 개발된 리눅스 배포판
	Oracle Linux	2006년부터 GNU GPL 라이선스에 따라 레드햇 소스코드를 공유하면서 오라클 어플라이언스에 최적화된 리눅스 배포판
	Scientific Linux	RHEL 기반의 자유 및 오픈소스 운영 체제로서 상용 기업형 배포판에 근접하도록 만들기 위한 목적으로 페르미 국립 가속기 연구소가 개발한 리눅스 배포판
Debian 계열	Ubuntu	- 마크 셔틀워스가 우분투 보급을 위해 세운 영국의 컴퓨터 소프트웨어 회사인 캐노니컬 유한회사(Canonical Ltd)에서 개발한 리눅스 배포판 - 전세계 사람 누구나 리눅스를 쉽게 사용하자는 미션을 바탕으로 사용자 편의성에 초점을 맞춘 리눅스
	Raspbian	마이크 톰슨과 피터그린이 독립 프로젝트로 시작한 라즈비안은 2012년 6월 처음 완성된 이래로 2015년부터 라즈베리파이 재단에 의해 공식 지원을 받고 있는 라즈베리파이 전용 리눅스
	ChromeOS / Chromium OS	- 크롬OS는 구글이 설계하고 개발한 웹 애플리케이션 전용 운영체제로 사용자 인터페이스는 크롬 브라우저와 유사 - 사용자가 대부분의 시간을 브라우저를 이용하여 인터넷을 탐색한다는 점에 착안을 두어 개발한 리눅스 기반 운영체제
Slackware 계열	Slackware	- 가장 먼저 대중화된 현존하는 가장 오래된 배포판으로 1993년 패트릭 볼커딩에 의해 개발 - 간결함을 설계 철학으로 하여 유닉스 자체 학습에 적합
	openSUSE	- 슬랙웨어 리눅스로부터 파생되었으며 독일에서 제작하였고 유럽에서 많이 사용 - 유료 라이선스인 SUSE Linux Enterprise와 무료 버전인 openSUSE 두 가지로 배포
	Vector Linux	로버트 S. 란게와 다렐 스타뎀에 의해 개발된 SOHO, Standard, Light, Live판을 갖는 슬랙웨어 계열 리눅스 운영체제

3. Linux

1) Linux 개요

● 리눅스 배포판

분류	종류	설명
Android 계열	Android	- 휴대용 기기에 최적화된 리눅스 기반 모바일 운영체제 - 리눅스 커널을 기반으로 안드로이드 런타임 실행 - 안드로이드 런타임상에는 웹브라우저, 이메일 클라이언트, 단문 메시지 서비스(SMS) 등 다양한 응용 프로그램이 구동하고 이러한 앱을 위한 풍부한 런타임 라이브러리 제공
	AOSP (Android Open Source Project)	- 안드로이드는 구글에 의해 개발되지만 소스코드는 AOSP를 통해 운영 - AOSP를 통해 다양한 디바이스에 최적화된 안드로이드를 포팅 및 커스터마이징할 수 있음
국내 계열	하모니카	2014년 과기정통부의 전신인 미래창조과학부에서 발표한 “공개 SW 활성화 계획”의 일환으로 정부가 주도로 개발한 리눅스 민트 기반 개방형 운영체제 - 기존 배포된 리눅스 민트보다 한글화가 잘 되어 있음
	넘버원 리눅스	독자적인 토종 리눅스 개발을 목표로 PCLinux 기반 KDE 환경을 제공하는 리눅스 배포판
	구름OS	과학기술정보통신부와 한국전자통신연구원(ETRI) 산하 국가보안기술연구소 및 한글과 컴퓨터가 개발한 개방형 OS

3. Linux

2) Ubuntu

- 편리한 설치와 사용법으로 초보자들이 연습하기 좋은 배포판
- 6개월 단위로 새 버전 발표, 2년 주기로 LTS(Long Term Support: 5년 간 기술지원 제공) 버전 발표

버전	코드네임	배포일	커널 버전	지원 종료일
6.06 LTS	Dapper Drake	2006년 06월 01일	2.6.15	2009년 07월
8.04 LTS	Hardy Heron	2008년 04월 24일	2.6.24	2011년 05월
10.04 LTS	Lucid Lynx	2010년 04월 29일	2.6.32	2013년 05월
12.04 LTS	Precise Pangolin	2012년 04월 26일	3.2.0	2017년 04월
14.04 LTS	Trusty Tahr	2014년 04월 17일	3.13.0	2019년 04월
16.04 LTS	Xenial Xerus	2016년 04월 21일	4.4.0	2021년 04월
18.04 LTS	Bionic Beaver	2018년 04월 27일	4.15	2023년 04월
20.04 LTS	Focal Fossa	2020년 04월 23일	5.4	2025년 04월 예정
22.04 LTS	Jammy Jellyfish	2022년 02월 22일	5.15	2027년 04월 예정
24.04 LTS	Noble Numbat	2024년 04월 25일	6.8	2029년 04월 예정

- 시스템 요구 사항

CPU	2 GHz dual core processor
메모리	4 GB
디스크 여유 공간	25 GB
그래픽카드 해상도	1024x768



3. Linux

3) Ubuntu 설치

What's new in Ubuntu 20.04 LTS?

Learn why 20.04 is the new reference platform
for secure cloud, edge compute and desktop.

[Register for the webinars](#)

[Download Ubuntu 20.04 LTS >](#)



3. Linux

4) 주요 명령어

- `ctrl + alt + t`: 터미널 실행
- `ctrl + shift + t`: 현재 터미널에서 새로운 탭으로 터미널 실행
- `ctrl + shift + w`: 탭으로 실행된 터미널 종료
- `ctrl + shift + q`: 현재 터미널 종료
- `ctrl + shift + f`: 터미널 내에서 문자열 검색
- `ctrl + shift + c`: 복사하기
- `ctrl + shift + v`: 붙여넣기
- `shift + PageUp`: 터미널 내의 상위 페이지 이동
- `shift + PageDown`: 터미널 내의 하위 페이지 이동

3. Linux

4) 주요 명령어

- **(sudo) apt**: 패키지 관리 명령어. update, upgrade, install, remove, purge, policy 등 추가 명령어를 붙여 사용
- **(sudo) chmod**: 파일/폴더 권한을 설정하는 명령어. 뒤에 8진법 숫자나 옵션을 통해 권한을 부여
- **(sudo) chown**: 파일/폴더의 소유권을 설정하는 명령어. 계정명(UID), 그룹명(GID), 파일 또는 디렉터리 형태로 사용
- **ls**: 디렉토리 목록 확인
- **cd**: 디렉토리 이동
- **mkdir**: 새 디렉토리 생성
- **rm**: 파일 또는 디렉토리 삭제
- **cp**: 파일 또는 디렉토리 복사
- **mv**: 파일 또는 디렉토리 이동 (이름을 변경할 때에도 사용)
- **touch**: 비어있는 파일 생성
- **cat**: 파일 내용 보기
- **pwd**: 터미널 내 현재 위치 확인
- **clear**: 터미널 내용 삭제

3. Linux

4) 주요 명령어

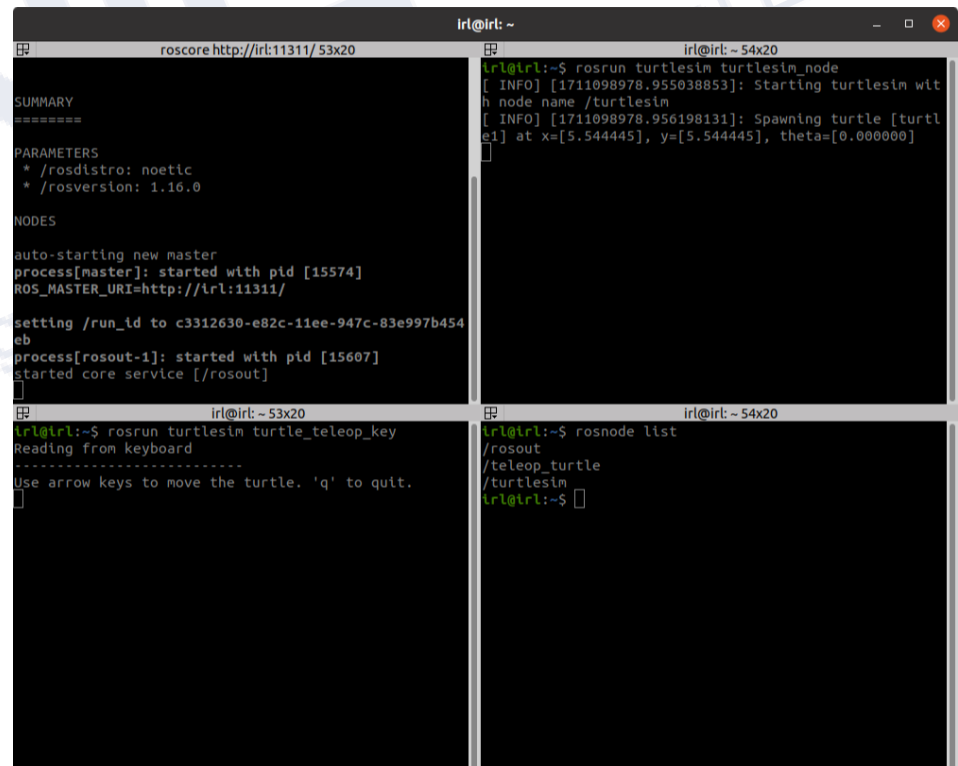
- **(sudo) apt**: 패키지 관리 명령어. update, upgrade, install, remove, purge, policy 등 추가 명령어를 붙여 사용
- **(sudo) chmod**: 파일/폴더 권한을 설정하는 명령어. 뒤에 8진법 숫자나 옵션을 통해 권한을 부여
- **(sudo) chown**: 파일/폴더의 소유권을 설정하는 명령어. 계정명(UID), 그룹명(GID), 파일 또는 디렉터리 형태로 사용
- **ls**: 디렉토리 목록 확인
- **cd**: 디렉토리 이동
- **mkdir**: 새 디렉토리 생성
- **rm**: 파일 또는 디렉토리 삭제
- **cp**: 파일 또는 디렉토리 복사
- **mv**: 파일 또는 디렉토리 이동 (이름을 변경할 때에도 사용)

3. Linux

4) 주요 명령어

터미널 확장 : Terminator

- Ctrl + Alt + T : 터미널 창 생성
- Ctrl + Shift + W : 현재 터미널 종료
- Ctrl + Shift + E : 수직 분할 (좌우)
- Ctrl + Shift + O : 수평 분할 (상하)
- Ctrl + Shift + 방향키 : 터미널 크기 조절
- Ctrl + Shift + F : 터미널 내 검색
- Alt + 방향키 : 터미널 이동
- F11 : 전체 화면



```

roscore http://irl:11311/53x20
SUMMARY
=====
PARAMETERS
 * /roscpp: noetic
 * /roscpp: 1.16.0
NODES
auto-starting new master
process[master]: started with pid [15574]
ROS_MASTER_URI=http://irl:11311/

setting /run_id to c3312630-e82c-11ee-947c-83e997b454
eb
process[roscpp-1]: started with pid [15607]
started core service [/roscpp]

irl@irl: ~ 53x20
irl@irl:~$ roscpp turtle_teleop_key
Reading from keyboard
-----
Use arrow keys to move the turtle. 'q' to quit.
irl@irl:~$

irl@irl: ~ 54x20
irl@irl:~$ roscpp turtle_node
[ INFO] [1711098978.955038853]: Starting turtle node
[ INFO] [1711098978.956198131]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
irl@irl:~$

irl@irl:~$ rosnode list
/roscpp
/turtle_node
/turtle1
irl@irl:~$
  
```

ROS

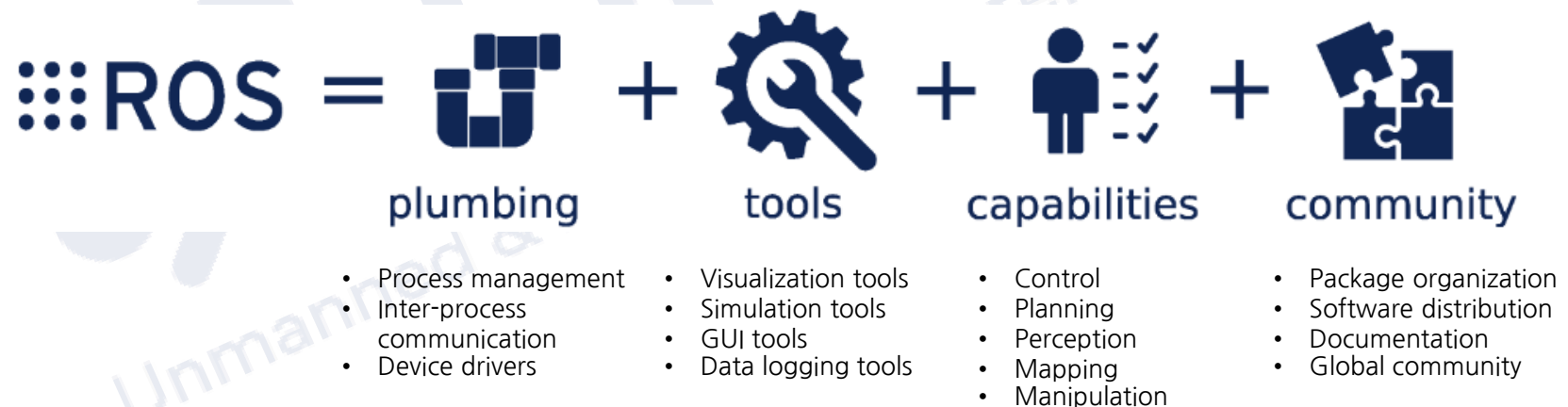


4. ROS

1) ROS 개요

● ROS1

- ROS is an **open-source, meta-operating system** for your robot.
- Building an ecosystem to enable robotics software development to collaborate at a global level
- Make it as easy to build robot software
- To support code reuse in robotics research and development.
- **Goal: sharing and collaboration**



4. ROS

1) ROS 개요

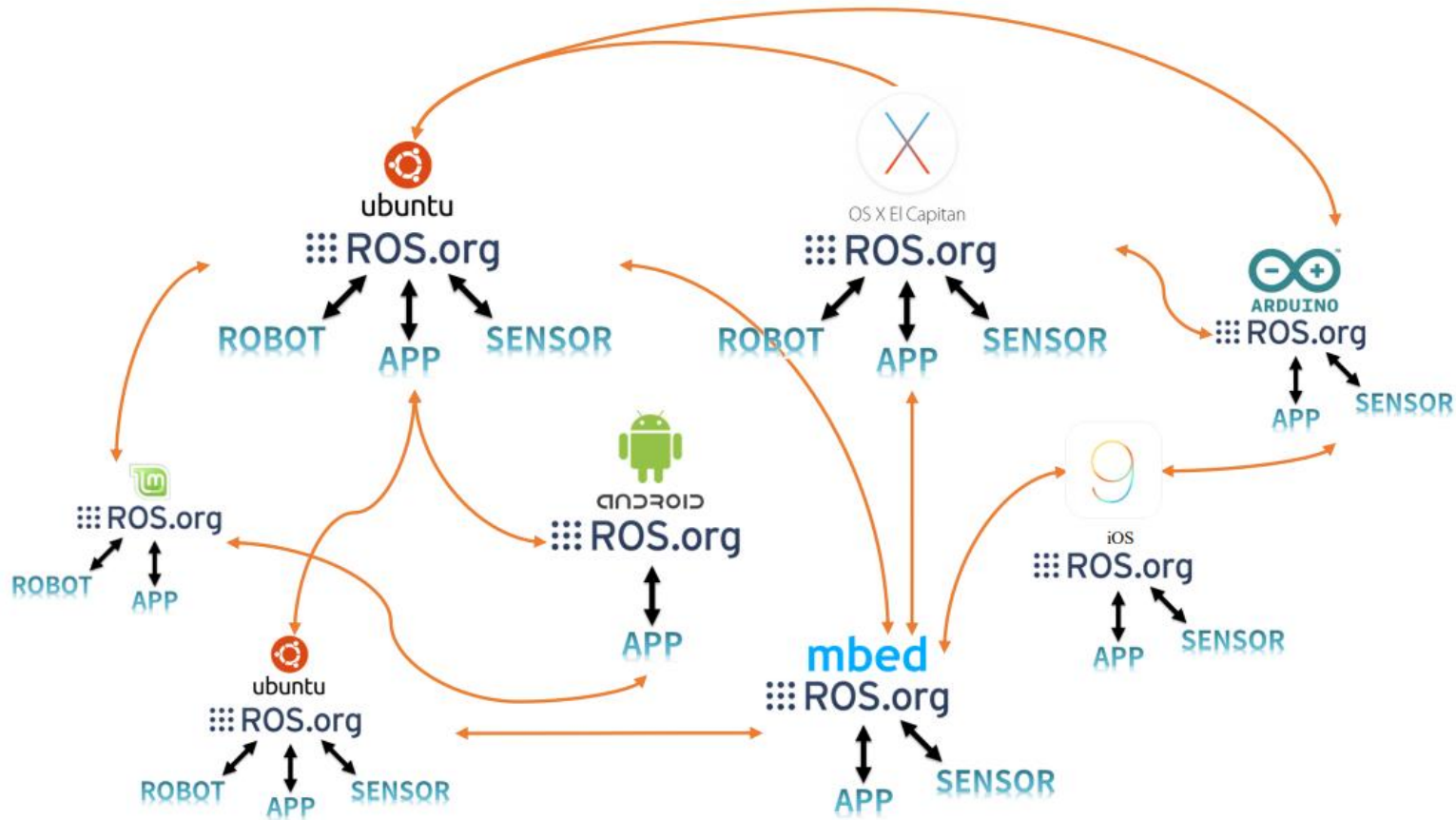
- 메타 운영체제 (Meta-Operating System)



4. ROS

1) ROS 개요

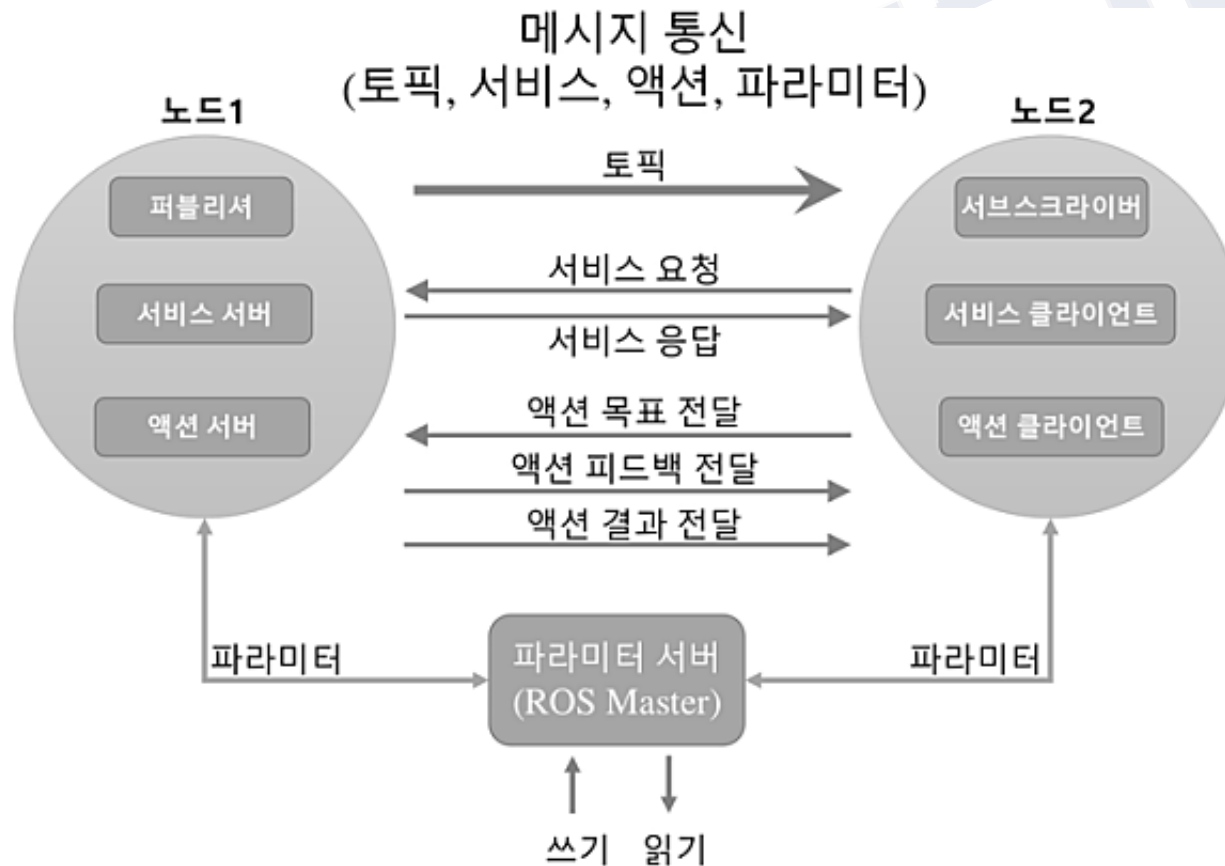
- 다양한 이기종 다바이스 간의 통신 지원



4. ROS

1) ROS 개요

- ROS1 통신 방식



4. ROS

1) ROS 개요

- ROS1의 문제점?

- Multiple robots
- Small embedded platforms
- Cross platform support
- Real-time systems
- Non-ideal networks
- Safety
- Security
- Production environments
- Prescribed patterns for building and structuring systems

API changes

Risk associated with changing the current ROS system

4. ROS

1) ROS 개요

- ROS2

- DDS
- Support for small embedded system
- Real-time control & deterministic execution & real-time communication
- Shorten time to market
- Designed for production
- Multi-platform
- Multi-domain
- No vendor lock-in
- Built on open standards
- Permissive open source license
- Global community
- Industry support
- Interoperability with ROS1

※ DDS

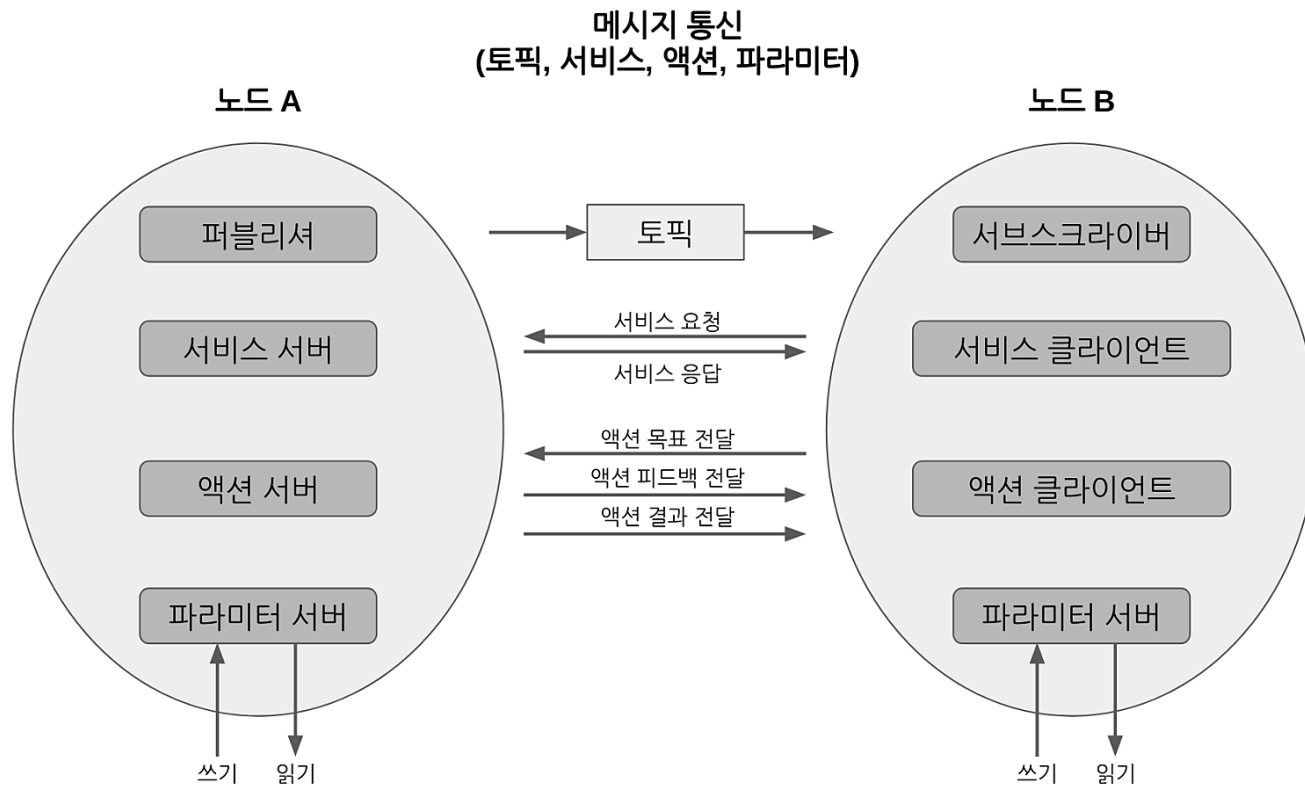
OMG(Object Management Group)에서 표준화한 실시간 publisher-subscriber 방식 통신 미들웨어



4. ROS

1) ROS 개요

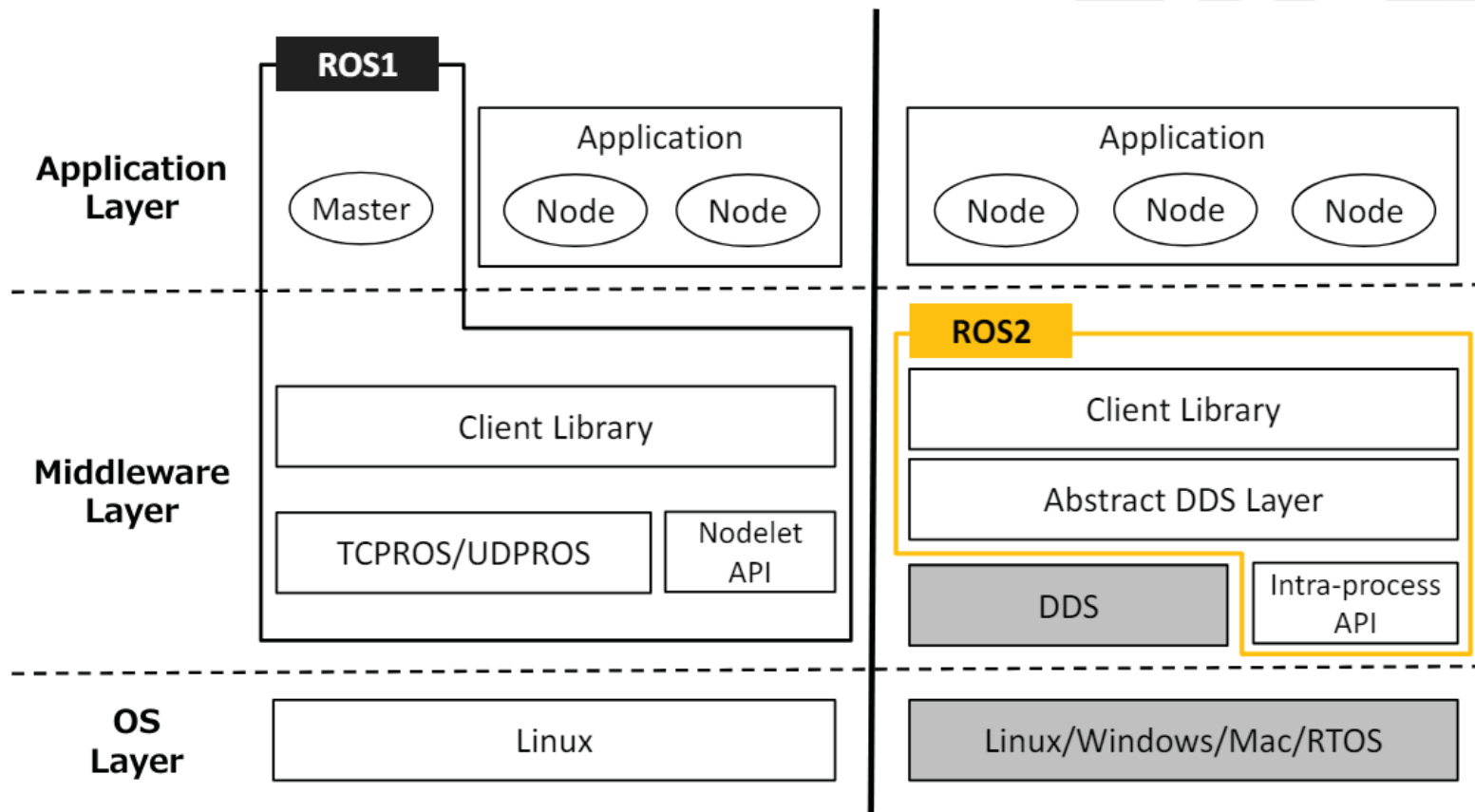
- ROS2 통신 방식



4. ROS

1) ROS 개요

- ROS1 vs ROS2



4. ROS

1) ROS 개요

- ROS1 vs ROS2

Features	ROS 1	ROS 2
Platforms	Linux	Linux, macOS, Windows
Languages	C++03, Python 2.7	C++14 (C++17), Python 3.5+
Communication	XMLRPC + TCPROS	DDS (RTPS)
Node Manager	ROS Master	No, use DDS's auto-discovery
Build System	roscpp → catkin (CMake)	ament (CMake), Python setuptools (Full support)
Build Tools	catkin_make, catkin_tools	colcon
Build Options	-	Multiple workspace, No non-isolated build, No devel space
Version Control System	roscpp → wstool, roscppinstall (*.roscppinstall)	vcstool (*.repos)
Multiple Nodes	one node in a process	multiple nodes in a process
Messages	*.msg, *.srv, *.action	*.msg, *.srv, *.action, *.idl
roslaunch	XML	Python, XML, YAML
Security	SROS	SROS 2, DDS-Security, Robotic Systems Threat Model
Real-time	external frameworks like OROCOS	real-time nodes when using a proper RTOS with carefully written user code
Graph API	remapping at startup time only	remapping at runtime
Embedded Systems	roscppserial, mROS	microROS, ros2arduino, H-ROS, Renesas DDS-XRCE(Micro-XRCE-DDS), AWS ARCLM

4. ROS

1) ROS 개요

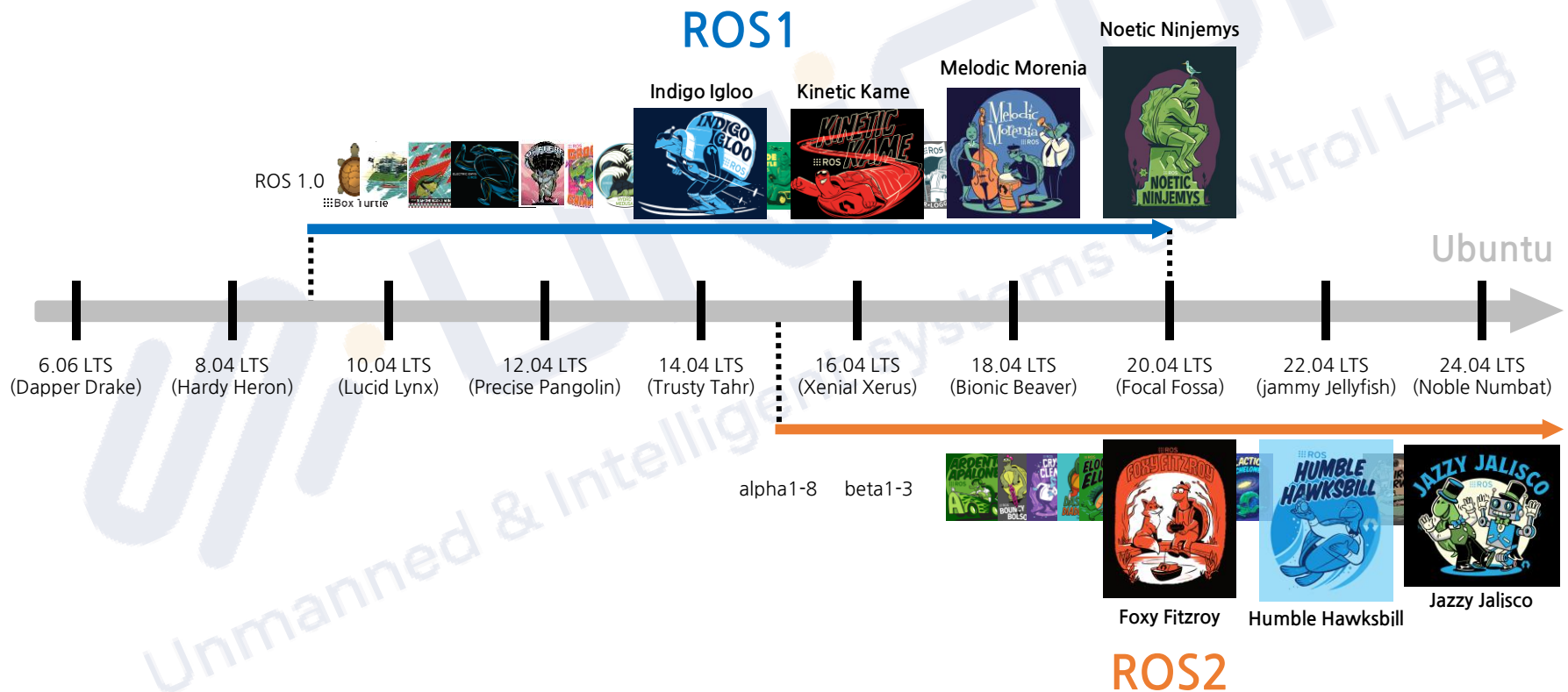
- ROS1 vs ROS2

ROS1	• 단일 로봇 • 워크스테이션급 컴퓨터 • Linux 환경 • 실시간 제어 지원하지 않음 • 안정된 네트워크 환경이 요구됨 • 주로 대학이나 연구소와 같은 아카데미 연구 용도
ROS2	• 복수대의 로봇 • 임베디드 시스템에서의 ROS 사용 • 실시간 제어 • 불안정한 네트워크 환경에서도 동작할 수 있는 유연함 • 멀티 플랫폼 (Linux, macOS, Windows) • 최신 기술 지원 (Zeroconf, Protocol Buffers, ZeroMQ, WebSockets, DDS 등) • 상업용 제품 지원

4. ROS

1) ROS 개요

- ROS1 vs ROS2

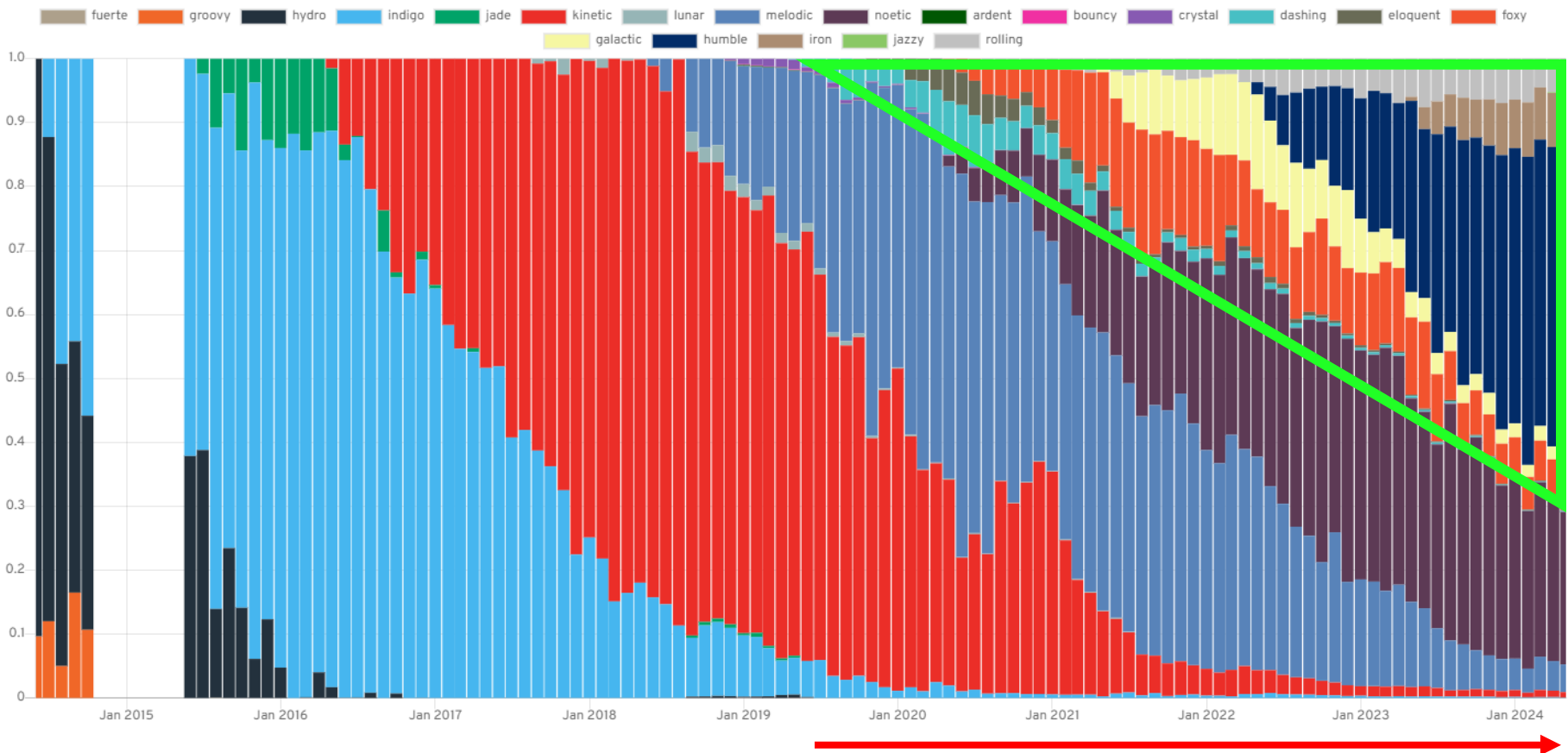


4. ROS

1) ROS 개요

- ROS1 vs ROS2

ROS Distro Usage by packages.ros.org traffic



ROS2 사용률 증가

4. ROS

1) ROS 개요

- Why ROS1?

- 입문자용으로 적합. ROS2 보다는 ROS1이 상대적으로 진입장벽이 낮음
- ROS1을 경험해보고 ROS2로 넘어가면 ROS의 전체적인 구조에 대한 이해도를 훨씬 크게 가져갈 수 있음
- 아직 ROS1 기반으로 개발하는 프로젝트가 많음
- 수많은 공식 및 비공식 패키지, 연습 예제 등



Ubuntu 20.04 LTS

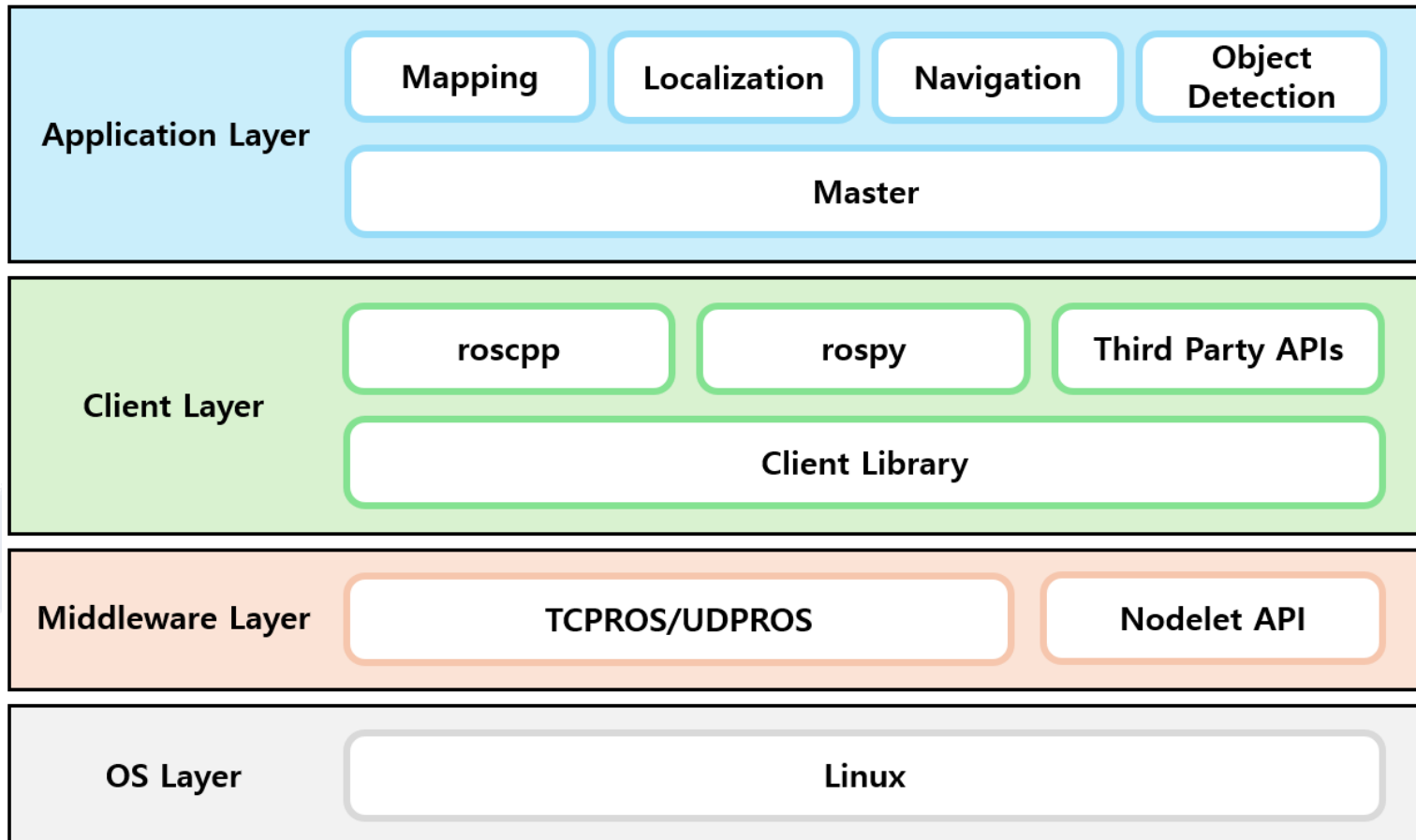


ROS1 Noetic Ninjemys

4. ROS

1) ROS 개요

- System Architecture



4. ROS

1) ROS 개요

- ROS wiki 활용

[ROS wiki link](https://docs.ros.org/en/jazzy/index.html)



Documentation

ROS (Robot Operating System) provides libraries and tools to help software developers create robot applications. It provides hardware abstraction, device drivers, libraries, visualizers, message-passing, package management, and more. ROS is licensed under an open source, BSD license.

Available Translations: [German](#) | [Spanish](#) | [French](#) | [Italian](#) | [Japanese](#) | [Korean](#) | [Brazilian Portuguese](#) | [Portuguese](#) | [Русский \(Russian\)](#) | [Thai](#) | [Turkish](#) | [简体中文](#) | [Ukrainian](#) | [Vietnamese](#) | [العربية](#)

ROS:

Introduction

An introduction to what is ROS.

Install

Install ROS on your machine.

Getting Started

Learn about various concepts, client libraries, and technical overview of ROS.

Tutorials

Step-by-step instructions for learning ROS hands-on

Contribute

How to get involved with the ROS community, such as submitting your own repository.

Support

What to do if something doesn't work as expected.

Quality Assurance

How to ensure that your ROS-based systems and your contributions to ROS are of high quality.

Software:

Distributions

View the different release Distributions for ROS.

● Packages

Search the 2000+ software libraries available for ROS.

Core Libraries

APIs by language and topic.

Common Tools

Common tools for developing and debugging ROS software.

Robots/Hardware:

Robots

Robots that you can use with ROS.

Sensors

Sensor drivers for ROS.

Motors

Motor controller drivers for ROS.

>> ROS 2 Documentation: <https://docs.ros.org/en/jazzy/index.html>

4. ROS

2) ROS Noetic 설치

- Ubuntu install of ROS Noetic

[ROS wiki noetic installation link](#)



4. ROS

2) ROS Noetic 설치

- Turtlesim을 활용한 ROS 동작 테스트

Terminal 1

```
roscore
```

Terminal 2

```
roslaunch turtlesim turtlesim_node
```

Terminal 3

```
roslaunch turtlesim turtle_teleop_key
```



키보드의 방향키를 이용하여 거북이를 움직이고, 오류 없이 잘 동작된다면 ROS Noetic 버전이 잘 설치되었다는 것을 의미

4. ROS

3) ROS1 기본 용어

- Master

- 노드 간 연결 및 메시지 통신을 위한 네임 서버 역할
- 마스터를 실행하면 각 노드의 이름을 등록하고 필요할 때 정보를 얻음
- 노드 간 연결과 토픽, 서비스 등 메시지 통신은 마스터 없이는 불가능

- Node

- ROS에서 실행되는 가장 작은 단위의 프로세서
- 예를 들어 이동 로봇의 경우, 센서 구동, 센서 데이터 변환, 장애물 인식, 모터 구동, 네비게이션 등의 기능별로 노드를 나눔

- Message

- 메시지를 통해 노드 간에 데이터를 송수신함
- Float, int, bool, string 등의 변수

4. ROS

3) ROS1 기본 용어

- Topic

- ROS에서 가장 많이 사용하는 메시지의 종류
- Publisher 노드는 자신의 토픽을 마스터에 등록한 후 해당 토픽에 대한 메시지를 송신
- Subscriber 노드는 수신하고자 하는 메시지를 송신하는 publisher에 대한 정보를 마스터에 요청하고, 이를 바탕으로 publisher 노드와 연결하여 토픽으로 메시지를 교환

- Publisher

- 토픽 메시지를 송신하는 것을 publish라 하며, 그 역할을 publisher가 수행
- 한 노드에서 여러 publisher가 선언될 수 있음

- Subscriber

- 토픽 메시지를 수신하는 것을 subscribe라 하며, 그 역할을 subscriber가 수행
- 한 노드에서 여러 subscriber가 선언될 수 있음

4. ROS

4) ROS1 기본 명령어

- **roscore**
 - ROS 마스터를 실행하는 명령어
 - 한 네트워크 환경 내에서 하나의 roscore만 실행되어야 함
- **roslaunch [패키지명] [노드명]**
 - ROS 패키지의 단일 노드를 실행시키는 명령어
- **roslaunch [패키지명] [런치파일명]**
 - 여러 노드를 한번에 실행시키는 명령어
 - 패키지 파라미터나 노드 이름 변경, 노드의 네임스페이스 구성, 환경 변수 설정 등의 부가 기능을 갖춘 노드 실행에 특화
 - '*.launch' 파일을 따로 작성하여 구성

4) ROS1 기본 명령어

- 83

4. ROS

5) 개발 환경 구축

- IDE (Integrated Development Environment)

VS code (Visual Studio Code) 추천

- linux, windows, macOS 모두 사용 가능한 크로스 플랫폼 에디터
- Extension을 통한 다양한 프로그래밍 언어 지원
- **Install:** <https://code.visualstudio.com/>
- **Extension:** C/C++, Cmake Tools, Python, ROS, URDF, Visual Studio IntelliCode, Project Manager, Code runner, Material Theme, ...



ROS Basic

5. ROS Basic

1) ROS의 주요 특징

● 통신 기능

- 노드 간 데이터 통신을 제공
- 통상적 미들웨어로 지칭되는 메시지 전달 인터페이스 지원
- **메시지 파싱 기능**
 - 로봇 개발 시에 빈번히 사용되는 통신 시스템 제공
 - 캡슐화 및 코드 재사용을 촉진하는 노드들 간의 메시지 전달 인터페이스
- **메시지의 기록 및 재생**
 - 노드 간 송/수신되는 데이터인 메시지를 저장하고 필요시에 재사용 가능
 - 저장된 메시지를 기반으로 반복적인 실험 가능, 알고리즘 개발에 용이함
- **메시지 사용으로 인한 다양한 프로그래밍 언어 사용 가능**
 - 노드 간의 데이터 교환이 메시지를 사용하기 때문에 각 노드는 서로 다른 언어로 작성 가능
 - 클라이언트 라이브러리: roscpp, rospy, roslisp, rosjava, roslua, roscs, roseus, PhaROS, rosR
- **분산 매개 변수 시스템**
 - 시스템에서 사용되는 변수를 글로벌 키값으로 작성하여 공유 및 수정하여 실시간으로 반영

5. ROS Basic

1) ROS의 주요 특징

● 로봇 관련 다양한 기능

■ 로봇에 대한 표준 메시지 정의

- 카메라, IMU, 레이저 등의 센서, 오도메트리, 경로 및 지도 등의 내비게이션 데이터 등의 표준 메시지를 정의하여 모듈화, 협업 작업을 유도, 효율성 향상

■ 로봇 기하학 라이브러리

- 로봇, 센서 등의 상대적 좌표를 트리화 시키는 TF(Transformation) 제공

■ 로봇 기술 언어

- 로봇의 물리적 특성을 설명하는 XML 문서 기술

■ 진단 시스템

- 로봇의 상태를 한눈에 파악할 수 있는 진단 시스템 제공

■ 센싱/인식

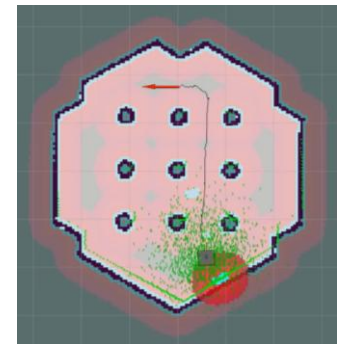
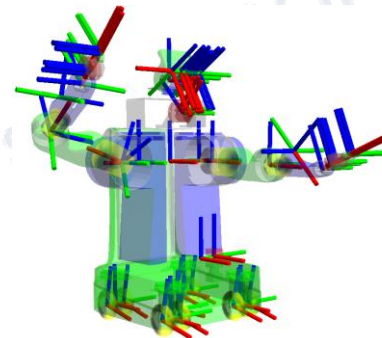
- 센서 드라이버, 센싱/인식 레벨의 라이브러리 제공

■ 내비게이션

- 로봇에서 많이 사용되는 로봇의 포즈(위치/자세) 추정, 지도내의 자기 위치 추정 제공
- 지도 작성에 필요한 SLAM, 작성된 지도 내에서 목적지를 찾아가는 Navigation 라이브러리를 제공

■ 매니플레이션

- 로봇 암에 사용되는 IK, FK는 물론 응용단의 Pick and Place를 지원하는 다양한 Manipulation 라이브러리 제공
- GUI 형태의 Manipulation tools 제공 (MoveIt!)



5. ROS Basic

1) ROS의 주요 특징

● 다양한 개발 도구

- 로봇 개발에 필요한 다양한 개발 도구를 제공
- 로봇 개발의 효율성 향상

▪ Command-Line Tools

- GUI 없이 ROS에서 제공되는 명령어로만 로봇 접근 및 거의 모든 ROS 기능 소화

▪ RViz

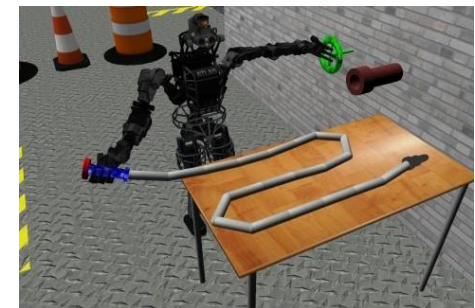
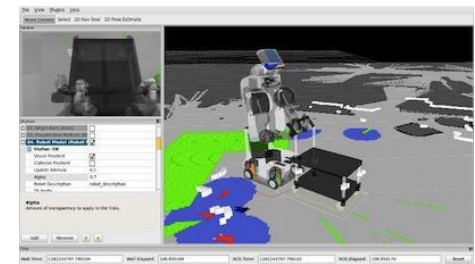
- 강력한 3D 시각화툴 제공
- 레이저, 카메라 등의 센서 데이터를 시각화
- 로봇 외형과 계획된 동작을 표현

▪ RQT

- 그래픽 인터페이스 개발을 위한 Qt 기반 프레임 워크 제공
- 노드와 그들 사이의 연결 정보 표시 (rqt_graph)
- 인코더, 전압, 또는 시간이 지남에 따라 변화하는 숫자를 플로팅 (rqt_plot)
- 데이터를 메시지 형태로 기록하고 재생 (rqt_bag)

▪ Gazebo

- 물리 엔진을 탑재, 로봇, 센서, 환경 모델 등을 지원 (3D simulator)
- ROS와의 높은 호환성



5. ROS Basic

2) 메시지 통신

- 메시지 종류

Type	Features		Description
Topic	Asynchronous	Unidirectional	Used when exchanging data continuously
Service	Synchronous	Bi-directional	Used when request processing requests and responds current states
Action	Asynchronous	Bi-directional	Used when it is difficult to use the service due to long response times after the request or when an intermediate feedback value is needed

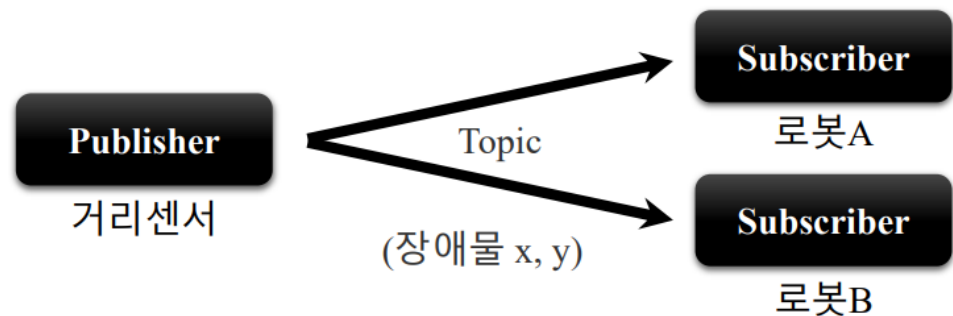
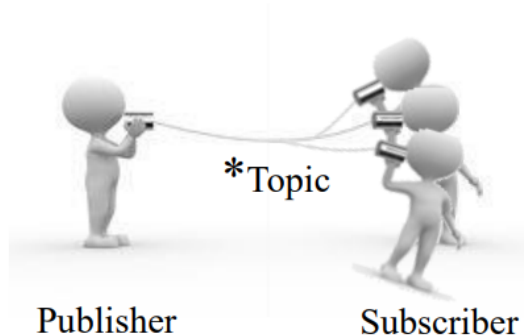
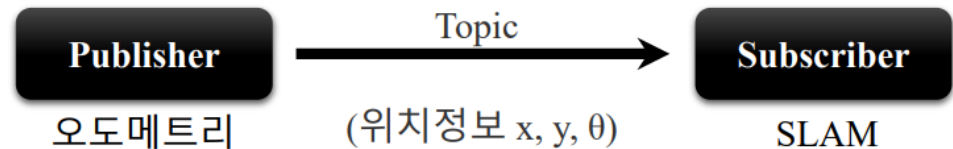
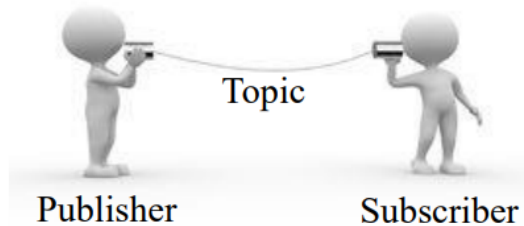
- **Topic** : Uni-directional message publish/subscribe **99%**
- **Service** : Bi-directional message request/response 0.9%
- **Action** : Bi-directional message goal/feedback/result 0.1%

5. ROS Basic

2) 메시지 통신

● Topic

- Publisher 노드가 특정 데이터타입의 topic을 발행하면 Subscriber 노드가 해당 토픽을 수신
- 단방향 통신이며, 지속적으로 메시지를 송수신하는 센서 데이터에 적합



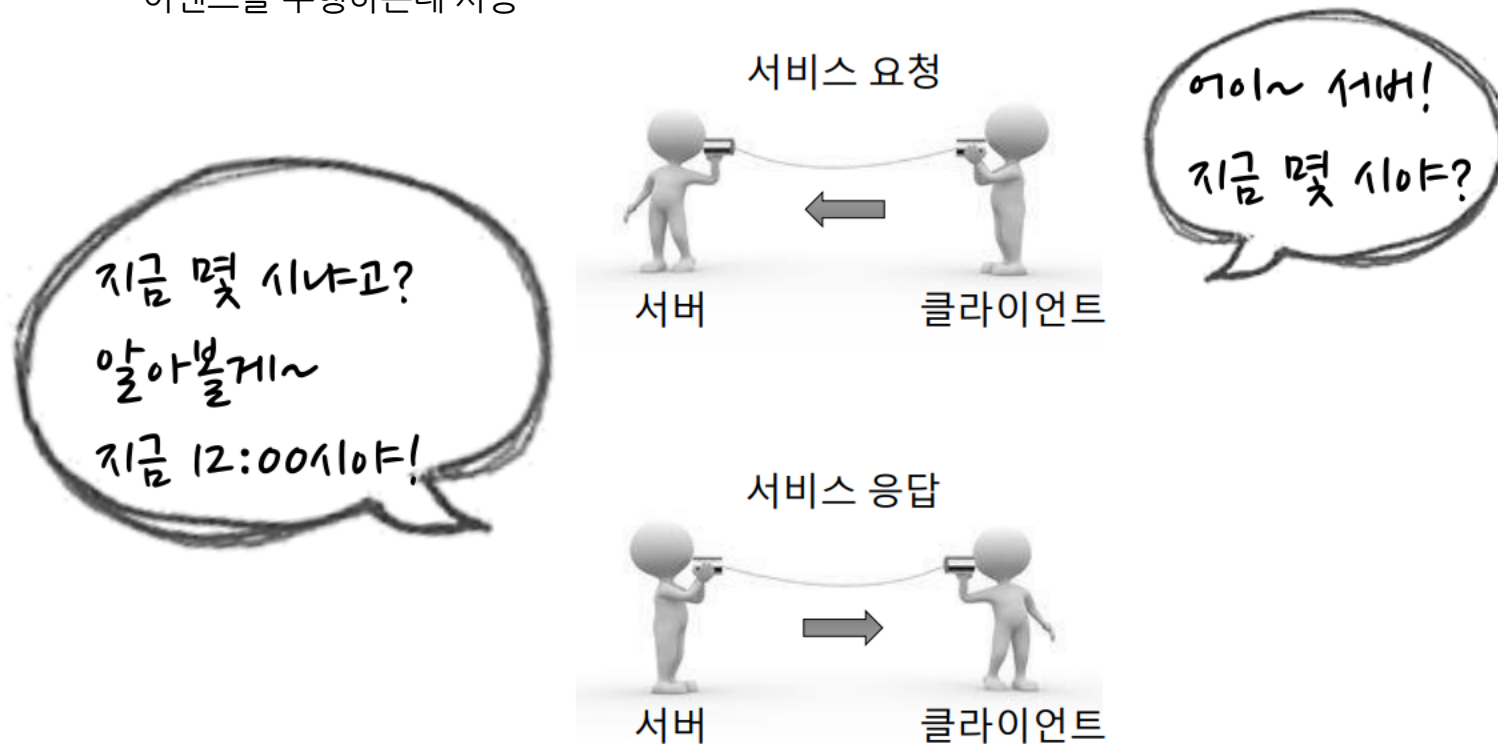
*Topic 에 대해 1:1의 Publisher, Subscriber 통신도 가능하며, 목적에 따라서 1:N, N:1, N:N 통신도 가능하다.

5. ROS Basic

2) 메시지 통신

● Service

- 서비스는 Server가 Client의 요청(request)에 응답(response)하는 양방향 통신을 의미
- 일회성 메시지 통신이며, Server와 Client는 동기화 되어 있다는 것이 특징
- 연결을 지속적으로 유지하지 않기 때문에 토픽을 대체하여 네트워크 부하를 줄이거나 특정 조건에서 특정 이벤트를 수행하는데 사용

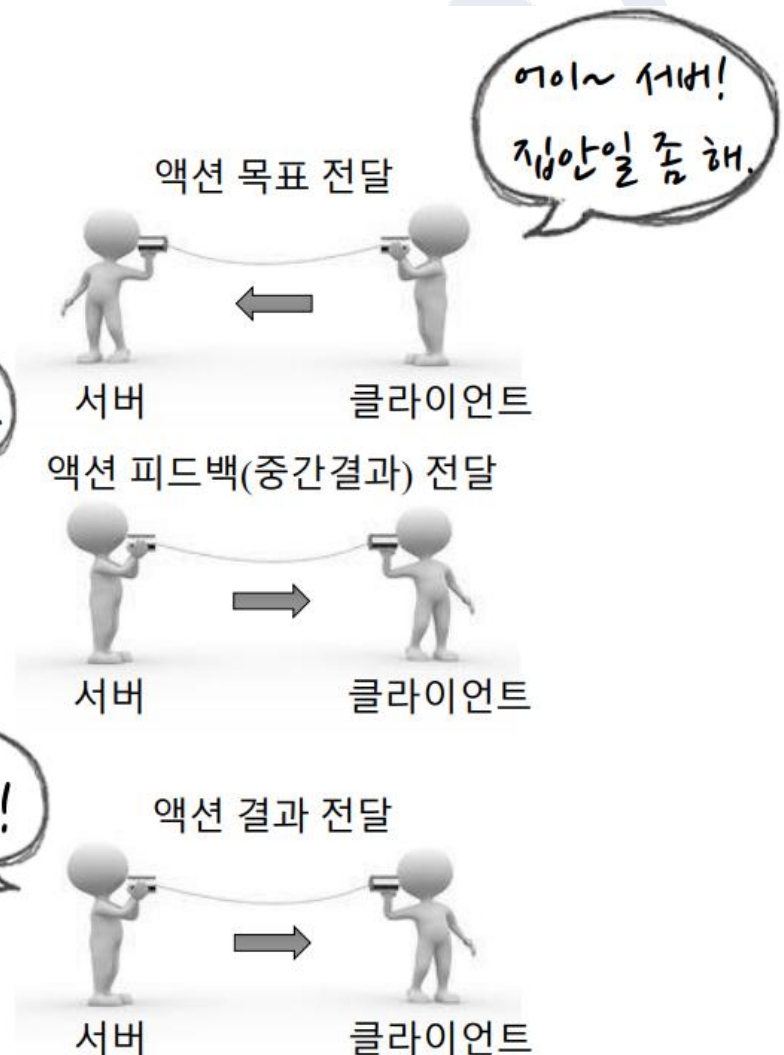


5. ROS Basic

2) 메시지 통신

● Action

- 액션은 Client가 목표(goal)를 전달하면 Server가 Client에게 중간 과정을 피드백(feedback)하며, 목표에 도달했을 때 최종 메시지(result)를 전달
- 진행 중인 목표를 취소하는 등의 복잡한 명령을 수행하는데 사용



5. ROS Basic

2) 메시지 통신

- Parameter

- 메시지 통신은 크게 topic, service, action으로 나눌 수 있음
- ROS 파라미터는 하나의 ros master 내에서 사용되는 전역 변수로 메시지 통신 방식을 이용함
- 할당된 파라미터는 기본값으로 설정되며, 필요시 값을 지정하여 사용함
- 노드를 실행 중인 상태에서도 파라미터 설정 기능을 통해 기존에 지정된 값을 변경 가능
- 노드 외부에서 노드의 흐름이나 설정, 처리 방식 등을 바꿀 수 있음
- 변화하는 환경에 유연하게 대처하는데 매우 유용함

5. ROS Basic

2) 메시지 통신

- Message data type

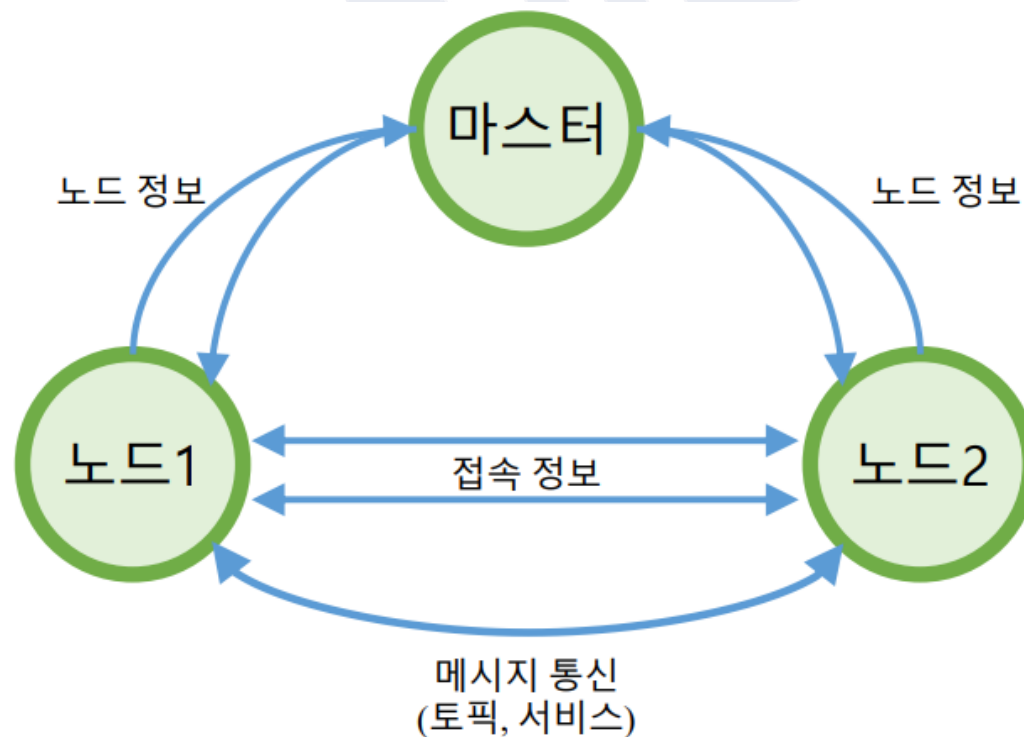
- 메시지는 항상 각각의 목적에 부합하는 데이터 형태를 가지고 있음
- 다양한 data type이 존재 (활용할 수 있는 자원이 매우 많음)
 - <http://wiki.ros.org/msg>
 - http://wiki.ros.org/common_msgs
- 대표적인 메시지 패키지
 - std_msgs (단순 자료형 메시지) : https://wiki.ros.org/std_msgs
 - sensor_msgs (센서 데이터 메시지) : https://wiki.ros.org/sensor_msgs
- 메시지 안에 메시지를 포함하는 데이터 구조 (구조체와 유사)
 - ex) geometry_msgs/PoseStamped
 - http://docs.ros.org/api/geometry_msgs/html/msg/PoseStamped.html
- 메시지들이 나열된 배열과 같은 구조
 - ex) float32[] ranges
 - ex) sensor_msgs/LaserScan
 - http://docs.ros.org/api/sensor_msgs/html/msg/LaserScan.html

5. ROS Basic

2) 메시지 통신

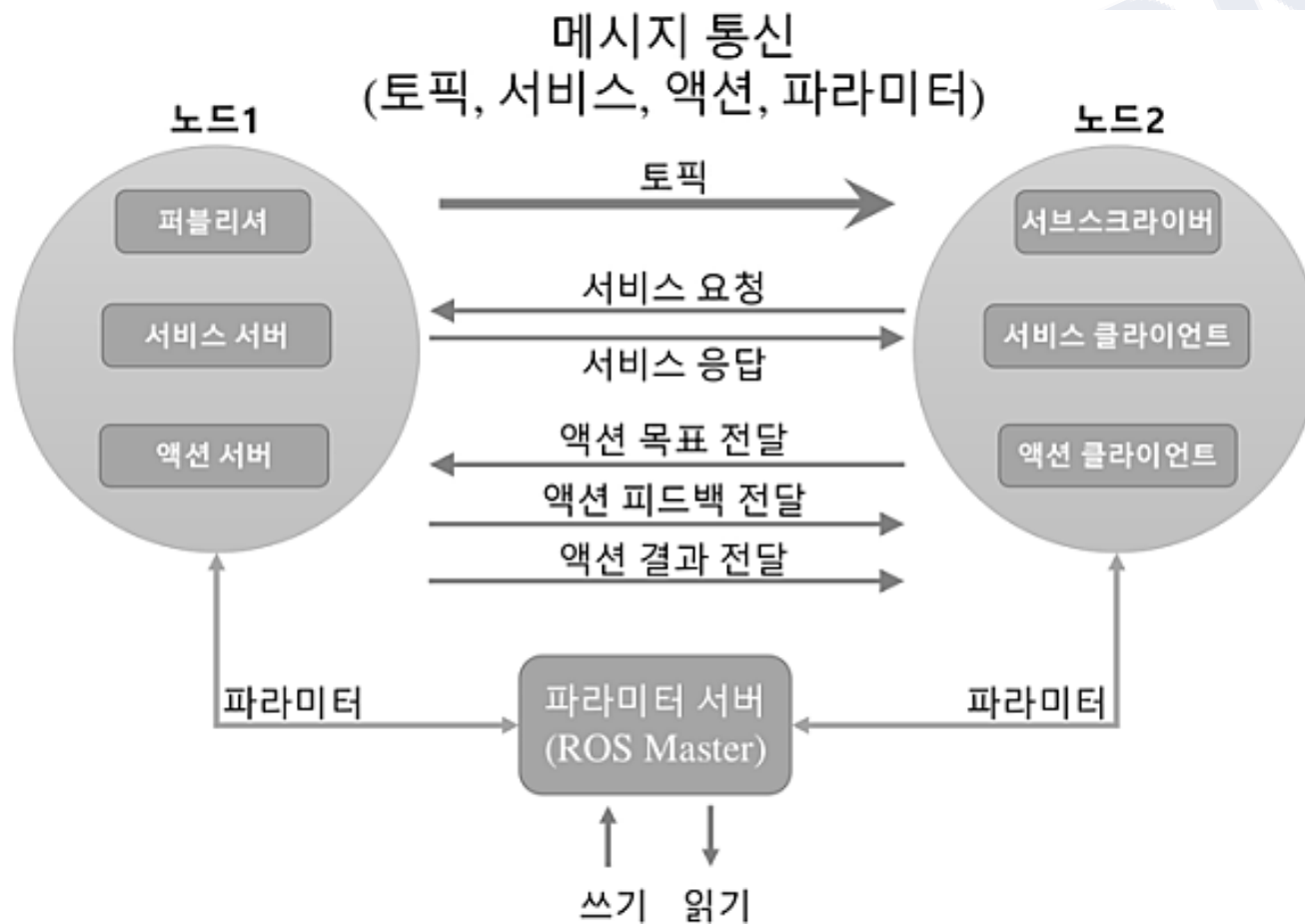
- ROS 통신의 이해

- 마스터 구동 후, publisher 또는 subscriber 노드의 정보 등록
- 마스터는 subscriber 노드에게 publisher 노드의 정보를 전달
- Publisher와 subscriber의 노드 간에 접속 요청 및 응답을 한 후 메시지 전송
- 토픽방식은 접속이 끊어지기 전까지 지속적으로 메시지를 전송하지만 서비스에서는 1회만 접속



5. ROS Basic

2) 메시지 통신



5. ROS Basic

3) Turtlesim 실습

Terminal 1

```
$ roscore
```

Terminal 2

```
$ rosrun turtlesim turtlesim_node
```

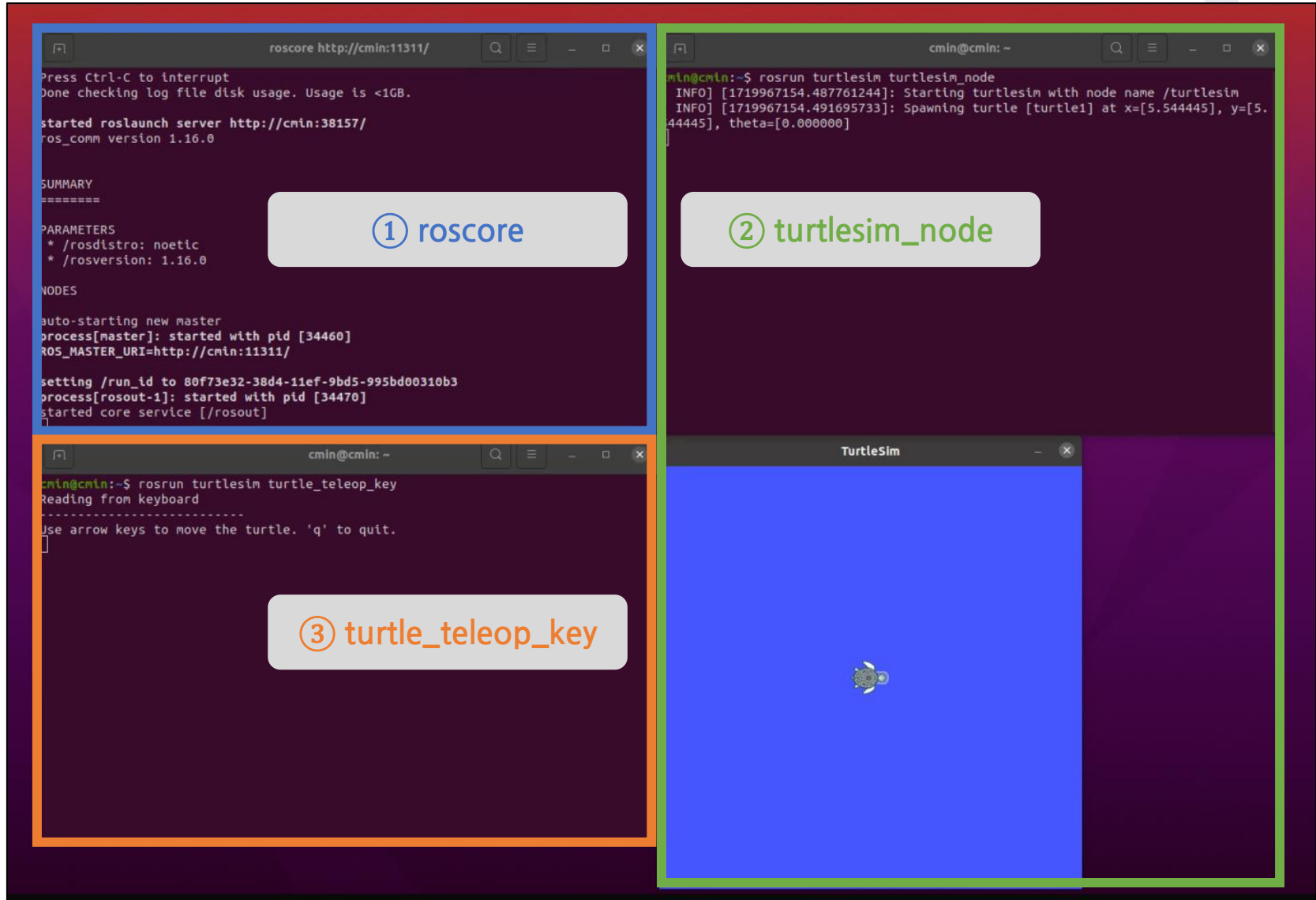
Terminal 3

```
$ rosrun turtlesim turtle_teleop_key
```



5. ROS Basic

3) Turtlesim 실습



The image shows a ROS Turtlesim setup with three terminal windows and the TurtleSim GUI.

- Terminal 1 (Top Left):** Running `roscore http://cmin:11311/`. It shows the roscore server starting and listening for connections. A label **① roscore** is overlaid on this window.
- Terminal 2 (Top Right):** Running `roslaunch turtlesim turtlesim_node`. It shows the turtlesim node starting and spawning a turtle named `turtle1` at coordinates `x=[5.544445], y=[5.44445]` with `theta=[0.000000]`. A label **② turtlesim_node** is overlaid on this window.
- Terminal 3 (Bottom Left):** Running `roslaunch turtlesim turtle_teleop_key`. It shows the turtle teleop key interface starting and waiting for keyboard input. A label **③ turtle_teleop_key** is overlaid on this window.
- TurtleSim GUI (Bottom Right):** A window titled "TurtleSim" showing a blue square environment with a small turtle icon in the center.

5. ROS Basic

3) Turtlesim 실습

- ROS node

```
$ rosnod list
```

```
cmn@cmn:~$ rosnod list
/rosout
/teleop_turtle
/turtlesim
```

```
$ rosnod info /turtlesim
```

```
cmn@cmn:~$ rosnod info /turtlesim
-----
Node [/turtlesim]
Publications:
* /rosout [rosgaph_msgs/Log]
* /turtle1/color_sensor [turtlesim/Color]
* /turtle1/pose [turtlesim/Pose]

Subscriptions:
* /turtle1/cmd_vel [geometry_msgs/Twist]
```

More information ...

```
$ rosnod -h
```

5. ROS Basic

3) Turtlesim 실습

- ROS topic

```
$ rostopic list
```

```
cmin@cmin:~$ rostopic list  
/rosout  
/rosout_agg  
/turtle1/cmd_vel  
/turtle1/color_sensor  
/turtle1/pose
```

```
$ rostopic info /turtle1/cmd_vel
```

```
cmin@cmin:~$ rostopic info /turtle1/cmd_vel  
Type: geometry_msgs/Twist  
  
Publishers:  
* /teleop_turtle (http://cmin:37847/)  
  
Subscribers:  
* /turtlesim (http://cmin:35987/)
```


5. ROS Basic

3) Turtlesim 실습

- ROS topic

```
$ rostopic echo /turtle1/cmd_vel
```

```
cm1n@cm1n:~$ rostopic echo /turtle1/cmd_vel
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

5. ROS Basic

3) Turtlesim 실습

- ROS topic

```
$ rostopic pub [option] [topic_name] [message_type] [message]
```

```
$ rostopic pub -r 10 /turtle1/cmd_vel geometry_msgs/Twist
"linear:
  x: 1.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"
```



※ 참고 (<https://www.ros.org/reps/rep-0103.html>)

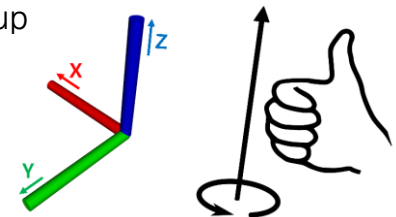
- 표준 단위 (SI 단위)

Quantity	Unit	Quantity	Unit
length	meter	frequency	hertz
angle	radian	force	newton
mass	kilogram	power	watt
time	second	voltage	volt
current	ampere	temperature	celsius

- 좌표 표현 방식

- x : forward, y : left, z : up

- 오른손 법칙



5. ROS Basic

3) Turtlesim 실습

- ROS service

```
$ rosservice list
```

```
cmin@cmin:~$ rosservice list
/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/rostopic_36507_1719923869037/get_loggers
/rostopic_36507_1719923869037/set_logger_level
/spawn
/teleop_turtle/get_loggers
/teleop_turtle/set_logger_level
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
```

```
$ rosservice info /clear
```

```
cmin@cmin:~$ rosservice info /clear
Node: /turtlesim
URI: rosrpc://cmin:47119
Type: std_srvs/Empty
Args:
```

5. ROS Basic

3) Turtlesim 실습

- ROS service

```
$ rosservice call /clear
```

```
$ rosservice call /kill [name]
```

```
$ rosservice call /reset
```

```
$ rosservice call /spawn [x y theta name]
```

5. ROS Basic

3) Turtlesim 실습

- msg & srv

```
$ rosmg info geometry_msgs/Twist
```

```
cmn@cmn:~$ rosmg info geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

```
$ rossrv info turtlesim/Spawn
```

```
cmn@cmn:~$ rossrv info turtlesim/Spawn
float32 x
float32 y
float32 theta
string name
...
string name
```

5. ROS Basic

3) Turtlesim 실습

- ROS parameter

```
$ rosparam list
```

```
cmin@cmin:~$ rosparam list
/rosdistro
/roslaunch/uris/host_cmin__38157
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

```
$ rosparam get /turtlesim/background_b
```

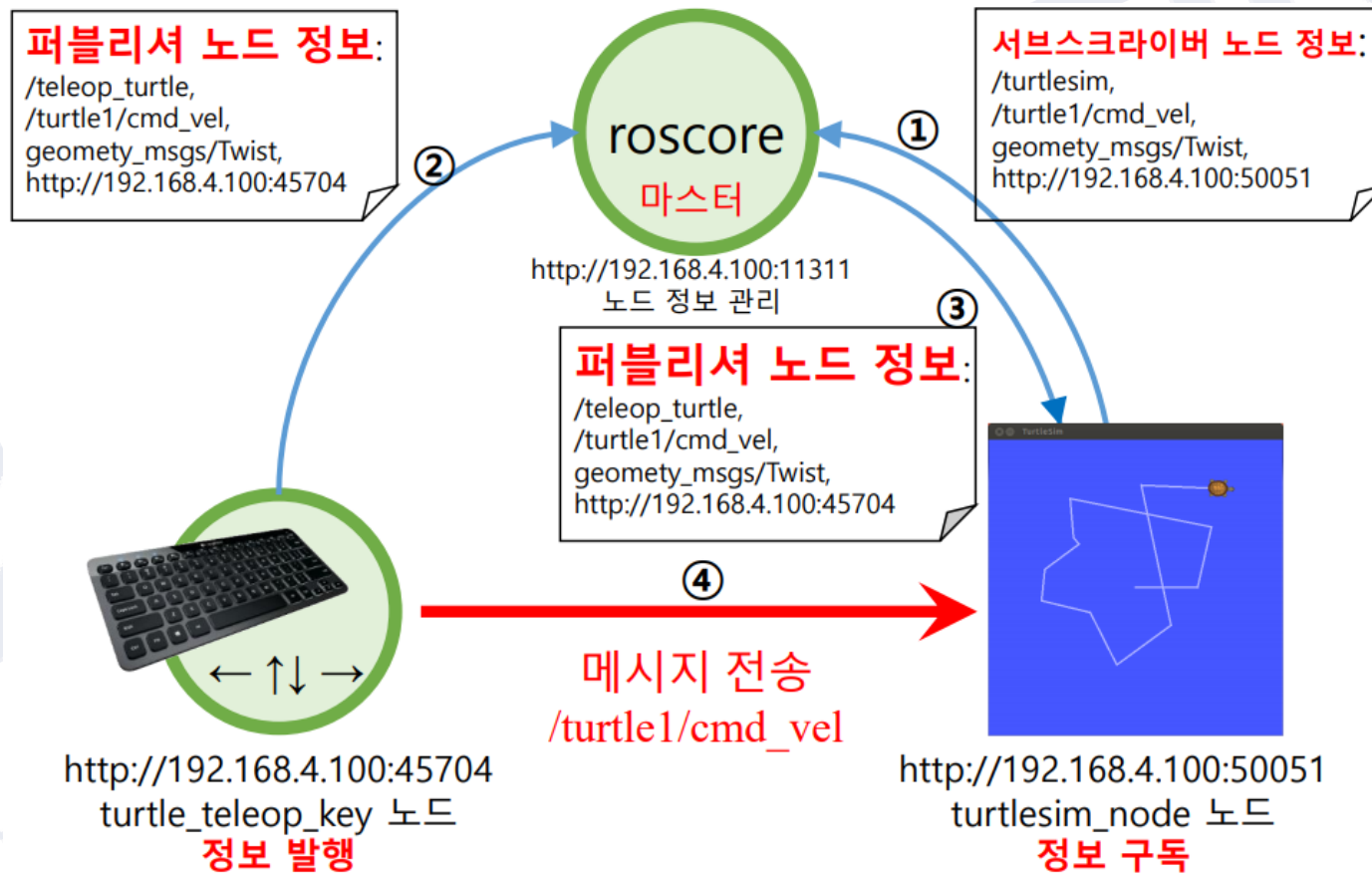
```
cmin@cmin:~$ rosparam get /turtlesim/background_b
255
```

```
$ rosparam set /turtlesim/background_b 150
$ rosservice call /clear
```


5. ROS Basic

3) Turtlesim 실습

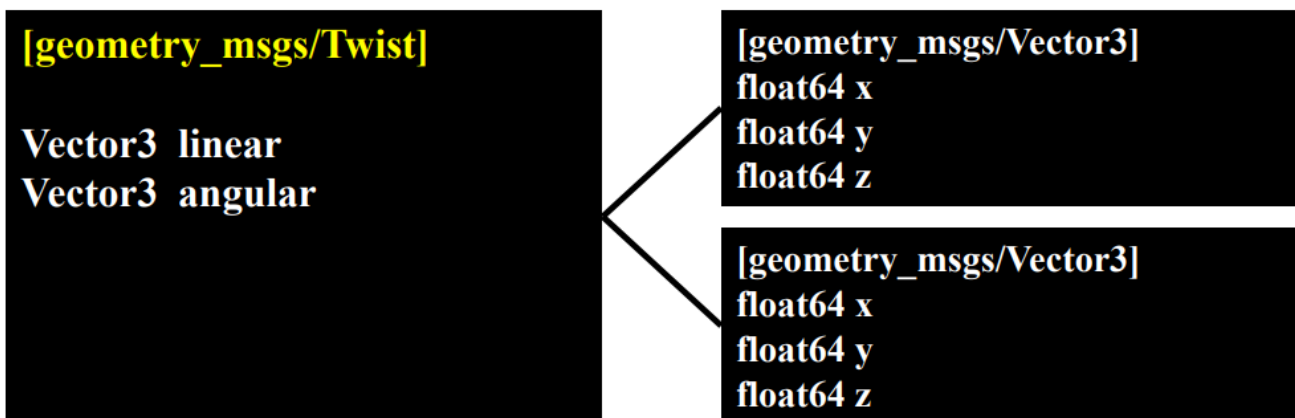
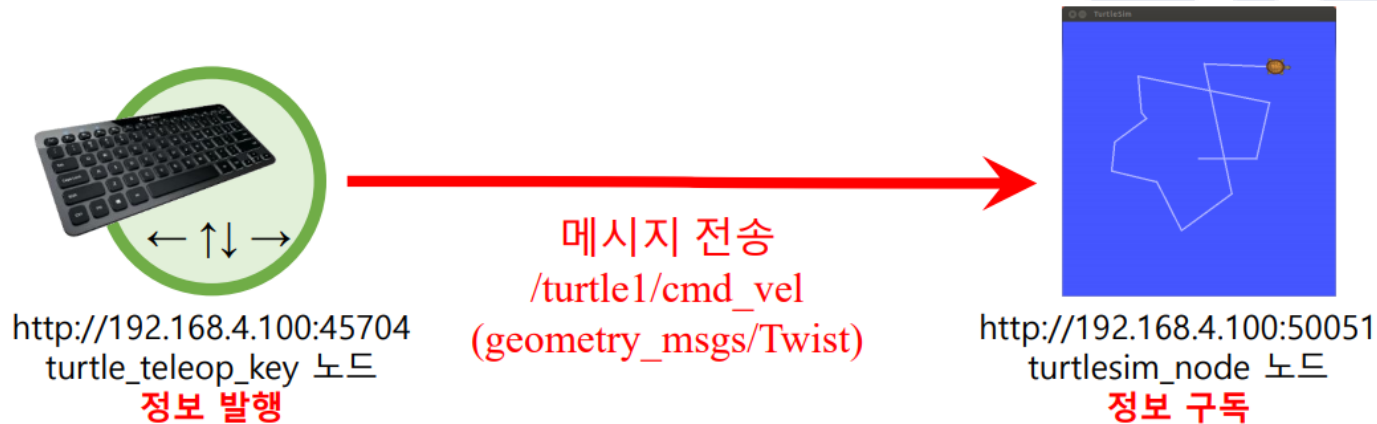
- 메시지 통신 구조



5. ROS Basic

3) Turtlesim 실습

- 메시지 통신 구조



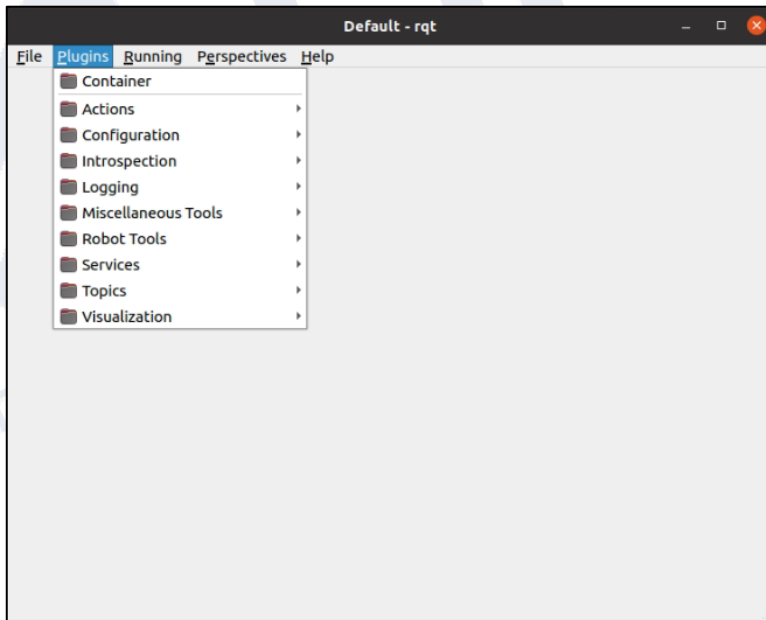
5. ROS Basic

3) Turtlesim 실습

- RQT: GUI tool

- ROS Fuerte 버전부터는 RQT라는 이름으로 기존의 rxbag, rxplot, rxgraph 등이 통합되어 rqt_bag, rqt_plot, rqt_graph 등을 플러그인으로 하는 ROS의 종합 GUI 툴로써 사용 가능해짐
- RQT는 Qt 기반으로 개발되었기 때문에 유저들이 자유롭게 플러그인을 개발하여 추가할 수도 있음
- 대표적인 플러그인: rqt_graph, rqt_plot, rqt_image_view
- 이외에도 다양한 플러그인 존재

```
$ rqt
```



5. ROS Basic

3) Turtlesim 실습

- RQT: GUI tool (Plugins)
 - **Action**
 - **Action Type Browser**: Action 타입의 데이터 구조를 확인
 - **Configuration**
 - **Dynamic Reconfigure**: 노드들에서 제공하는 설정값 변경을 위한 GUI
 - **Launch**: roslaunch의 GUI 버전
 - **Introspection**
 - **Node Graph**: 구동중인 노드들의 관계도 및 메시지의 흐름을 확인 가능한 그래프 뷰
 - **Package Graph**: 노드의 의존 관계를 표시하는 그래프 뷰
 - **Process Monitor**: 실행중인 노드들의 CPU 사용률, 메모리 사용률, 스레드 수 등을 확인
 - **Logging**
 - **Bag**: ROS 데이터 로깅
 - **Console**: 노드들에서 발생하는 경고(Warning), 에러(Error) 등의 메시지를 확인
 - **Logger Level**: ROS의 Debug, Info, Warn, Error, Fatal 로거 정보를 선택하여 표시
 - **Miscellaneous Tools**
 - **Python Console**: 파이썬 콘솔 화면
 - **Shell**: 셸을 구동
 - **Web**: 웹 브라우저를 구동

5. ROS Basic

3) Turtlesim 실습

- RQT: GUI tool (Plugins)

- Robot Tools

- **Diagnostics Viewer:** 로봇 디바이스 및 에러 확인
- **Moveit! Monitor:** 로봇 팔 planning에 사용되는 Moveit! 데이터를 확인
- **Robot Steering:** 로봇 원격 조종 GUI 툴
- **Runtime Monitor:** 실시간으로 노드들에서 발생하는 에러 및 경고를 확인

- Service

- **Service Caller:** 구동중인 서비스 서버에 접속하여 서비스를 요청
- **Service Type Browser:** 서비스 타입의 데이터 구조를 확인

- Topics

- **Message Publisher:** 토픽을 GUI 환경에서 발행
- **Message Type Browser:** 토픽 타입의 데이터 구조를 확인
- **Topic Monitor:** 사용자가 선택한 토픽의 정보를 확인

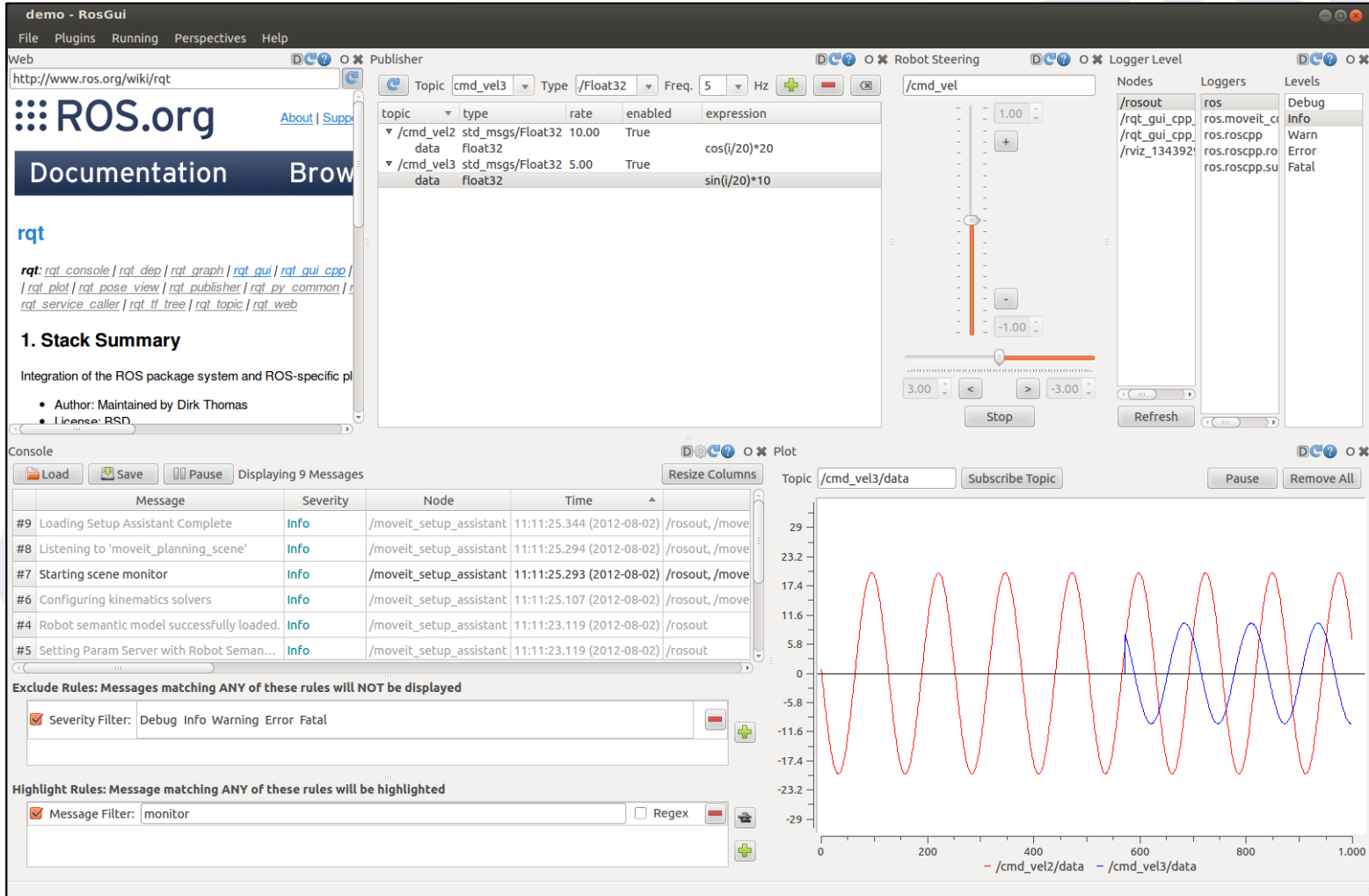
- Visualization

- **Image View:** 카메라의 영상 데이터를 확인
- **Navigation Viewer:** 로봇의 현재위치 및 목표지점 확인
- **Plot:** 2차원 데이터 플롯 GUI 플러그인, 2차원 데이터의 도식화
- **Pose View:** 현재 TF의 위치 및 모델의 위치 표시
- **RViz:** 3차원 시각화 툴 플러그인
- **TF Tree:** TF 관계를 트리로 나타내는 그래프 뷰

5. ROS Basic

3) Turtlesim 실습

- RQT: GUI tool (Plugins)



The screenshot displays the RQT (ROS Query Tool) interface, which is a GUI for interacting with ROS. The interface is divided into several panels:

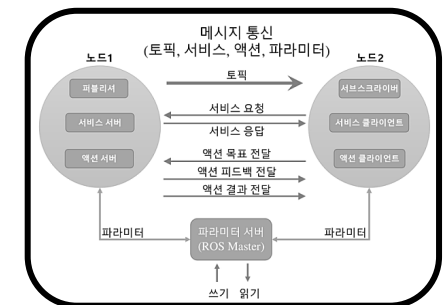
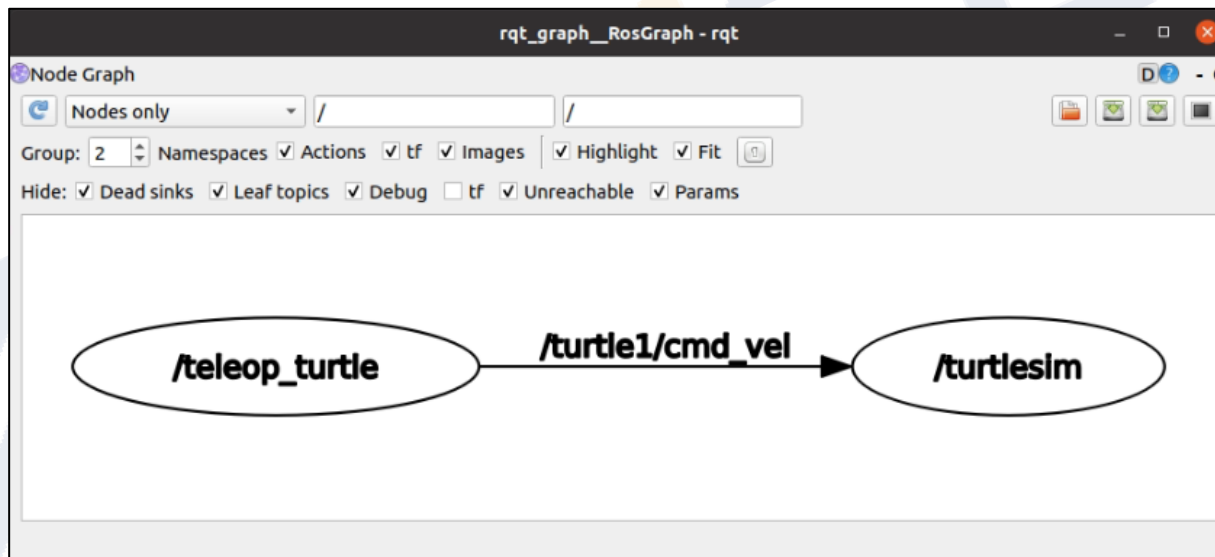
- Web Panel:** Shows the ROS.org website with the 'rqt' package documentation. The 'Stack Summary' section is visible, indicating it is maintained by Dirk Thomas and licensed under BSD.
- Topic Publisher Panel:** Allows publishing to a topic. The 'Topic' is set to 'cmd_vel3', the 'Type' is 'Float32', and the 'Freq.' is '5 Hz'. The 'expression' field contains 'sin(i/20)*10'. A slider control is also present for the 'cmd_vel' topic.
- Console Panel:** Displays a list of messages from the '/moveit_setup_assistant' node. The messages are filtered by severity (Debug, Info, Warning, Error, Fatal). The messages include 'Loading Setup Assistant Complete', 'Listening to 'moveit_planning_scene'', 'Starting scene monitor', 'Configuring kinematics solvers', 'Robot semantic model successfully loaded.', and 'Setting Param Server with Robot Sema...'. The 'Exclude Rules' and 'Highlight Rules' sections are also visible.
- Plot Panel:** Shows a plot of the 'cmd_vel3/data' topic. The plot displays two sinusoidal waves: a red wave for '/cmd_vel2/data' and a blue wave for '/cmd_vel3/data'. The x-axis represents time from 0 to 1000, and the y-axis represents velocity from -29 to 29.

5. ROS Basic

3) Turtlesim 실습

- RQT: GUI tool (Plugins)

```
$ rqt_graph
```

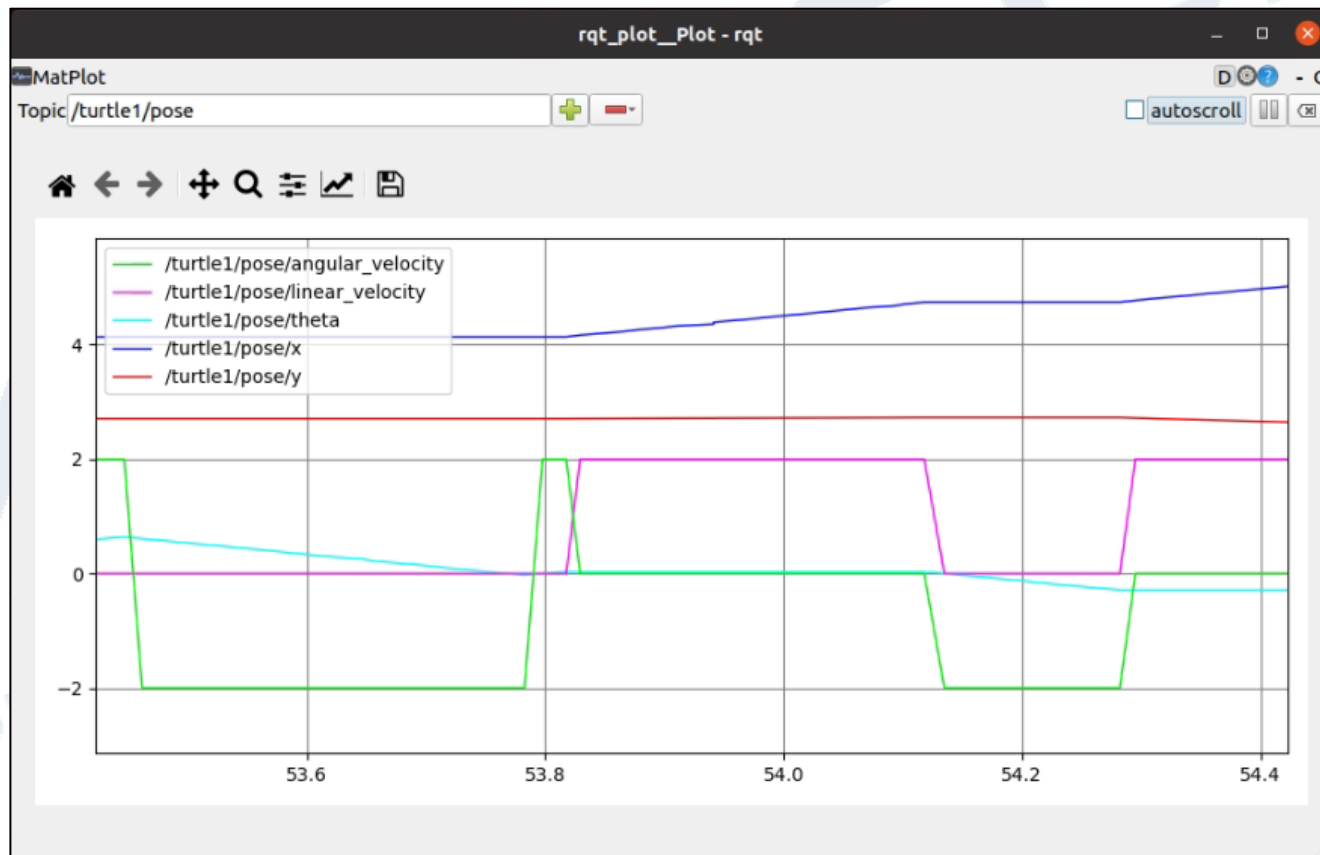


5. ROS Basic

3) Turtlesim 실습

- RQT: GUI tool (Plugins)

```
$ rqt_plot
```



5. ROS Basic

3) Turtlesim 실습

- ROS bag

- rosbag 은 현재 동작 중인 노드의 토픽을 record 하고 play 할 수 있음
- 토픽을 기록하고 재생할 때 사용할 수 있는 많은 명령어와 옵션을 제공함 ([Commandline](#))
- bag 파일을 저장할 디렉터리 생성

```
$ mkdir ~/bagfiles  
$ cd ~/bagfiles
```

- 모든 토픽을 record

```
$ rosbag record -a
```

- 특정 토픽만 record

```
$ rosbag record /turtle1/cmd_vel /turtle1/pose
```

5. ROS Basic

3) Turtlesim 실습

- ROS bag
 - bag file의 정보 확인

```
$ rosbag info [file_name]
```

```
cm1n@cm1n:~/bagfiles$ rosbag info 2024-07-05-17-34-37.bag
path:          2024-07-05-17-34-37.bag
version:       2.0
duration:      10.1s
start:         Jul 05 2024 17:34:37.00 (1720168478.00)
end:           Jul 05 2024 17:34:48.11 (1720168488.11)
size:          100.7 KB
messages:      1300
compression:   none [1/1 chunks]
types:         geometry_msgs/Twist [9f195f881246fdfa2798d1d3eebca84a]
               rosgraph_msgs/Log   [acffd30cd6b6de30f120938c17c593fb]
               turtlesim/Color     [353891e354491c51aabe32df673fb446]
               turtlesim/Pose      [863b248d5016ca62ea2e895ae5265cf9]
topics:        /rosout              3 msgs      : rosgraph_msgs/Log
               /turtle1/cmd_vel     61 msgs      : geometry_msgs/Twist
               /turtle1/color_sensor 618 msgs     : turtlesim/Color
               /turtle1/pose        618 msgs     : turtlesim/Pose
```

- bag file play

```
$ rosbag play [file_name]
```

ROS Programming

6. ROS Programming

1) 개발 환경 구축

- Create ROS workspace

- **catkin**: 빌드 권장 방식
- catkin은 표준화된 CMake 규칙을 따르고, 외부 코드 베이스 통합이나 소프트웨어 릴리스를 원하는 사용자에게 더 많은 유연성을 제공

```
$ mkdir -p ~/catkin_ws/src  
$ cd ~/catkin_ws  
$ catkin_make
```

- **catkin_make**: catkin 작업공간을 빌드하는 편리한 툴. 작업공간에서 처음 실행하면 build, devel 폴더가 생성되고, src 폴더에 최상위 수준의 CMakeLists.txt 가 생성
- **build**: 빌드 공간. src에 있는 catkin 패키지를 빌드하기 위해 CMake가 호출되는 장소.
- **devel**: 개발 공간. 패키지, 라이브러리, 헤더 파일 등의 실행 파일을 저장
- **src**: 소스 공간. 빌드하고자 하는 패키지의 소스 코드를 저장하는 공간

```
catkin_ws  
├── build  
├── devel  
└── src  
  
3 directories, 0 files
```

6. ROS Programming

1) 개발 환경 구축

- Create ROS workspace

- devel 폴더 안에는 여러 setup.*sh 파일이 있음. 이 파일들 중 하나(여기서는 setup.bash)를 소스하면 이 작업 공간이 ROS 환경 위에 오버레이됨

```
$ source ~/catkin_ws/devel/setup.bash
```

- 설정 스크립트를 통해 작업공간이 올바르게 오버레이되었는지 확인하려면 ROS_PACKAGE_PATH 환경 변수가 현재 디렉토리를 포함하고 있는지 확인

```
$ echo $ROS_PACKAGE_PATH
```

```
cmin@cmin:~$ echo $ROS_PACKAGE_PATH
/home/cmin/catkin_ws/src:/opt/ros/noetic/share
```

```
catkin_ws
├── build
│   ├── atomic_configure
│   ├── bin
│   ├── catkin
│   ├── catkin_generated
│   ├── CATKIN_IGNORE
│   ├── catkin_make.cache
│   ├── CMakeCache.txt
│   ├── CMakeFiles
│   ├── cmake_install.cmake
│   ├── CTestConfiguration.ini
│   ├── CTestCustom.cmake
│   ├── CTestTestfile.cmake
│   ├── gtest
│   ├── Makefile
│   └── test_results
├── devel
│   ├── cmake.lock
│   ├── env.sh
│   ├── lib
│   ├── local_setup.bash
│   ├── local_setup.sh
│   ├── local_setup.zsh
│   ├── setup.bash
│   ├── setup.sh
│   ├── _setup_util.py
│   └── setup.zsh
└── src
    └── CMakeLists.txt -> /opt/ros/noetic/share/catkin/cmake
11 directories, 18 files
```

6. ROS Programming

1) 개발 환경 구축

- Create catkin package

- 'catkin_create_pkg' 명령어를 통해 새로운 패키지를 생성
- 생성하고자하는 패키지 이름과 해당 패키지가 의존하는 종속성 목록을 입력
- 기본적으로 많이 입력하는 종속성: std_msgs, rospy, roscpp, ...

```
$ catkin_create_pkg <package_name> [depend1] [depend2] ...
```

- 먼저 src 폴더로 이동 후 catkin_create_pkg 명령어로 새로운 패키지 생성

```
$ cd ~/catkin_ws/src
$ catkin_create_pkg my_package std_msgs rospy roscpp
```

- 새로운 패키지 빌드 및 소싱

```
$ cd ~/catkin_ws
$ catkin_make
$ source ~/catkin_ws/devel/setup.bash
```

- rospack 명령어를 통한 패키지 종속성 확인

```
$ rospack depends1 my_package
```


6. ROS Programming

1) 개발 환경 구축

● Create catkin package

- include: C/C++ include header, python class 정의 파일 저장 폴더
- src: 소스 파일 저장 폴더
- CMakeLists.txt: CMake 빌드할 때 적용되는 컴파일 규칙 관련 파일
- package.xml: 패키지 manifest (정보를 담고 있는 파일. 이름, 저작자, 라이선스, 의존성 등)

```
catkin_ws/src/my_package
├── CMakeLists.txt
├── include
├── package.xml
└── src

2 directories, 2 files
```

매니페스트 파일

[한글 10개 언어](#)
[문서](#) [토론](#)
[읽기](#) [편집](#) [역사 보기](#) [도구](#)

위키백과, 우리 모두의 백과사전.

매니페스트 파일(manifest file)은 **컴퓨팅**에서 집합의 일부 또는 논리정연한 단위인 파일들의 그룹을 위한 **메타데이터**를 포함하는 파일이다. 예를 들어, 컴퓨터 프로그램의 파일들은 이름, 버전 번호, 라이선스, 프로그램의 구성 파일들을 가질 수 있다.

이 용어는 **화물 목록**(ship manifest)이 선원 및 화물을 나열하는 화물 수송 절차로부터 가져온 것이다.

패키지 매니페스트 [편집]

리눅스 배포판은 소프트웨어 배포를 위해 **패키지 관리자**에 크게 의존한다. 이 스킴에서 패키지는 매니페스트 파일을 포함하는 **압축 파일**이다. 주 목적은 배포판에 포함된 파일들을 열거하는 것으로, 다양한 패키징 도구들에 의해 처리되거나 인간의 소비를 위해 사용된다. 매니페스트들은 추가적인 정보를 담고 있는 경우가 있는데, 이를테면 **JAR**(**자바**로 작성된 소프트웨어 배포 시 사용하는 패키지 포맷)의 경우 버전 번호, 그리고 실행을 위한 **엔트리 포인트**를 지정할 수 있다. 매니페스트는 개별 파일의 **암호화 해시**나 **체크섬**을 선택적으로 포함할 수 있다. 매니페스트 파일을 위한 암호화 서명을 만듦으로써 배포판 패키지의 모든 내용이 진실하고 무결한지를 확인할 수 있으며 파일 중 어떤 것이라도 변경하면 매니페스트 파일의 체크섬의 유효성에 부합하지 않게 된다.

6. ROS Programming

1) 개발 환경 구축

- CMakeLists.txt

```
cmake_minimum_required(VERSION 3.0.2) → CMake의 최소 요구 버전
project(my_package) → 패키지(프로젝트) 이름

## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
find_package(catkin REQUIRED COMPONENTS → 빌드할 때 요구되는 의존성 패키지
  roscpp
  rospy
  std_msgs
)
```

6. ROS Programming

1) 개발 환경 구축

- CMakeLists.txt Custom message, service, action 파일이 있는 경우 작성

```
## Generate messages in the 'msg' folder
# add_message_files(
#   FILES
#   Message1.msg      → Custom message 파일이 있는 경우 추가
#   Message2.msg
# )

## Generate services in the 'srv' folder
# add_service_files(
#   FILES
#   Service1.srv      → Service 파일이 있는 경우 추가
#   Service2.srv
# )

## Generate actions in the 'action' folder
# add_action_files(
#   FILES
#   Action1.action     → Action 파일이 있는 경우 추가
#   Action2.action
# )

## Generate added messages and services with any dependencies listed here
# generate_messages(
#   DEPENDENCIES        → Custom message를 생성할 때 의존하는 메시지를 설정
#   std_msgs
# )
```

6. ROS Programming

1) 개발 환경 구축

- CMakeLists.txt C/C++ 기반의 노드가 있는 경우 작성

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES my_package
#  CATKIN_DEPENDS roscpp rospy std_msgs
#  DEPENDS system_lib
)

## Specify additional locations of header files
## Your package locations should be listed before other locations
include_directories(
# include
  ${catkin_INCLUDE_DIRS}
)

## Declare a C++ library
# add_library(${PROJECT_NAME}
#   src/${PROJECT_NAME}/my_package.cpp
# )

## Declare a C++ executable
## With catkin_make all packages are built within a single CMake context
## The recommended prefix ensures that target names across packages don't collide
# add_executable(${PROJECT_NAME}_node src/my_package_node.cpp)

## Add cmake target dependencies of the executable
## same as for the library above
# add_dependencies(${PROJECT_NAME}_node ${${PROJECT_NAME}_EXPORTED_TARGETS}
#   ${catkin_EXPORTED_TARGETS})

## Specify libraries to link a library or executable target against
# target_link_libraries(${PROJECT_NAME}_node
#   ${catkin_LIBRARIES}
# )
```

→ Catkin 빌드 옵션
Include 폴더 경로, 라이브러리, catkin 빌드 의존성,
시스템 라이브러리 의존성을 기술

→ Include 폴더 지정

→ 빌드 후 생성할 라이브러리 지정

→ C/C++ 소스 코드파일을 실행할
ROS 노드를 지정

→ 노드에 대한 의존성을 지정

→ 노드를 생성하기 전 필요한 라이브러리와 실행 파일을 링크

6. ROS Programming

1) 개발 환경 구축

- package.xml

```

<?xml version="1.0"?>    → XML 버전. 문서의 문법을 정의
<package format="2">      → 패키지 시작 및 형식 지정 (v2)
  <name>my_package</name>   → 패키지 이름
  <version>1.0.0</version>  → 패키지 버전
  <description>Simple description</description> → 패키지 설명
  <maintainer email="cmin87394@gmail.com">Cheolmin Jeong</maintainer> → 패키지 관리자 연락처
  <license>BSD 3-Clause</license> → 패키지의 라이선스
  <url type="website">http://wiki.ros.org/my_package</url> → 패키지를 설명하는 웹페이지, 저장소 등의 주소
  <author email="cmin87394@gmail.com">Cheolmin Jeong</author> → 패키지 개발자의 연락처

  <buildtool_depend>catkin</buildtool_depend> → 빌드 시스템의 의존성
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
  <build_export_depend>roscpp</build_export_depend>
  <build_export_depend>rospy</build_export_depend>
  <build_export_depend>std_msgs</build_export_depend>
  <exec_depend>roscpp</exec_depend>
  <exec_depend>rospy</exec_depend>
  <exec_depend>std_msgs</exec_depend>
</package>
  
```

패키지 의존성

6. ROS Programming

1) 개발 환경 구축

- bashrc에 alias 명령어 정의

- 자주 사용하는 명령어를 단축어로 정의할 수 있음
- bashrc를 편집기로 열기

```
$ gedit ~/.bashrc
```

- 아래와 같은 alias 단축어 추가

```
alias eb='gedit ~/.bashrc'  
alias vb='vim ~/.bashrc'  
alias sb='source ~/.bashrc'  
alias cw='cd ~/catkin_ws'  
alias cs='cd ~/catkin_ws/src'  
alias cm='cd ~/catkin_ws && catkin_make'  
alias sai='sudo apt install -y'  
alias saup='sudo apt update -y && sudo apt upgrade -y'  
alias rb='sudo reboot'  
alias sdn='sudo shutdown now'  
alias rd='sudo rdate time.bora.net'
```

- bashrc를 소싱

```
$ source ~/.bashrc
```

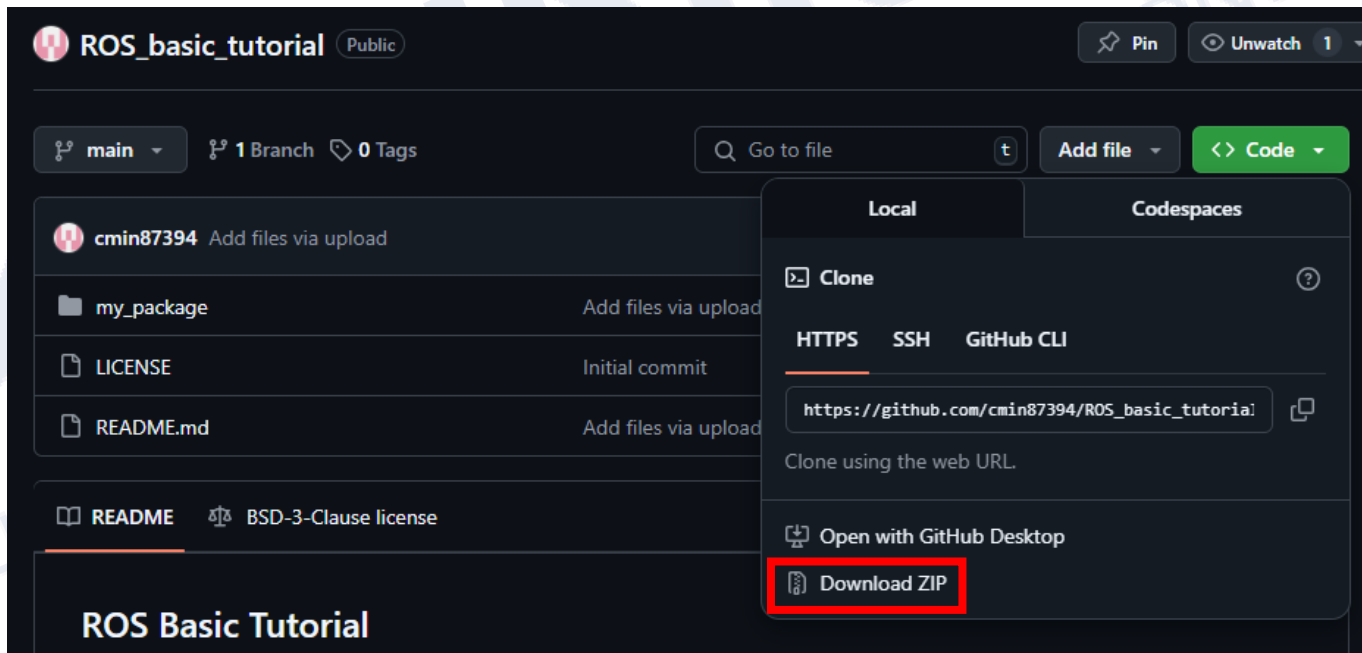
6. ROS Programming

강의 자료

```
$ git clone https://github.com/cmin87394/ROS_basic_tutorial
```

또는 github에서 직접 다운로드

▶ https://github.com/cmin87394/ROS_basic_tutorial



6. ROS Programming

2) Topic node

● Publisher

```
#!/usr/bin/env python3  → SheBang(!). 인터프리터 지시자. Python3의 경로를 찾아서 실행시켜 줌

import rospy  → rospy 모듈. Python 기반 ROS node를 작성할 때는 항상 필요함
from std_msgs.msg import String  → std_msg 메시지 패키지에서 String 메시지를 불러옴

def talker():  → talker 함수 정의
    pub = rospy.Publisher('chatter', String, *queue_size=1)  → Publisher 선언
                                                            ([topic_name], [message_data_type], [queue_size])
    rospy.init_node('talker')  → 노드를 초기화 및 노드의 이름 정의
                                * queue_size는 항상 1로 하는 것을 권장함
    rate = rospy.Rate(10)  → 10Hz 만큼의 주기를 rate 변수로 할당
    while not rospy.is_shutdown():  → 반복문. rospy.is_shutdown(): 노드가 종료되었는지 여부
        hello_str = "%s hello world %s"%(rospy.get_caller_id(), rospy.get_time())
        rospy.loginfo(hello_str)  → ROS information 형태로 데이터 출력 (print와 유사함)
        pub.publish(hello_str)  → 미리 정의된 string 데이터를 publish
                                ↗ publish할 string 타입의 데이터를 정의
        rate.sleep()  → 미리 정의된 주기만큼 sleep

if __name__ == '__main__':  → 해당 python 파일이 모듈 형태가 아닌 직접 실행되었을 경우에만 동작하는 조건문
    try:
        talker()  → talker 함수 호출
    except rospy.ROSInterruptException:  → 노드가 ctrl+c 등으로 종료되면 실행되는 예외 구문
        pass
```

6. ROS Programming

2) Topic node

- Subscriber

```
#!/usr/bin/env python3

import rospy
from std_msgs.msg import String

def callback(data):  → Subscriber의 콜백 함수. subscribe 받는 토픽이 콜백 함수의 데이터로 할당 됨
    rospy.loginfo(rospy.get_caller_id() + " I heard %s"%data.data)
    → 토픽의 데이터를 ROS information 형태로 출력

def listener():  → listener 함수 정의
    rospy.init_node('listener')
    rospy.Subscriber('chatter', String, callback)  → Subscriber 선언
                                                    ([topic_name], [message_data_type], [callback_function])
    rospy.spin()  → spin() 함수는 publish 또는 subscribe를
                  주기적으로 반복시킴

if __name__ == '__main__':
    listener()  → listener 함수 호출
```

6. ROS Programming

2) Topic node

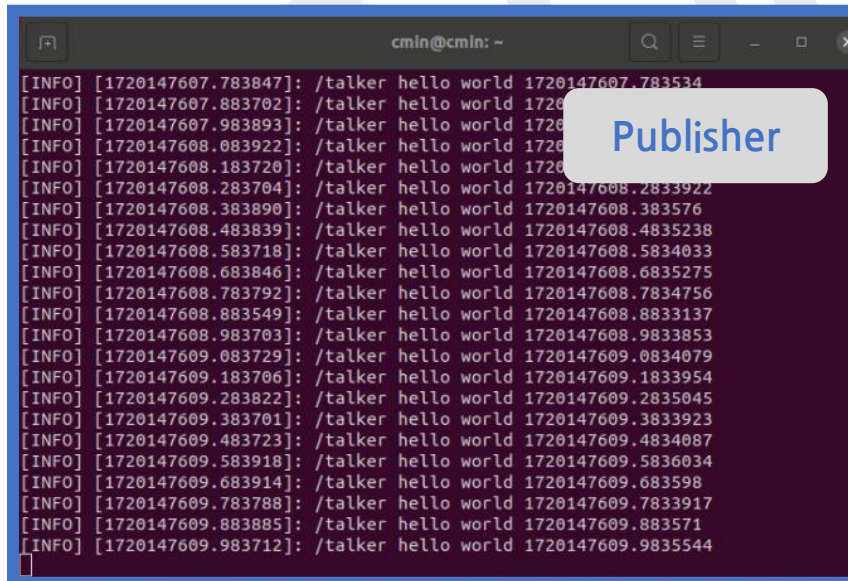
- 노드 실행

```
$ sudo chmod 777 ~/catkin_ws/src/my_package/src/*
```

```
$ roscore
```

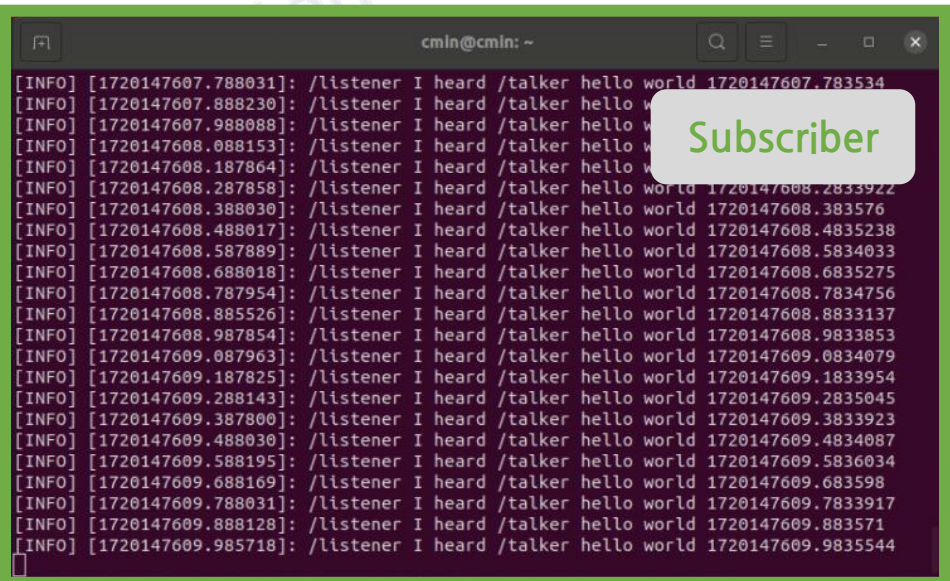
```
$ rosrun my_package talker.py
```

```
$ rosrun my_package listener.py
```



```

cmln@cmln: ~
[INFO] [1720147607.783847]: /talker hello world 1720147607.783534
[INFO] [1720147607.883702]: /talker hello world 1720147607.883534
[INFO] [1720147607.983893]: /talker hello world 1720147607.983534
[INFO] [1720147608.083922]: /talker hello world 1720147608.0835275
[INFO] [1720147608.183720]: /talker hello world 1720147608.1835238
[INFO] [1720147608.283704]: /talker hello world 1720147608.2833922
[INFO] [1720147608.383890]: /talker hello world 1720147608.383576
[INFO] [1720147608.483839]: /talker hello world 1720147608.4835238
[INFO] [1720147608.583718]: /talker hello world 1720147608.5834033
[INFO] [1720147608.683846]: /talker hello world 1720147608.6835275
[INFO] [1720147608.783792]: /talker hello world 1720147608.7834756
[INFO] [1720147608.883549]: /talker hello world 1720147608.8833137
[INFO] [1720147608.983703]: /talker hello world 1720147608.9833853
[INFO] [1720147609.083729]: /talker hello world 1720147609.0834079
[INFO] [1720147609.183706]: /talker hello world 1720147609.1833954
[INFO] [1720147609.283822]: /talker hello world 1720147609.2835045
[INFO] [1720147609.383701]: /talker hello world 1720147609.3833923
[INFO] [1720147609.483723]: /talker hello world 1720147609.4834087
[INFO] [1720147609.583918]: /talker hello world 1720147609.5836034
[INFO] [1720147609.683914]: /talker hello world 1720147609.683598
[INFO] [1720147609.783788]: /talker hello world 1720147609.7833917
[INFO] [1720147609.883885]: /talker hello world 1720147609.883571
[INFO] [1720147609.983712]: /talker hello world 1720147609.9835544
  
```



```

cmln@cmln: ~
[INFO] [1720147607.788031]: /listener I heard /talker hello world 1720147607.783534
[INFO] [1720147607.888230]: /listener I heard /talker hello world 1720147607.883534
[INFO] [1720147607.988088]: /listener I heard /talker hello world 1720147607.983534
[INFO] [1720147608.088153]: /listener I heard /talker hello world 1720147608.0835275
[INFO] [1720147608.187864]: /listener I heard /talker hello world 1720147608.1835238
[INFO] [1720147608.287858]: /listener I heard /talker hello world 1720147608.2833922
[INFO] [1720147608.388030]: /listener I heard /talker hello world 1720147608.383576
[INFO] [1720147608.488017]: /listener I heard /talker hello world 1720147608.4835238
[INFO] [1720147608.587889]: /listener I heard /talker hello world 1720147608.5834033
[INFO] [1720147608.688018]: /listener I heard /talker hello world 1720147608.6835275
[INFO] [1720147608.787954]: /listener I heard /talker hello world 1720147608.7834756
[INFO] [1720147608.885526]: /listener I heard /talker hello world 1720147608.8833137
[INFO] [1720147608.987854]: /listener I heard /talker hello world 1720147608.9833853
[INFO] [1720147609.087963]: /listener I heard /talker hello world 1720147609.0834079
[INFO] [1720147609.187825]: /listener I heard /talker hello world 1720147609.1833954
[INFO] [1720147609.288143]: /listener I heard /talker hello world 1720147609.2835045
[INFO] [1720147609.387800]: /listener I heard /talker hello world 1720147609.3833923
[INFO] [1720147609.488030]: /listener I heard /talker hello world 1720147609.4834087
[INFO] [1720147609.588195]: /listener I heard /talker hello world 1720147609.5836034
[INFO] [1720147609.688169]: /listener I heard /talker hello world 1720147609.683598
[INFO] [1720147609.788031]: /listener I heard /talker hello world 1720147609.7833917
[INFO] [1720147609.888128]: /listener I heard /talker hello world 1720147609.883571
[INFO] [1720147609.985718]: /listener I heard /talker hello world 1720147609.9835544
  
```

6. ROS Programming

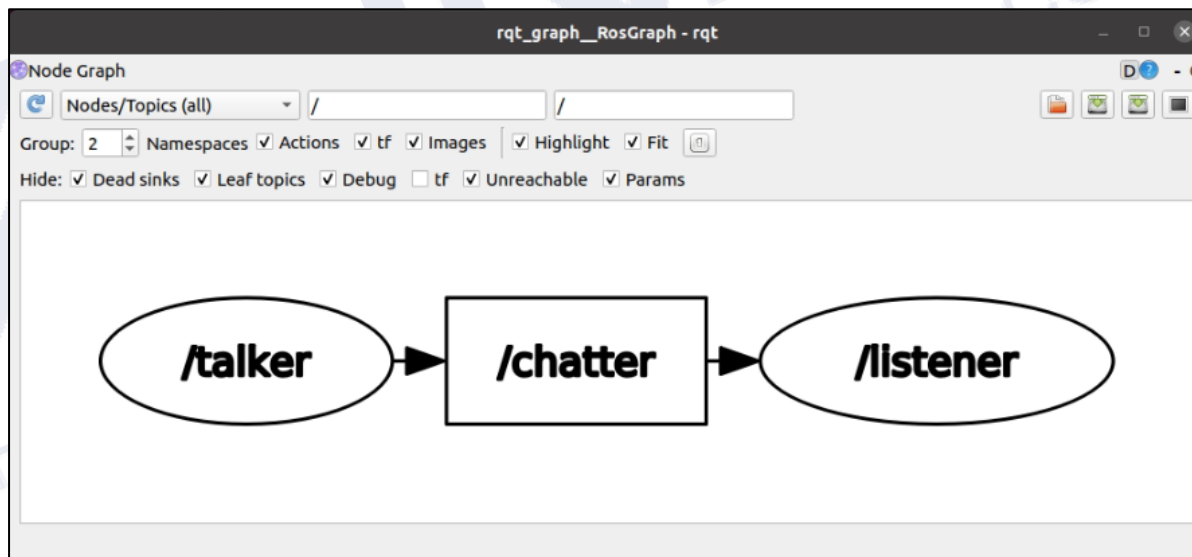
2) Topic node

- 토픽 확인

```
$ rostopic list
```

```
$ rostopic hz /chatter
```

```
$ rqt_graph
```



6. ROS Programming

2) Topic node

- 연습 문제

- 터틀심 거북이의 현재 위치를 sub 받아서 1초마다 print 및 publish 하는 노드 작성

Hint1 - 거북이의 현재 위치 관련 토픽: /turtlesim1/pose

Hint2 - /turtlesim1/pose 의 메시지 데이터 타입:

```
float32 x  
float32 y  
float32 theta  
float32 linear_velocity  
float32 angular_velocity
```

6. ROS Programming

3) Custom message

- msg 파일 생성 및 빌드

- 패키지 내에 msg 폴더 및 msg 빈 파일 생성

```
$ roscd my_package  
$ mkdir msg  
$ gedit MsgTutorial.msg
```

- 다음과 같이 작성 후 저장

```
time stamp  
string name  
int64 age  
float64 height
```

↓
데이터
타입

↓
변수명

6. ROS Programming

3) Custom message

- msg 파일 생성 및 빌드
 - CMakeLists.txt 와 package.xml 파일에 아래 내용 추가

CMakeLists.txt

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)

add_message_files(
  DIRECTORY
  msg
)

generate_messages(
  DEPENDENCIES
  std_msgs
)

catkin_package(
  CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
)
```


6. ROS Programming

3) Custom message

- msg 파일 생성 및 빌드
 - CMakeLists.txt 와 package.xml 파일에 아래 내용 추가

package.xml

```
<build_depend>message_generation</build_depend>  
<build_export_depend>message_generation</build_export_depend>  
<exec_depend>message_runtime</exec_depend>
```

- 패키지 빌드

```
$ cd ~/catkin_ws && catkin_make
```

6. ROS Programming

3) Custom message

- Simple message test

```
#!/usr/bin/env python3

import rospy
from my_package.msg import MsgTutorial → msg 디렉터리 내의 MsgTutorial을 불러옴

def simple_msg(): → simple_msg 함수 정의
    rospy.init_node('simple_msg')
    pub = rospy.Publisher('my_topic', MsgTutorial, queue_size=1)
    data = MsgTutorial() → custom message 타입을 변수에 할당
    rospy.loginfo("Publish start")
    while not rospy.is_shutdown():
        data.stamp = rospy.Time.now()
        data.name = 'Steve'
        data.age = 25
        data.height = 175.5
        pub.publish(data) → 데이터 publish
        rospy.Rate(1).sleep()

if __name__=="__main__":
    simple_msg() → simple_msg 함수 호출
```

publish 할 데이터 값을 입력

6. ROS Programming

3) Custom message

- 노드 실행

```
$ sudo chmod 777 ~/catkin_ws/src/my_package/src/*
```

```
$ roscore
```

```
$ rosrun my_package simple_msg_test.py
```

```
$ rostopic list
```

```
cmln@cmln:~$ rostopic list
/my_topic
/rosout
/rosout_agg
```

```
$ rostopic echo /my_topic
```

```
---
stamp:
  secs: 1720165954
  nsecs: 253032684
name: "Steve"
age: 25
height: 175.5
---
```

6. ROS Programming

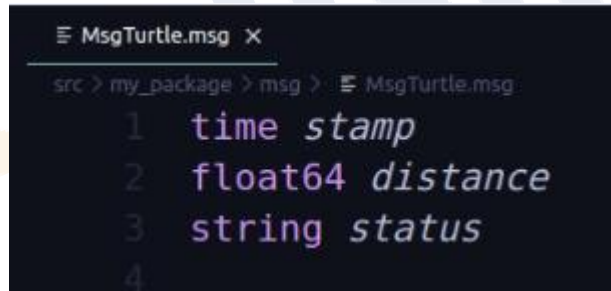
3) Custom message

- 연습 문제

- 커스텀 메시지를 통해 터틀심의 거북이가 벽에 가까워질 때 토픽으로 경고해주는 노드 작성

Hint1 - 터틀심에서 거북이가 움직일 수 있는 좌표 범위: 0.0 ~ 11.0

Hint2 - 커스텀 메시지 형식:



```
MsgTurtle.msg X
src > my_package > msg > MsgTurtle.msg
1  time stamp
2  float64 distance
3  string status
4
```

※ 메시지 파일을 못 찾을 때 - 빌드 초기화 후 다시 빌드

```
$ cd ~/catkin_ws
$ rm -r build devel
$ catkin_make
```

6. ROS Programming

4) Service node

- srv 파일 생성 및 빌드

- 목표: 두 정수의 합을 구하는 서비스 구현
- 패키지 내에 srv 폴더 및 srv 빈 파일 생성

```
$ roscd my_package
$ mkdir srv
$ gedit AddTwoInts.srv
```

- 다음과 같이 작성 후 저장

```
int64 a
int64 b      → service request
--- → request, response 구분자
int64 sum    → service response
```

↓ ↓
 데이터 변수명
 타입

6. ROS Programming

4) Service node

- srv 파일 생성 및 빌드
 - CMakeLists.txt 와 package.xml 파일에 아래 내용 추가

CMakeLists.txt

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)

add_service_files(
  DIRECTORY
  srv
)

generate_messages(
  DEPENDENCIES
  std_msgs
)

catkin_package(
  CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
)
```

6. ROS Programming

4) Service node

- srv 파일 생성 및 빌드
 - CMakeLists.txt 와 package.xml 파일에 아래 내용 추가

package.xml

```
<build_depend>message_generation</build_depend>  
<build_export_depend>message_generation</build_export_depend>  
<exec_depend>message_runtime</exec_depend>
```

- 패키지 빌드

```
$ cd ~/catkin_ws && catkin_make
```


6. ROS Programming

4) Service node

● Server

* srv 파일을 빌드하면 [service_name]Request, [service_name]Response 는 자동으로 생성됨

```
#!/usr/bin/env python3
```

```
import rospy
```

```
from my_package.srv import AddTwoInts, * AddTwoIntsResponse
```

→ srv 디렉터리 내의 AddTwoInts를 불러옴
AddTwoIntsResponse 는 client에게 reponse를 전달하기 위해 호출

```
def handle_add_two_ints(req):
```

→ AddTwoIntsRequest 인스턴스와 함께 호출된 함수

```
    print("Returning [%s + %s = %s]"%(req.a, req.b, (req.a + req.b)))
```

```
    return AddTwoIntsResponse(req.a + req.b)
```

→ AddTwoIntsResponse 인스턴스 반환

```
def add_two_ints_server():
```

→ add_two_ints_server 함수 정의

```
    rospy.init_node('add_two_ints_server')
```

```
    s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_ints)
```

```
    print("Ready to add two ints.")
```

```
    rospy.spin()
```

→ request가 올 때마다 반복적으로 처리

→ Service 선언
([service_name], [service_type], [handler_function])
모든 request는 handler function으로 전달되고 response를 반환

```
if __name__ == "__main__":
```

```
    add_two_ints_server()
```

→ add_two_ints_server 함수 호출

6. ROS Programming

4) Service node

- Client

```
#!/usr/bin/env python3

import sys
import rospy
from my_package.srv import AddTwoInts

def add_two_ints_client(x, y):  → add_two_ints_client 함수 정의
    rospy.wait_for_service('add_two_ints')  → add_two_ints 서비스가 요청 가능할 때까지 대기
    try:
        add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)
        resp1 = add_two_ints(x, y)  → 입력 변수로 service를 호출하여
        return resp1.sum  → 변수 반환  반환 받은 결과값을 변수로 할당
    except rospy.ServiceException as e:  → 호출이 실패할 경우의 예외 구문
        print("Service call failed: %s"%e)

def usage():  → usage 함수 정의
    return "%s [x y]"%sys.argv[0]
```

ServiceProxy 선언 (service를 호출하는 핸들) ([service_name], [service_type])

6. ROS Programming

4) Service node

- Client

```

if __name__ == "__main__":
    if len(sys.argv) == 3: → 실행 시 입력한 파라미터가 3개일 경우 (첫번째 값은 항상 실행 파일의 이름)
        x = int(sys.argv[1]) → 두번째 값을 변수 x로 할당
        y = int(sys.argv[2]) → 세번째 값을 변수 y로 할당
    else:
        print(usage()) → 입력 파라미터의 수가 부족한 경우 usage 함수 호출
        sys.exit(1) → 오류 발생 및 실행 파일 종료
    print("Requesting %s+%s"%(x, y))
    print("%s + %s = %s"%(x, y, add_two_ints_client(x, y))) → add_two_ints_client 함수 호출
                                                             반환되는 값을 출력
  
```

6. ROS Programming

4) Service node

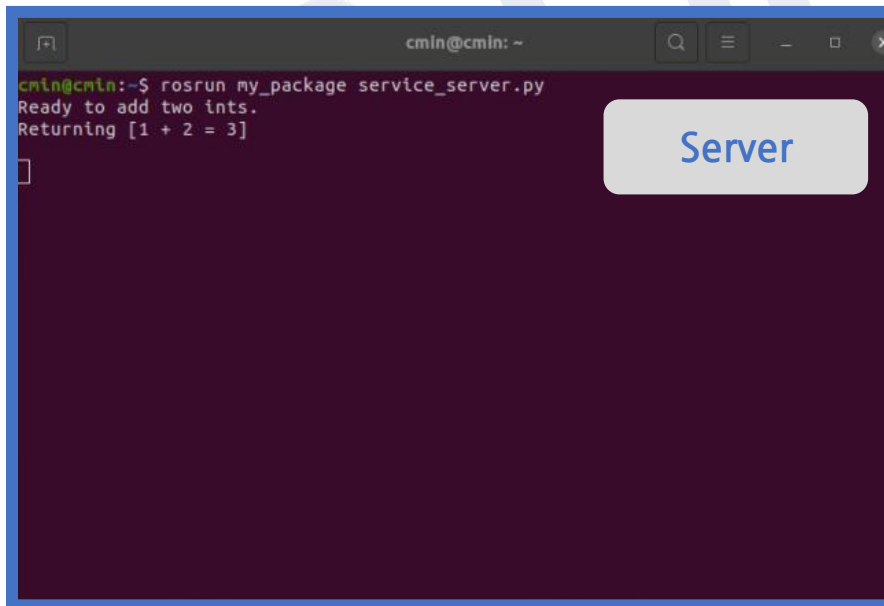
- 노드 실행

```
$ sudo chmod 777 ~/catkin_ws/src/my_package/src/*
```

```
$ roscore
```

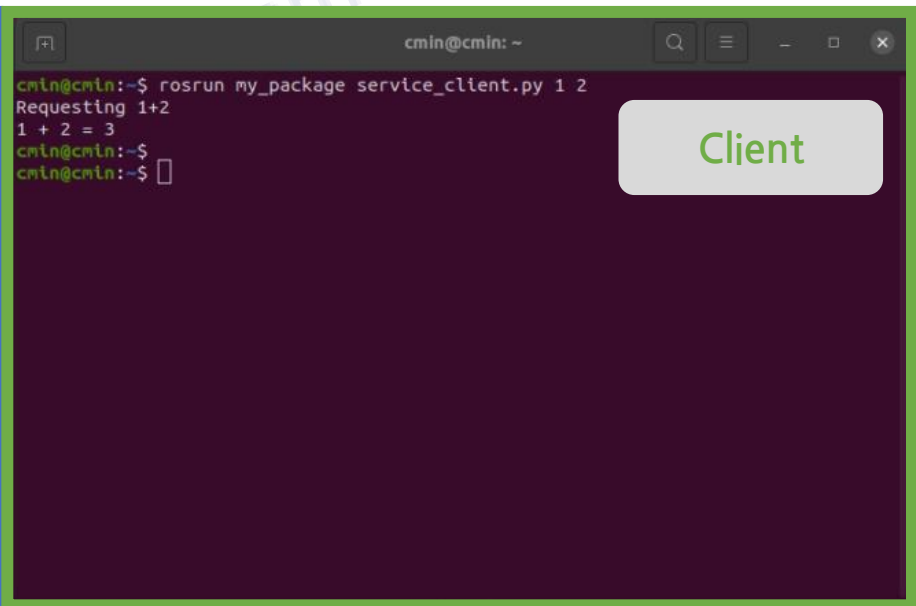
```
$ rosrun my_package service_server.py
```

```
$ rosrun my_package service_client.py 1 2
```



A terminal window titled 'cmin@cmin: ~' with a search icon, menu icon, and window controls. The terminal output shows the command 'rosrun my_package service_server.py' being executed. The output text is: 'Ready to add two ints.', 'Returning [1 + 2 = 3]', and a prompt character '[]'. A light blue rounded rectangle with the word 'Server' is overlaid on the right side of the terminal.

```
cmin@cmin: ~  
cmin@cmin:~$ rosrun my_package service_server.py  
Ready to add two ints.  
Returning [1 + 2 = 3]  
[]
```



A terminal window titled 'cmin@cmin: ~' with a search icon, menu icon, and window controls. The terminal output shows the command 'rosrun my_package service_client.py 1 2' being executed. The output text is: 'Requesting 1+2', '1 + 2 = 3', and two prompt characters '[]'. A light green rounded rectangle with the word 'Client' is overlaid on the right side of the terminal.

```
cmin@cmin: ~  
cmin@cmin:~$ rosrun my_package service_client.py 1 2  
Requesting 1+2  
1 + 2 = 3  
cmin@cmin:~$  
cmin@cmin:~$ []
```

6. ROS Programming

4) Service node

- 서비스 확인

```
$ rosservice list
```

```
cm1n@cm1n:~$ rosservice list
/add_two_ints
/add_two_ints_server/get_loggers
/add_two_ints_server/set_logger_level
/rosout/get_loggers
/rosout/set_logger_level
```

```
$ rosservice info /add_two_ints
```

```
cm1n@cm1n:~$ rosservice info /add_two_ints
Node: /add_two_ints_server
URI: rosrpc://cm1n:41547
Type: my_package/AddTwoInts
Args: a b
```

```
$ rossrv info my_package/AddTwoInts
```

```
cm1n@cm1n:~$ rossrv info my_package/AddTwoInts
int64 a
int64 b
---
int64 sum
```

```
$ rosservice call /add_two_ints 5 3
```

```
cm1n@cm1n:~$ rosservice call /add_two_ints 5 3
sum: 8
```

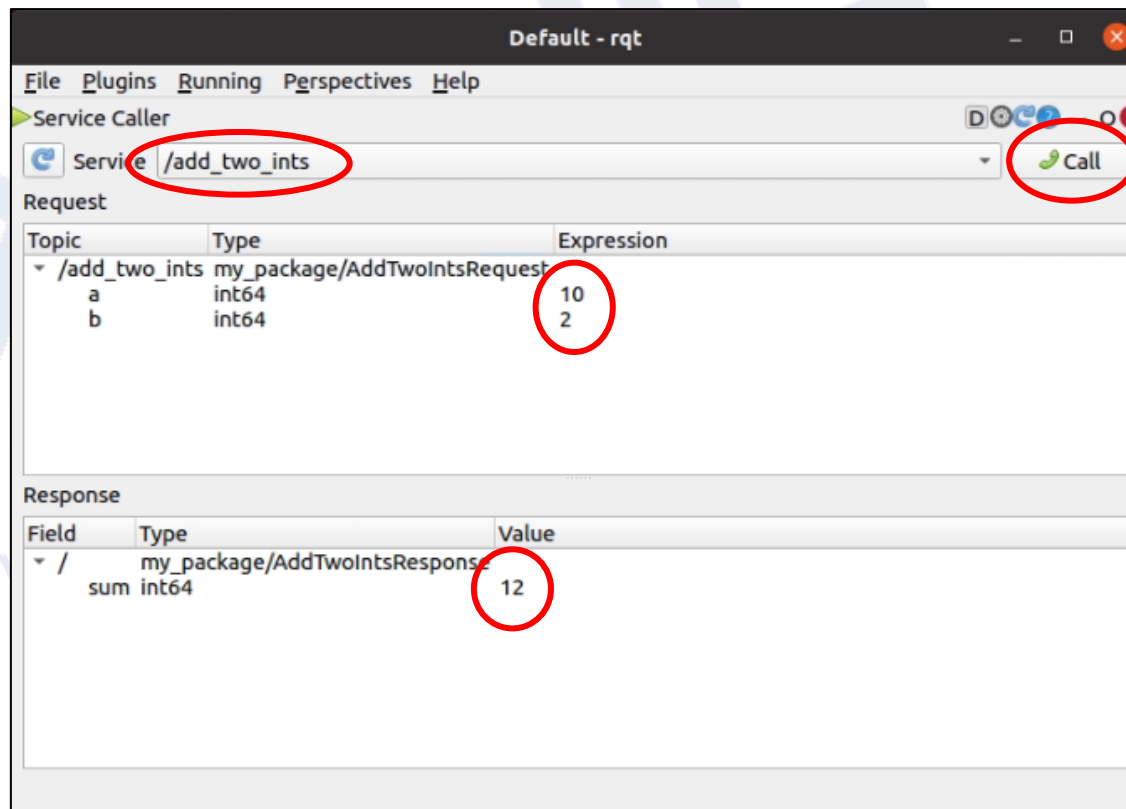
6. ROS Programming

4) Service node

- 서비스 확인

```
$ rqt
```

- Service Caller
- Service select → a, b input → Call → sum output



6. ROS Programming

4) Service node

- 연습 문제

- 터틀심 거북이의 누적 이동거리가 5m가 될 때마다 /clear 서비스를 요청하는 노드 작성

Hint1 - 거북이의 위치 관련 토픽: /turtlesim1/pose

Hint2 - /clear 서비스 정보:

```
cmin@cmin:~$ rosservice info /clear
Node: /turtlesim
URI: rosrpc://cmin:42635
Type: std_srvs/Empty
Args:
```


6. ROS Programming

5) Action node

- action 파일 생성 및 빌드

- 목표: 피보나치(Fibonacci) 수열을 구하는 액션 구현 ([wikipedia](https://en.wikipedia.org/wiki/Fibonacci_sequence))
- 패키지 내에 action 폴더 및 action 빈 파일 생성

```
$ roscd my_package  
$ mkdir action  
$ gedit Fibonacci.action
```

- 다음과 같이 작성 후 저장

```
int32 order          → action goal  
---  
int32[] sequence     → action result  
---  
int32[] sequence     → action feedback
```

↓
데이터
타입

↓
변수명

6. ROS Programming

5) Action node

- action 파일 생성 및 빌드
 - CMakeLists.txt 와 package.xml 파일에 아래 내용 추가

CMakeLists.txt

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
  actionlib_msgs
)

add_action_files(
  DIRECTORY
  action
)

generate_messages(
  DEPENDENCIES
  std_msgs
  actionlib_msgs
)

catkin_package(
  CATKIN_DEPENDS roscpp rospy std_msgs message_runtime actionlib_msgs
)
```

6. ROS Programming

5) Action node

- action 파일 생성 및 빌드

- CMakeLists.txt 와 package.xml 파일에 아래 내용 추가

package.xml

```
<build_depend>actionlib_msgs</build_depend>
<build_export_depend>actionlib_msgs</build_export_depend>
<exec_depend>actionlib_msgs</exec_depend>
```

- 패키지 빌드

```
$ cd ~/catkin_ws && catkin_make
```

- 빌드가 끝나면 action 관련 메시지 파일들이 자동 생성

```
$ cd ~/catkin_ws/devel/share/my_package/msg
```

```
cmn@cmn:~/catkin_ws/devel/share/my_package/msg$ ls
FibonacciActionFeedback.msg*  FibonacciActionResult.msg*  FibonacciResult.msg*
FibonacciActionGoal.msg*     FibonacciFeedback.msg*
FibonacciAction.msg*         FibonacciGoal.msg*
```

6. ROS Programming

5) Action node

- Server

```

#!/usr/bin/env python3

import rospy
import actionlib → actionlib 모듈을 불러옴
from my_package.msg import FibonacciAction, FibonacciFeedback, FibonacciResult

feedback = FibonacciFeedback()
result = FibonacciResult() ] 빌드 후 자동으로 생성된 Fibonacci 메시지 중 필요한 것만 불러옴
                           ] feedback 과 result 를 변수로 할당

def execute_cb(goal): → action의 콜백 함수인 execute_cb 정의
    success = True
    feedback.sequence = []
    feedback.sequence.append(0)
    feedback.sequence.append(1) ] 배열의 초기값을 설정
    rospy.loginfo("%s: Executing, creating fibonacci sequence of order %i with
seeds %i, %i"('%fibonacci', goal.order, feedback.sequence[0], feedback.sequence[1]))
    for i in range(1, goal.order): → 피보나치 수열 반복문 시작
        if server.is_preempt_requested():
            rospy.loginfo("%s: Preempted"('%fibonacci'))
            server.set_preempted()
            success = False
            break ] client가 goal을 취소하도록 요청하면 이를 처리하는 구문
  
```

6. ROS Programming

5) Action node

● Server

↗ 배열의 마지막 두 수를 더하여 배열에 추가
피드백 변수에 할당

```

feedback.sequence.append(feedback.sequence[i] + feedback.sequence[i-1])
server.publish_feedback(feedback) → 현재 feedback 정보를 topic 형태로 publish
rospy.loginfo("feedback: %s"%feedback.sequence) → 현재 feedback 정보를 ros information 형태로 출력
rospy.Rate(1).sleep() → 1Hz 만큼 sleep
if success: → 피보나치 수열 연산이 완료되었는지 여부
    result.sequence = feedback.sequence → 최종 배열값을 result 변수에 할당
    rospy.loginfo("%s: Succeeded"%'fibonacci')
    server.set_succeeded(result) → action result를 client에게 반환

def fibonacci_server(): → fibonacci_server 함수 정의
    global server
    rospy.init_node('fibonacci_server')
    server = actionlib.SimpleActionServer('fibonacci', FibonacciAction,
execute_cb=execute_cb, auto_start=False) → Action server 선언
    server.start() → server 동작 시작 ([action_name], [action_type], [execute_callback], [auto_start])
    rospy.spin() → client의 요청이 올 때마다 반복적으로 처리 client가 요청한 goal은 callback 함수로 전달되고 feedback과 result를 반환

if __name__ == '__main__':
    fibonacci_server() → fibonacci_server 함수 호출
  
```

6. ROS Programming

5) Action node

- Client

```

#!/usr/bin/env python3

import sys
import rospy
import actionlib
from my_package.msg import FibonacciAction, FibonacciGoal

def fibonacci_client(x):
    rospy.init_node('fibonacci_client')
    try:
        client = actionlib.SimpleActionClient('fibonacci', FibonacciAction)
        client.wait_for_server()
        goal = FibonacciGoal(order=x)
        client.send_goal(goal)
        client.wait_for_result()
        return client.get_result()
    except rospy.ROSInterruptException:
        print("program interrupted before completion", file=sys.stderr)
  
```

→ 빌드 후 자동으로 생성된 Fibonacci 메시지 중 필요한 것만 불러옴
 → fibonacci_client 함수 정의
 Action client 선언
 ([action_name], [action_type])
 → server 작동 시작까지 대기
 x 변수를 goal로 지정하고 server에 요청
 → result 반환까지 대기

6. ROS Programming

5) Action node

- Client

```
if __name__ == '__main__':  
    if len(sys.argv) == 2:  
        x = int(sys.argv[1])  
    else:  
        print("%s [%x]"%sys.argv[0])  
        sys.exit(1)  
    print("Result:", ', '.join([str(n) for n in fibonacci_client(x).sequence]))
```

→ Fibonacci 수열의 결과를 출력

6. ROS Programming

5) Action node

- 노드 실행

```
$ sudo chmod 777 ~/catkin_ws/src/my_package/src/*
```

```
$ roscore
```

```
$ rosrun my_package action_server.py
```

```
$ rosrun my_package action_client.py 10
```

```
cmin@cmin: ~
cmin@cmin:~$ rosrun my_package action_server.py
[INFO] [1720135089.019414]: fibonacci: Executing, creating fibonacci sequence of order 10 with seeds 0, 1
[INFO] [1720135089.020674]: feedback: [0, 1, 1]
[INFO] [1720135090.023532]: feedback: [0, 1, 1, 2]
[INFO] [1720135091.029037]: feedback: [0, 1, 1, 2, 3]
[INFO] [1720135092.033767]: feedback: [0, 1, 1, 2, 3, 5]
[INFO] [1720135093.036906]: feedback: [0, 1, 1, 2, 3, 5, 8]
[INFO] [1720135094.042323]: feedback: [0, 1, 1, 2, 3, 5, 8, 13]
[INFO] [1720135095.047440]: feedback: [0, 1, 1, 2, 3, 5, 8, 13, 21]
[INFO] [1720135096.052784]: feedback: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[INFO] [1720135097.056568]: feedback: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
[INFO] [1720135098.061499]: fibonacci: Succeeded
```

Server

```
cmin@cmin: ~
cmin@cmin:~$ rosrun my_package action_client.py 10
Result: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55
cmin@cmin:~$
```

Client

6. ROS Programming

5) Action node

- 액션 피드백 토픽 확인

```
$ rostopic list
```

```
$ rostopic echo /fibonacci/feedback
```

```
---
header:
  seq: 9
  stamp:
    secs: 1720135097
    nsecs: 56095600
  frame_id: ''
status:
  goal_id:
    stamp:
      secs: 1720135089
      nsecs: 18799066
    id: "/fibonacci_client-1-1720135089.019"
  status: 1
  text: "This goal has been accepted by the simple action server"
feedback:
  sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
---
```

6. ROS Programming

5) Action node

- 연습 문제

- 좌표를 입력하면 터틀심 거북이가 입력한 좌표까지 이동하고 가는 도중 남은 거리를 피드백해주는 액션 노드 작성

Hint1 - 거북이의 위치 관련 토픽: /turtlesim1/pose

Hint2 - 목표지점까지의 x, y 거리: $x_{distance} = x_{goal} - x_{pose}$, $y_{distance} = y_{goal} - y_{pose}$

Hint3 - 목표지점까지 총 거리: $Total\ distance = \sqrt{x_{distance}^2 + y_{distance}^2}$

Hint4 - 목표지점을 바라보는 방향 각도: $theta_{goal} = \arctan \frac{y_{distance}}{x_{distance}}$

Hint5 - python numpy 모듈의 arctan2 함수 사용 (각도의 부호에 관계없이 arctan를 적용할 수 있음)

6. ROS Programming

6) Parameter

- Simple parameter test

```
#!/usr/bin/env python3

import rospy

def simple_param():
    rospy.init_node('simple_param')
    rospy.set_param('P', 1.0)
    rospy.set_param('I', 1.0)
    rospy.set_param('D', 1.0)

    while not rospy.is_shutdown():
        P_gain = rospy.get_param('P')
        I_gain = rospy.get_param('I')
        D_gain = rospy.get_param('D')
        rospy.loginfo("P: %f, I: %f, D: %f"%(P_gain, I_gain, D_gain))
        rospy.Rate(1).sleep()

if __name__=="__main__":
    simple_param()
```

파라미터 초기값 설정

파라미터 값을 변수로 할당

6. ROS Programming

6) Parameter

- 노드 실행

```
$ sudo chmod 777 ~/catkin_ws/src/my_package/src/*
```

```
$ roscore
```

```
$ rosrun my_package simple_param_test.py
```

- 노드를 실행하면 초기값으로 설정된 $P=1.0$, $I=1.0$, $D=1.0$ 으로 출력됨

```
[INFO] [1720161917.902130]: P: 1.000000, I: 1.000000, D: 1.000000
```

- P gain을 0.5로 설정

```
$ rosparm set /P 0.5
```

- 값이 변화된 것을 확인할 수 있음

```
[INFO] [1720161918.912891]: P: 0.500000, I: 1.000000, D: 1.000000
```

6. ROS Programming

6) Parameter

- 연습 문제
 - 파라미터가 변할 수 있는 최대, 최소값 지정하기



6. ROS Programming

7) ROS launch

- *.launch file

- rosrn 이 하나의 노드를 실행하는 명령어라면, **roslaunch** 는 하나 이상의 노드를 실행 시킬 수 있음
- 패키지의 파라미터 설정, 노드 이름 변경, 노드 그룹 설정, 환경 변수 설정 등의 옵션 추가 가능
- 자주 사용되는 launch 태그

<launch>	: roslaunch 구문의 시작과 끝
<node>	: 노드 실행에 대한 태그. 패키지, 노드명, 실행명 입력 가능
<include>	: 런치 실행에 대한 태그. 다른 런치 파일을 불러올 수 있음
<remap>	: 노드에서 사용 중인 노드 이름, 토픽 이름 등의 ROS 변수의 이름을 변경
<param>	: 파라미터 이름, 타입, 값 등을 설정
<rosparam>	: rosparam 명령어와 같이 load, dump, delete 등의 옵션 사용 가능
<group>	: 실행되는 노드를 그룹화할 때 사용
<arg>	: 런치 파일 내에 변수를 정의. 파라미터를 변경할 때 사용

- Launch 실행 명령어

```
$ roslaunch [package_name] [file_name]
```

6. ROS Programming

7) ROS launch

- Example1 - talker & listener (topic)

```
$ roscd my_package
$ mkdir launch
$ cd launch
$ gedit topic.launch
```

- 아래와 같이 작성

```
<launch>
  <node pkg="my_package" type="talker.py" name="talker1" output="screen"/>
  <node pkg="my_package" type="listener.py" name="listener1" output="screen"/>
  <node pkg="my_package" type="talker.py" name="talker2" output="screen"/>
  <node pkg="my_package" type="listener.py" name="listener2" output="screen"/>
</launch>
```

- **pkg**: 패키지 이름
- **type**: 실제 실행할 노드 이름
- **name**: 필요시 노드 이름을 변경할 때 적용가능. 일반적으로는 type과 이름을 같게 설정함
- **output**: screen 옵션을 주면 해당 노드의 출력이 터미널에 표시됨

6. ROS Programming

7) ROS launch

- Example1 - talker & listener (topic)

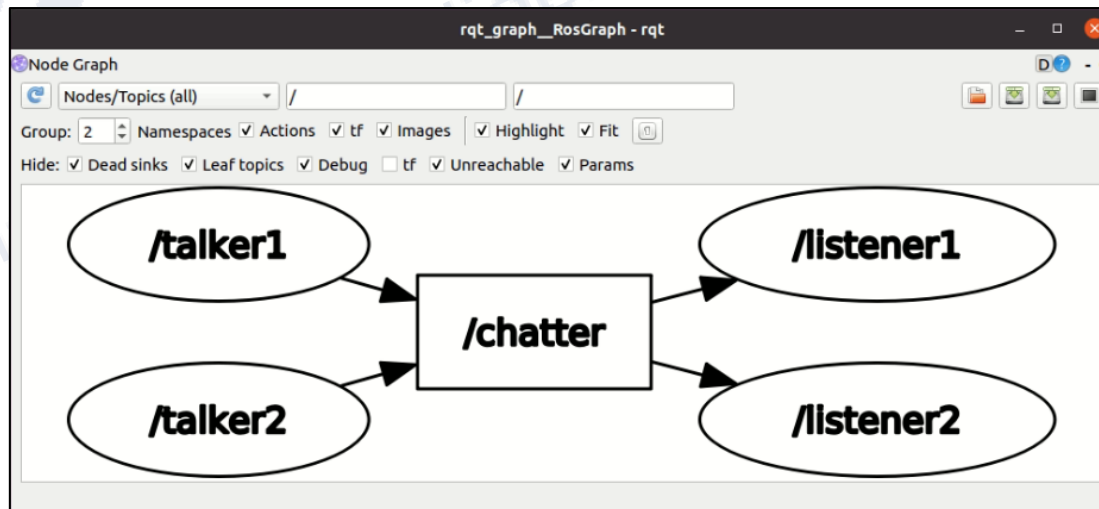
```
$ roslaunch my_package topic.launch
```

```
$ rosnode list
```

- 결과적으로 하나의 노드 파일을 두 가지 이름으로 변경하여 각각 실행할 수 있었음

```
cmln@cmln:~$ rosnode list  
/listener1  
/listener2  
/rosout  
/talker1  
/talker2
```

- 하지만 토픽명은 변경해주지 않았기 때문에 하나의 토픽을 서로 공유하고 있는 상태임

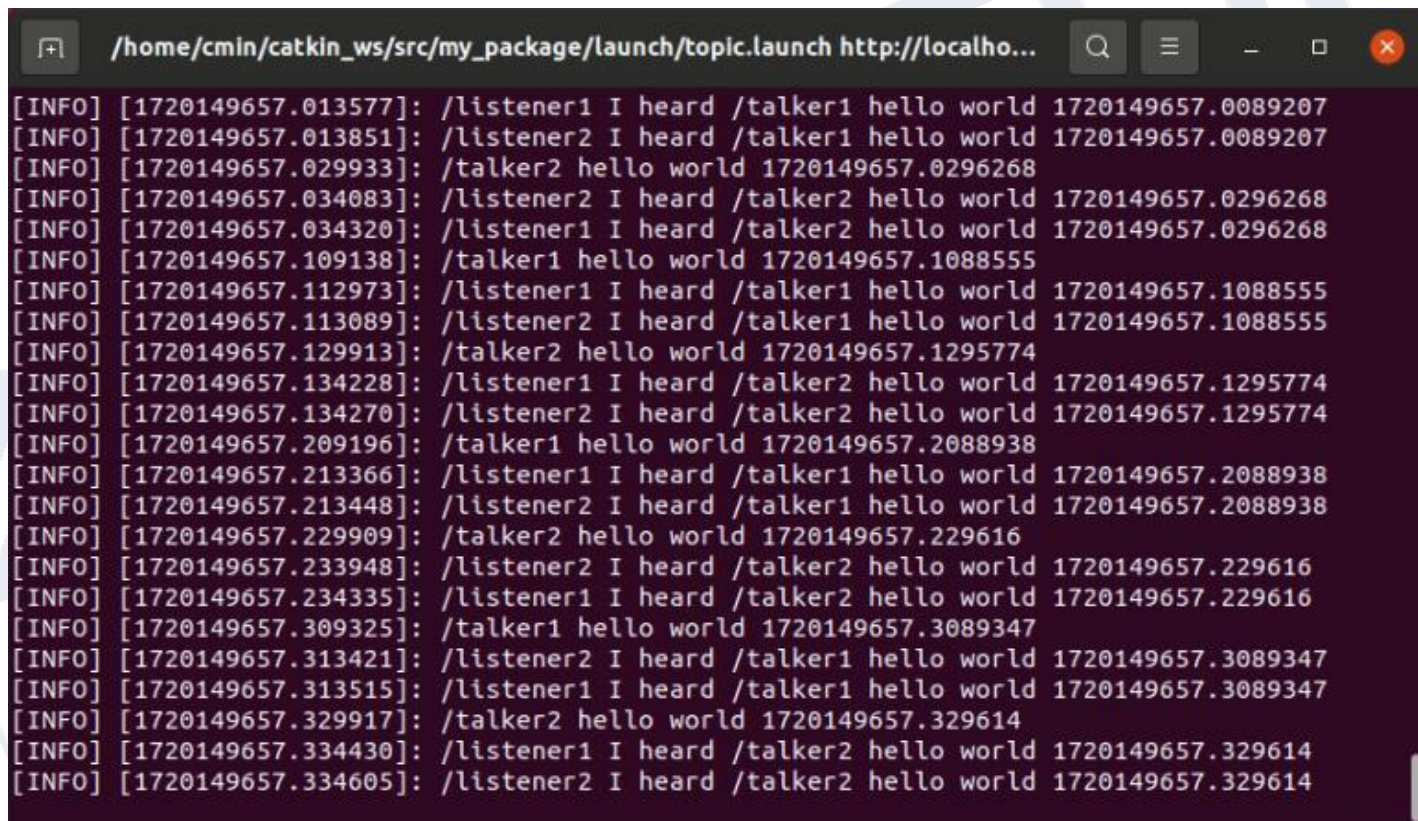


6. ROS Programming

7) ROS launch

- Example1 - talker & listener (topic)

- 따라서 listener1, listener2는 talker1, talker2의 토픽을 동시에 받고 있음



```
[INFO] [1720149657.013577]: /listener1 I heard /talker1 hello world 1720149657.0089207
[INFO] [1720149657.013851]: /listener2 I heard /talker1 hello world 1720149657.0089207
[INFO] [1720149657.029933]: /talker2 hello world 1720149657.0296268
[INFO] [1720149657.034083]: /listener2 I heard /talker2 hello world 1720149657.0296268
[INFO] [1720149657.034320]: /listener1 I heard /talker2 hello world 1720149657.0296268
[INFO] [1720149657.109138]: /talker1 hello world 1720149657.1088555
[INFO] [1720149657.112973]: /listener1 I heard /talker1 hello world 1720149657.1088555
[INFO] [1720149657.113089]: /listener2 I heard /talker1 hello world 1720149657.1088555
[INFO] [1720149657.129913]: /talker2 hello world 1720149657.1295774
[INFO] [1720149657.134228]: /listener1 I heard /talker2 hello world 1720149657.1295774
[INFO] [1720149657.134270]: /listener2 I heard /talker2 hello world 1720149657.1295774
[INFO] [1720149657.209196]: /talker1 hello world 1720149657.2088938
[INFO] [1720149657.213366]: /listener1 I heard /talker1 hello world 1720149657.2088938
[INFO] [1720149657.213448]: /listener2 I heard /talker1 hello world 1720149657.2088938
[INFO] [1720149657.229909]: /talker2 hello world 1720149657.229616
[INFO] [1720149657.233948]: /listener2 I heard /talker2 hello world 1720149657.229616
[INFO] [1720149657.234335]: /listener1 I heard /talker2 hello world 1720149657.229616
[INFO] [1720149657.309325]: /talker1 hello world 1720149657.3089347
[INFO] [1720149657.313421]: /listener2 I heard /talker1 hello world 1720149657.3089347
[INFO] [1720149657.313515]: /listener1 I heard /talker1 hello world 1720149657.3089347
[INFO] [1720149657.329917]: /talker2 hello world 1720149657.329614
[INFO] [1720149657.334430]: /listener1 I heard /talker2 hello world 1720149657.329614
[INFO] [1720149657.334605]: /listener2 I heard /talker2 hello world 1720149657.329614
```

6. ROS Programming

7) ROS launch

- Example1 - talker & listener (topic)

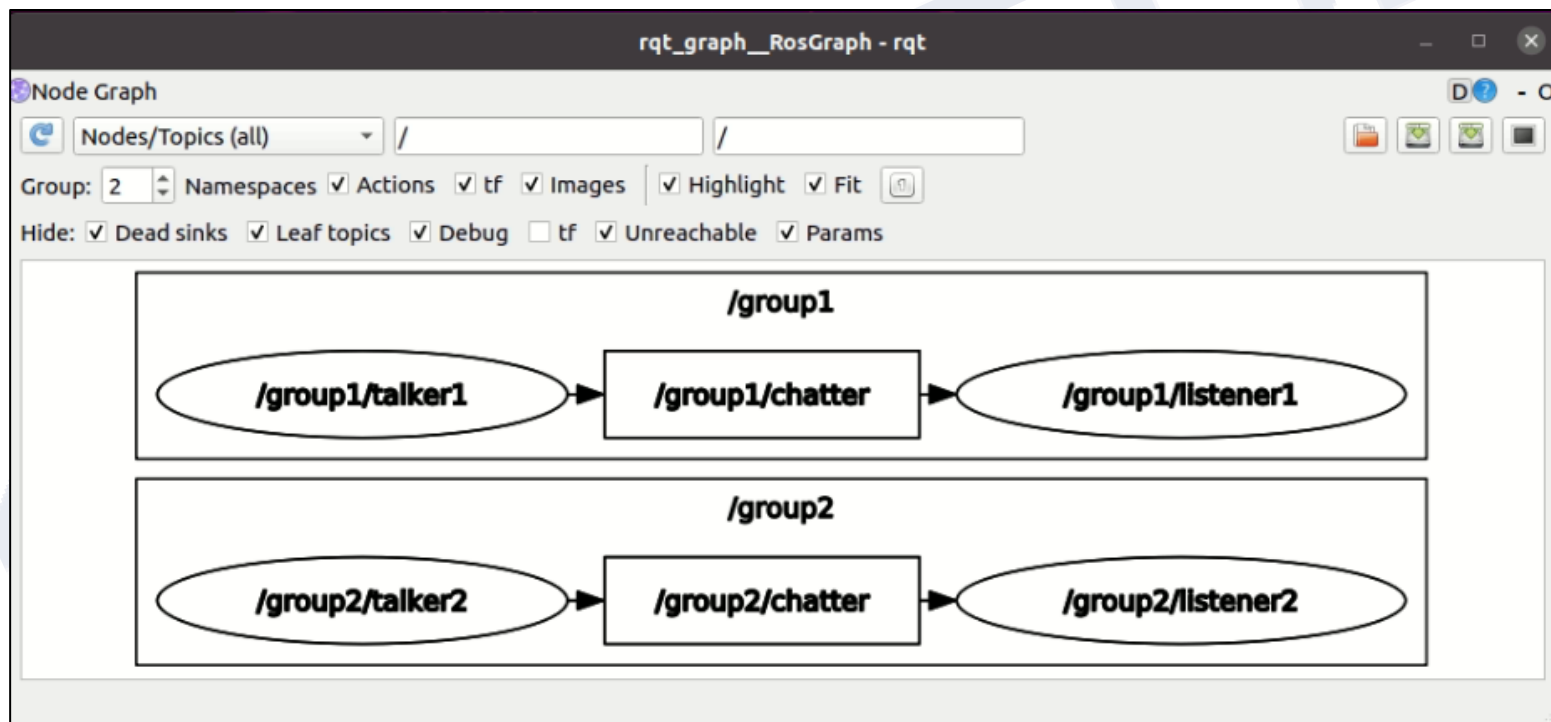
- talker1-listener1, talker2-listener2 각각 통신을 하도록 변경
- 기존 런치 파일을 아래와 같이 수정

```
<launch>
  <group ns="group1">
    <node pkg="my_package" type="talker.py" name="talker1" output="screen"/>
    <node pkg="my_package" type="listener.py" name="listener1" output="screen"/>
  </group>
  <group ns="group2">
    <node pkg="my_package" type="talker.py" name="talker2" output="screen"/>
    <node pkg="my_package" type="listener.py" name="listener2" output="screen"/>
  </group>
</launch>
```

6. ROS Programming

7) ROS launch

- Example1 - talker & listener (topic)
 - 변경 후 결과



6. ROS Programming

7) ROS launch

- Example1 - talker & listener (topic)
 - 변경 후 결과

```

/home/cmin/catkin_ws/src/my_package/launch/topic.launch http://localhost:11311
[INFO] [1720151125.506144]: /group1/listener1 I heard /group1/talker1 hello world 1720151125.5018733
[INFO] [1720151125.519700]: /group2/talker2 hello world 1720151125.5194097
[INFO] [1720151125.523718]: /group2/listener2 I heard /group2/talker2 hello world 1720151125.5194097
[INFO] [1720151125.602158]: /group1/talker1 hello world 1720151125.6018808
[INFO] [1720151125.605911]: /group1/listener1 I heard /group1/talker1 hello world 1720151125.6018808
[INFO] [1720151125.619698]: /group2/talker2 hello world 1720151125.6194074
[INFO] [1720151125.680492]: /group2/listener2 I heard /group2/talker2 hello world 1720151125.6194074
[INFO] [1720151125.702170]: /group1/talker1 hello world 1720151125.701874
[INFO] [1720151125.706058]: /group1/listener1 I heard /group1/talker1 hello world 1720151125.701874
[INFO] [1720151125.719677]: /group2/talker2 hello world 1720151125.7193854
[INFO] [1720151125.723478]: /group2/listener2 I heard /group2/talker2 hello world 1720151125.7193854
[INFO] [1720151125.802207]: /group1/talker1 hello world 1720151125.8019364
[INFO] [1720151125.805811]: /group1/listener1 I heard /group1/talker1 hello world 1720151125.8019364
[INFO] [1720151125.819730]: /group2/talker2 hello world 1720151125.819379
[INFO] [1720151125.823830]: /group2/listener2 I heard /group2/talker2 hello world 1720151125.819379
[INFO] [1720151125.902175]: /group1/talker1 hello world 1720151125.901894
[INFO] [1720151125.906006]: /group1/listener1 I heard /group1/talker1 hello world 1720151125.901894
[INFO] [1720151125.919969]: /group2/talker2 hello world 1720151125.9196208
[INFO] [1720151125.924231]: /group2/listener2 I heard /group2/talker2 hello world 1720151125.9196208
[INFO] [1720151126.002156]: /group1/talker1 hello world 1720151126.0018628
[INFO] [1720151126.006245]: /group1/listener1 I heard /group1/talker1 hello world 1720151126.0018628
[INFO] [1720151126.019854]: /group2/talker2 hello world 1720151126.0194535
[INFO] [1720151126.024002]: /group2/listener2 I heard /group2/talker2 hello world 1720151126.0194535

```

6. ROS Programming

7) ROS launch

- Example2 - turtlesim

- 목표: 움직임을 따라하는 거북이

```
$ gedit turtle.launch
```

```
<launch>
  <group ns="turtlesim1">
    <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
    <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key"
output="screen"/>
  </group>
  <group ns="turtlesim2">
    <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  </group>
  <node pkg="turtlesim" type="mimic" name="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>
</launch>
```

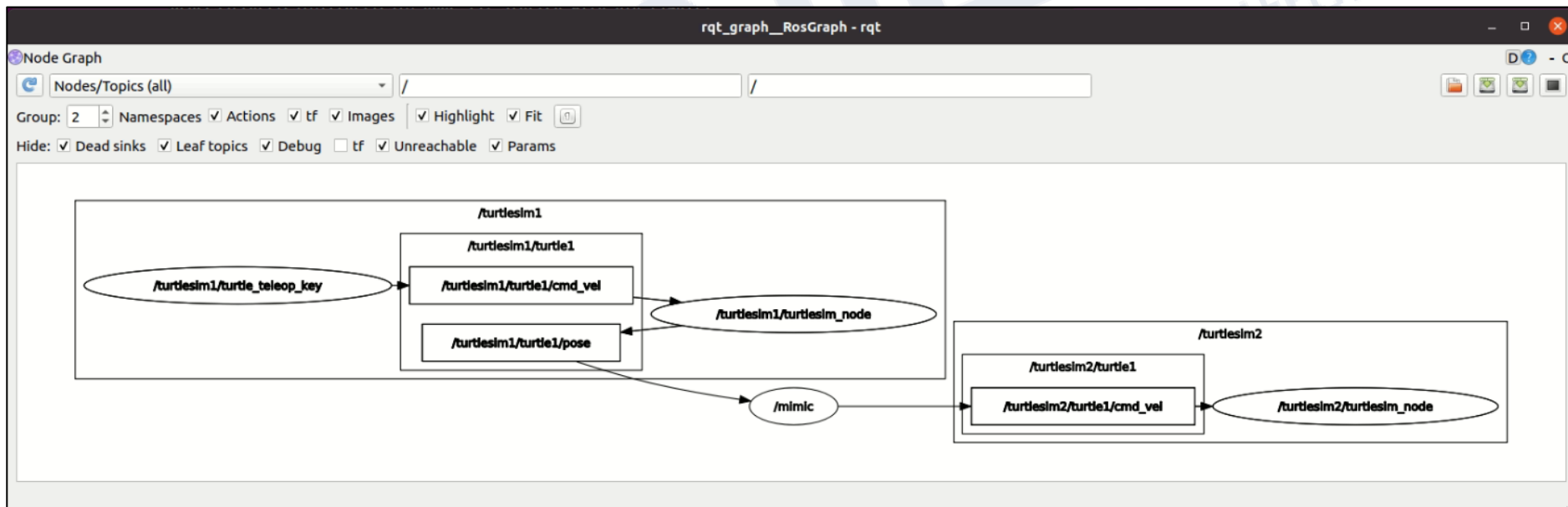
6. ROS Programming

7) ROS launch

- Example2 - turtlesim

```
$ roslaunch my_package turtle.launch
```

- turtle1의 pose를 turtle2의 속도 명령으로 매칭해주는 mimic 노드에 의해 turtle2는 turtle1의 움직임을 따라하게 됨



6. ROS Programming

7) ROS launch

- Example3 - include

- **목표:** 런치 파일 내에 하나 이상의 런치 파일을 포함할 수 있는 include 태그에 대한 예제를 작성
- 이전 turtle.launch 파일을 두 개의 파일로 나누어 생성

turtle_sub1.launch

```
<launch>
  <group ns="turtlesim1">
    <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
    <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key"
output="screen"/>
  </group>
</launch>
```

turtle_sub2.launch

```
<launch>
  <group ns="turtlesim2">
    <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  </group>
  <node pkg="turtlesim" type="mimic" name="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>
</launch>
```

6. ROS Programming

7) ROS launch

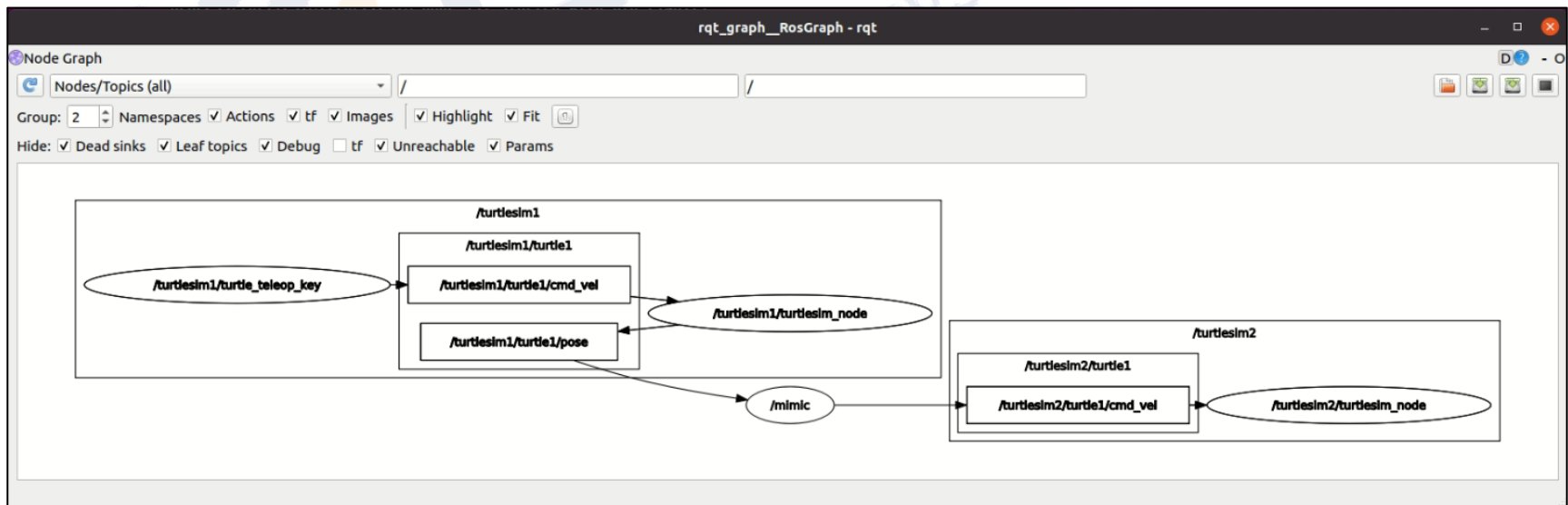
- Example3 - include

- 두 개의 런치 파일을 포함하는 메인 런치 파일을 작성

turtle_main.launch

```
<launch>
  <include file="$(find my_package)/launch/turtle_sub1.launch"/>
  <include file="$(find my_package)/launch/turtle_sub2.launch"/>
</launch>
```

- 실행시키면 예제 2와 동일한 결과를 얻을 수 있음



6. ROS Programming

7) ROS launch

- Example3 - include
 - 터미널에서 turtle1을 조작하면 turtle2가 데이터를 받고 따라 움직이는 것을 확인할 수 있음



6. ROS Programming

7) ROS launch

- 연습 문제
 - 세개의 turtlesim 노드를 켜고 동시에 각자 다른 동작을 하도록 하는 노드 및 런치 파일 작성

Appendix

[1] ROS wiki, <https://wiki.ros.org>

[2] ROS 2 Documentation, <https://docs.ros.org/en/jazzy/index.html>

[3] 표윤석 박사님 ROS 강의,

https://www.youtube.com/watch?v=ot_D9N-H4lQ&list=PLRG6WP3c31_VlFtFAxSke2NG_DumVZPgw

[4] 오로카, 오픈 로보틱스 커뮤니티, <https://cafe.naver.com/openrt>

[5] Ubuntu, <https://ubuntu.com>

- shell 명령어

명령어	중요도	명령어 풀이	세부 설명
roscd	★★★	ros+cd(changes directory)	지정한 ROS 패키지의 디렉터리로 이동
rosls	★☆☆	ros+ls(lists files)	ROS 패키지의 파일 목록 확인
rosed	★☆☆	ros+ed(editor)	ROS 패키지의 파일 편집
roscp	★☆☆	ros+cp(copies files)	ROS 패키지의 파일 복사
rospd	☆☆☆	ros+pushd	ROS 디렉터리 인덱스에 디렉터리 추가
rosd	☆☆☆	ros+directory	ROS 디렉터리 인덱스 확인

■ 실행 명령어

명령어	중요도	명령어 풀이	세부 설명
roscore	★★★★	ros+core	- master(ROS 네임 서비스) - rosout(로그 기록) - parameter server(파라미터 관리)
roslaunch	★★★★	ros+launch	노드를 여러 개 실행 및 실행 옵션 설정
rosclean	★★★☆☆	ros+clean	ROS 로그 파일을 검사하거나 삭제

▪ 정보 명령어

명령어	중요도	명령어 풀이	세부 설명
rostopic	★★★★	ros+topic	ROS 토픽 정보 확인
rosservice	★★★★	ros+service	ROS 서비스 정보 확인
roscall	★★★★	ros+node	ROS 노드 정보 확인
rosparam	★★★★	ros+param(parameter)	ROS 파라미터 정보 확인, 수정
roscat	★★★★	ros+cat	ROS 패키지 목록, 재생
rosmmsg	★★★☆☆	ros+msg	ROS 메시지 정보 확인
rossrv	★★★☆☆	ros+srv	ROS 서비스 정보 확인
rosversion	★★☆☆☆	ros+version	ROS 패키지 및 배포 릴리즈 버전 정보 확인
roswtf	☆☆☆☆	ros+wtf	ROS 시스템 검사

- catkin 명령어

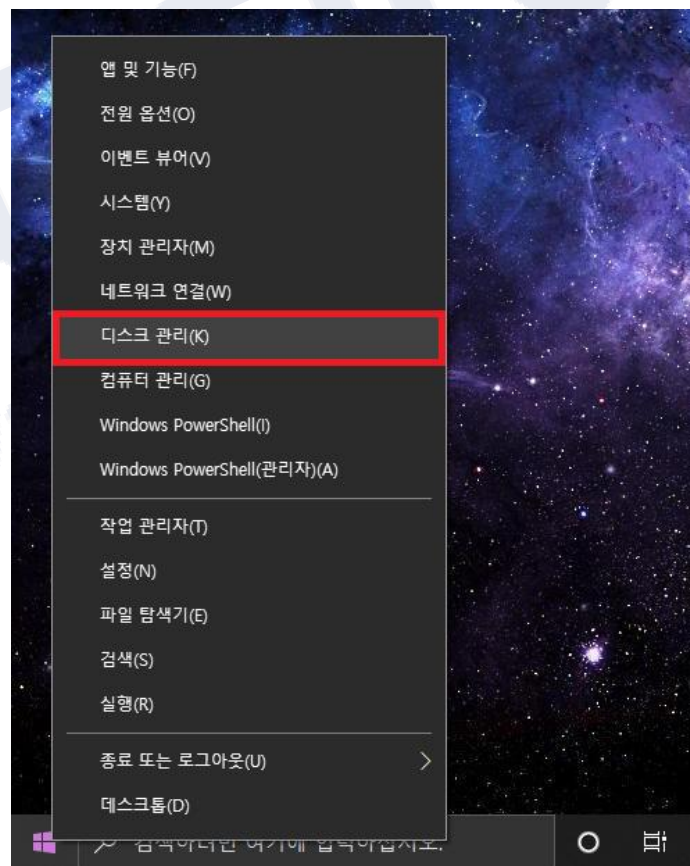
명령어	중요도	세부 설명
catkin_create_pkg	★★★	패키지 자동 생성
catkin_make	★★★	캐킨 빌드 시스템에 기반을 둔 빌드
catkin_eclipse	★★☆	캐킨 빌드 시스템으로 생성한 패키지를 이클립스에서 사용할 수 있게 변경
catkin_prepare_release	★★☆	릴리즈할 때 사용되는 로그 정리 및 버전 태깅
catkin_generate_changelog	★★☆	릴리즈할 때 CHANGELOG.rst 파일 생성 또는 업데이트
catkin_init_workspace	★★☆	캐킨 빌드 시스템의 작업 폴더 초기화
catkin_find	★☆☆	캐킨 검색

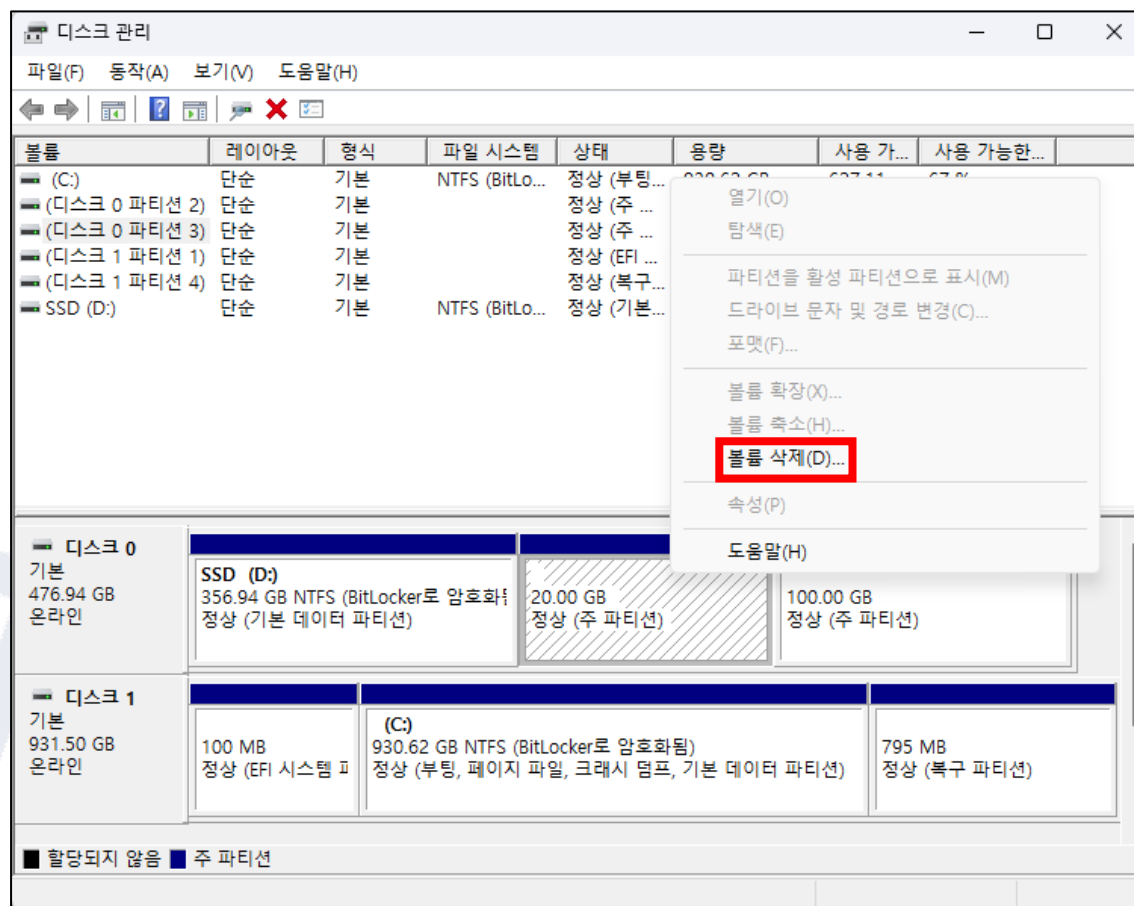
- package 명령어

명령어	중요도	명령어 풀이	세부 설명
rospack	★★★	ros+pack(age)	ROS 패키지와 관련된 정보 보기
roinstall	★★☆	ros+install	ROS 추가 패키지 설치
rosdep	★★☆	ros+dep(endencies)	해당 패키지의 의존성 파일 설치
rosllocate	☆☆☆	ros+locate	ROS 패키지 정보 관련 명령어
roscrcate-pkg	☆☆☆	ros+create-pkg	ROS 패키지 자동 생성(구 rosbuilt 시스템에서 사용)
rosmake	☆☆☆	ros+make	ROS 패키지를 빌드(구 rosbuilt 시스템에서 사용)

※ 현재 ubuntu 부팅으로 설정되어 있다면, 반드시 메인보드의 bios 세팅으로 진입해 boot option의 디스크 부팅 우선순위를 ubuntu가 아닌 windows boot manager로 변경하고 시작할 것

- Window로 부팅
- 시작메뉴를 오른쪽 클릭하여 디스크 관리 창으로 진입

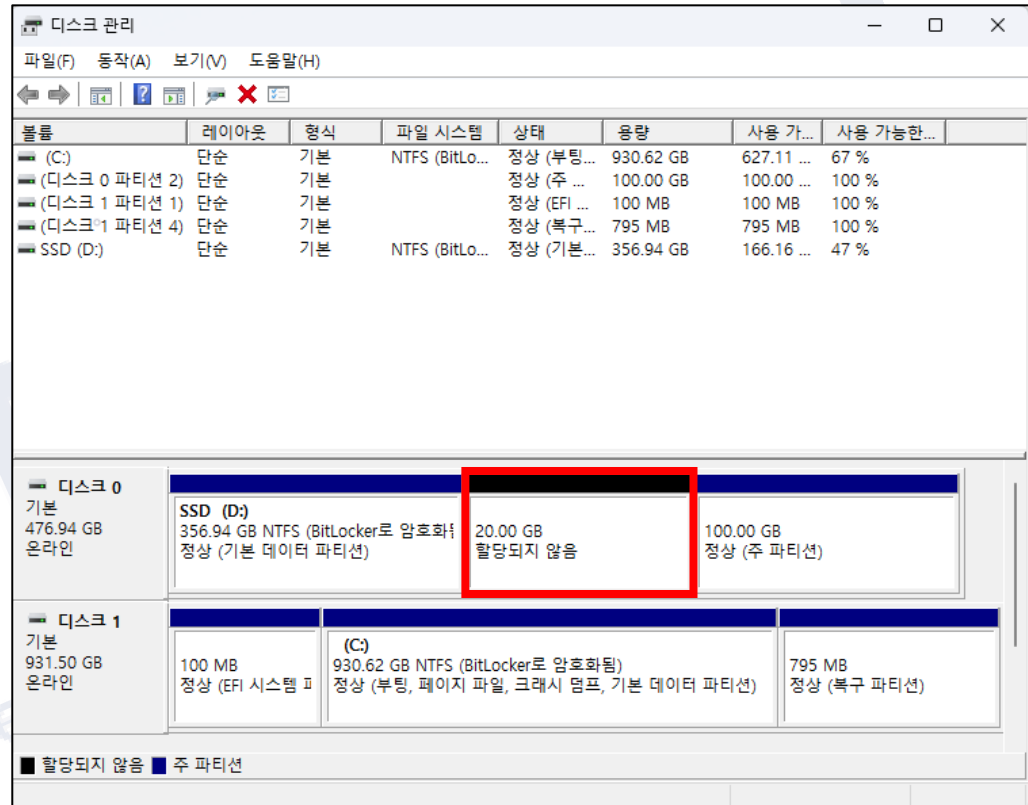
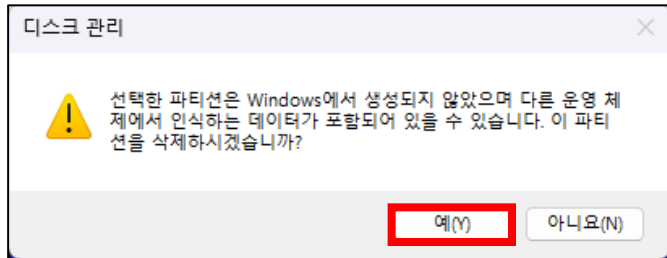




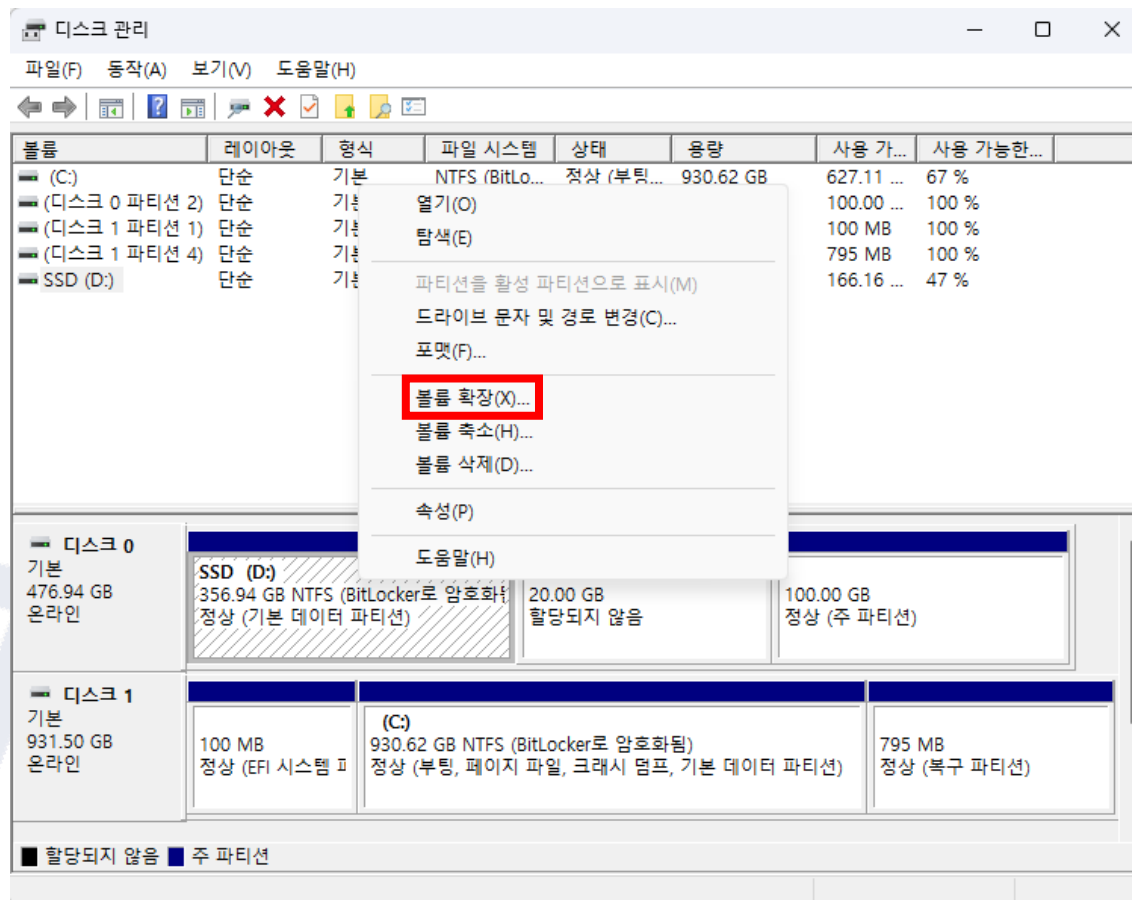
- 삭제하고자 하는 ubuntu 파티션을 오른쪽 클릭
- 볼륨 삭제 선택 (ubuntu의 모든 내용 삭제)

Appendix

Ubuntu 파티션 삭제 방법



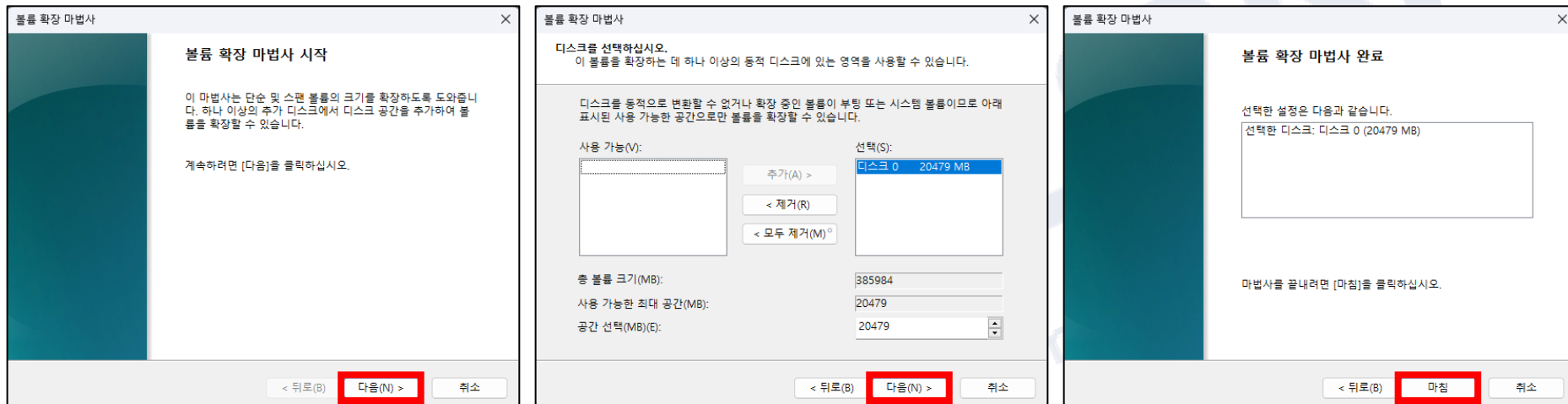
- 경고문이 나타났을 때 확인 후 **예**를 선택
- 볼륨이 삭제되면 **활당되지 않음** 상태로 변함



- 할당되지 않은 파티션을 합치고자 하는 주 파티션에 오른쪽 클릭
- 볼륨 확장 선택

Appendix

Ubuntu 파티션 삭제 방법



- 볼륨 확장 마법사 창이 나타나면 내용을 확인 후 마지막까지 다음을 눌러 진행
- 확장할 디스크의 사이즈는 자동으로 측정됨
- 마침을 누르면 볼륨 확장 완료

QnA

E-mail : cmin87394@gmail.com

Mobile : 010-3357-8739

Thank you.

